



République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique



Université des Sciences et de la Technologie Houari Boumediene

Faculté d'Informatique
Département SIQ

Mémoire de Master

Filière : Informatique

Spécialité :
Réseau Et System Distribue

Thème

Conception et mise en place d'un Cluster HPC avec des solutions Open Source

Sujet Proposé par :
Mr : NEFFAH Mohamed

Présenté par :
Behloul Oussama
Chouabi Salsabil

Soutenu le : / / 2025

Devant le jury composé de :

Mr BOUKALA MD Cherif Président
Mr BOUBENIA Mohamed Membre

Binôme N° : RSD / 25
Année Universitaire : 2024/2025

Remerciements

Tout d'abord, nous tenons à exprimer notre profonde gratitude à **ALLAH**, le Tout-Puissant, de nous avoir illuminé et ouvert les portes du savoir, ainsi que de nous avoir donné la force, le courage et la patience nécessaires à la réalisation de ce mémoire.

Nous souhaitons également exprimer nos plus vifs remerciements à notre promoteur **M. NEFFAH Mohamed**, pour son encadrement éclairé, son expertise pointue et sa guidance précieuse tout au long de ce parcours. Sa disponibilité, ses conseils avisés et son soutien indéfectible ont été d'une importance capitale pour la réalisation de ce projet. Nous avons eu l'opportunité de bénéficier de ses connaissances approfondies dans le domaine, ce qui a enrichi notre expérience académique de manière significative.

Par ailleurs, nous souhaitons exprimer notre profonde gratitude envers les membres du jury, **M. BOUKALA MD Cherif** et **M. BOUBENIA Mohamed**. Leurs évaluations attentives, Nous sommes reconnaissants de l'expertise et de l'attention qu'ils ont portées à ce projet, ce qui a contribué à son enrichissement et à son excellence.

Nous tenons à exprimer notre profonde gratitude envers **Mme. MEHDI Malika** et **M. SOKR Sif Eddine** qui nous ont guidé, inspiré et soutenu pour ce projet. Votre passion, votre dévouement et votre engagement à transmettre le savoir ont eu un impact significatif sur notre vie. Vos encouragements et vos conseils avisés nous ont permis de nous dépasser et de croire en nos capacités. Nous sommes reconnaissants pour les précieuses leçons que nous avons apprises grâce à vous. Merci infiniment pour votre influence positive et pour avoir fait une différence dans notre vie. nous tenons à exprimer notre profonde gratitude envers nos enseignants, qui nous ont appris bien plus que le programme universitaire durant ces cinq années.

Enfin, nous tenons à remercier tous ceux qui ont contribué de près ou de loin à la réalisation de ce projet, qu'il s'agisse de collègues, d'amis, de membres de notre famille ou de l'institution académique qui nous a offert cette opportunité. Leur soutien moral, leurs encouragements et leur inspiration ont été des moteurs essentiels dans l'accomplissement de cette étape majeure de notre parcours académique.

Nous tenons à souligner que ces remerciements ne suffisent pas à exprimer pleinement notre gratitude envers toutes les personnes qui ont contribué à la réussite de ce projet.

Leur engagement et leur support inébranlable ont été des facteurs déterminants dans la réalisation de ce travail.

Veuillez agréer, cher binôme, cher promoteur et membres du jury, l'expression de notre plus profonde reconnaissance et de notre respect le plus sincère.

Dédicaces

À mes parents, mes piliers inébranlables, qui m'ont soutenue et encouragée tout au long de ma vie. Votre amour, votre dévouement et vos sacrifices ont façonné la personne que je suis aujourd'hui. Cette dédicace est un témoignage de ma gratitude éternelle pour tout ce que vous avez fait et continuez de faire pour moi. Je vous aime du fond du cœur.

À mes chers frères et sœurs qui m'ont toujours soutenu et encouragé durant toutes ces années. Qu'ils trouvent ici un témoignage des liens solides qui nous unissent, en leur souhaitant un avenir prospère et florissant.

À mes collègues de la section M2 RSD, une équipe exceptionnelle qui a su dépasser les obstacles et repousser les limites.

À mes amis extraordinaires B. Rahim, B. Yacine, Z. Anis et B. Islem qui ont été mes partenaires indéfectibles tout au long de notre parcours. Vos compétences, votre soutien inconditionnel et votre camaraderie ont été essentiels pour surmonter les défis et atteindre nos objectifs communs. Cette dédicace témoigne de notre lien fort et de notre amitié précieuse. Merci d'avoir été là à chaque étape de cette belle aventure.

A ceux qui osent rêver et persévérer, qui repoussent les limites et embrassent les défis.

Oussama

Dédicaces

À ma chère mère,

Pour son amour inestimable, sa patience et ses prières qui m'accompagnent chaque jour, tes sacrifices quotidiens et ton regard rempli de fierté sont les piliers de tout ce que je suis devenue.

*Ce mémoire est le fruit de tes efforts autant que des miens.
Je t'aime au-delà des mots, et je te le dédie de tout mon cœur.*

À mon père,

Pour ta droiture, ton soutien constant et ta confiance en moi, qui m'ont toujours portée plus loin. Merci d'être là, toujours, avec calme et force.

À ma tante Wassila,

Tu as laissé un vide immense, mais ton amour reste vivant.

Ta bienveillance, ton sourire, ta chaleur humaine... tout en toi me manque.

*Tu as traversé l'épreuve avec une force et une dignité qui forcent l'admiration.
Même dans les moments les plus difficiles, tu gardais ce sourire doux et cette tendresse qui nous réconfortaient tous.*

Ton courage, ta foi, et ton amour continuent de m'inspirer chaque jour. Tu as laissé une empreinte indélébile dans ma vie.

Ce mémoire t'est dédié, avec tout mon amour.

Qu'Allah t'accorde Sa paix, Sa lumière, et le plus haut degré du paradis.

À mon oncle Allaoua,

Ton souvenir reste vivant dans mon cœur. Puisse Dieu t'envelopper de Sa lumière.

À mes cousines,

Assia, Marwa, Malek, Aya. Plus que des cousines, vous êtes mes sœurs de cœur, mes alliées dans chaque étape de la vie. Merci pour les souvenirs partagés, les rires, les confidences et les moments de réconfort.

À Maïssa, ma cousine de toujours, présente à mes côtés depuis mon plus jeune âge, avec tendresse et complicité.

À Mimi,

À chaque moment difficile, tu es là, sans que j'aie besoin de parler.

Tu crois en moi, tu me fais confiance, tu veux sincèrement mon bien... et ça, c'est rare.

Ta présence m'apaise, ton soutien me porte, et ton amitié est l'un des plus beaux cadeaux que la vie m'ait offerts. Merci d'être simplement toi.

À mes amies Manar et Meriem

Pour les délires, les confidences, le fou rire, et cette amitié vraie qui rend tout plus doux. Merci d'avoir été là, tout simplement.

À mon binôme, pour avoir partagé cette aventure à sa manière

Salsabil

I. Table De Matières

I.	Table De Matières	I
II.	Table De Figures	V
III.	Liste Des Tableaux	VI
IV.	Liste Des Abréviations	VII
	Introduction Générale	1
	Chapitre 1. Présentation du Contexte	3
1.1.	Introduction	3
1.2.	Présentation de Sonatrach	3
1.3.	Introduction à La Sismique et Aux Besoins de Calcul Intensif	4
1.3.1.	Enjeux Du Traitement Sismique Chez Sonatrach	5
1.3.2.	Besoins en calcul haute performance pour la sismique	6
1.4.	Problématique	6
1.5.	Objectifs du projet de cluster HPC	7
1.6.	Périmètre et limitations du projet	8
1.7.	Conclusion	9
	Chapitre 2. État de l'art et concepts théoriques	10
2.1.	Introduction au calcul intensif et au HPC	10
2.2.	Définition d'un cluster HPC	10
2.3.	Domaines d'application du HPC	11
2.4.	Architecture des clusters HPC	11
2.4.1.	Rôle des nœuds dans un cluster HPC	12
2.4.2.	Structure matérielle d'un système HPC	13
2.5.	Gestionnaires de tâches en environnement HPC	15
2.5.1.	Rôle et importance des gestionnaires de tâches	15
2.5.2.	Exemples de gestionnaires de tâches	16
2.5.3.	Fonctionnalités clés des gestionnaires de jobs modernes	17
2.6.	Stockage haute performance	17
2.6.1.	Objectifs du stockage HPC	18
2.6.2.	Architecture du stockage HPC	18
2.7.	Modèles de calcul dans un cluster HPC	19
2.7.1.	Le Calcul parallèle	19
2.7.2.	Méthodes de parallélisation	20
2.7.3.	Exemples simplifiés de traitements parallèles	22
2.8.	Données sismiques et contraintes applicatives chez Sonatrach	22
2.8.1.	Le traitement des données sismiques	22
2.8.2.	Acquisition terrestre	24
2.8.3.	Méthodes de traitement sismique et infrastructures HPC	25
2.8.4.	Outils et logiciels sismiques	25
2.8.5.	Optimisation et intelligence artificielle dans le traitement sismique	25

2.9. Études de cas existantes.....	25
2.9.1. Utilisation de ProMAX et de Geovation à l'ENAGEO (cas concret)	26
2.9.2. Applications internes chez Sonatrach	26
2.10. Conclusion	27
Chapitre 3. Conception de l'architecture HPC.....	28
3.1. Introduction	28
3.2. Architecture matérielle du cluster HPC	28
3.2.1. Description des serveurs utilisés	28
3.2.2. Réseau et interconnexions	28
3.2.3. Stockage partagé	29
3.3. Architecture logique du cluster HPC.....	30
3.3.1. Rôles des nœuds.....	30
3.3.2. Communication et intégration des services	31
3.3.3. L'architecture logique du cluster	31
3.4. Environnement technologique	31
3.4.1. AlmaLinux comme base système	31
3.4.2. SLURM comme gestionnaire de ressources	32
3.4.3. Système de fichiers distribué	34
3.4.4. Open OnDemand : Interface utilisateur web	35
3.4.5. Prometheus & Grafana : Supervision	35
3.4.6. LDAP : Gestion centralisée des utilisateurs	36
3.4.7. Ansible : Automatisation du déploiement.....	37
3.5. Conception de l'interface graphique pour Seismic Unix	38
3.5.1. Objectifs de conception.....	38
3.6. Architecture fonctionnelle en couches.....	39
3.6.1. Couche d'exécution (Exécution Layer)	39
3.6.2. Couche Middleware	40
3.6.3. Couche Utilisateur (User Layer).....	41
3.7 Conclusion	43
Chapitre 4. Implémentation d'un cluster HPC.....	44
4.1. Introduction	44
4.2. Déploiement manuel initial	44
4.2.1. Configuration de base des nœuds	44
4.2.2. Mise en place du réseau et synchronisation	44
4.3. Automatisation avec Ansible	45
4.3.1. Rôles Ansible développés	45
4.4. Configuration de SLURM	45
4.4.1. Test de soumission de jobs.....	46
4.5. Déploiement de BeeGFS :	46
4.6. Supervision avec Grafana / Prometheus	47
4.7. Dashboards développés	47
4.8. Interface utilisateur avec Open OnDemand	48
4.9. Implémentation de l'interface utilisateur pour Seismic Unix	48
4.9.1. Architecture de l'interface.....	48
4.9.2. Fonctionnalités principales.....	49

4.9.3.	Technologies utilisées.....	49
4.10.	Conclusion	49
Chapitre 5.	Intégration d'intelligence artificiel.....	50
5.1.	Introduction	50
5.1.1.	Définition	50
5.1.2.	Les étapes clés pour une méthode de Machine Learning	50
5.2.	Types d'apprentissage automatique	51
5.2.1.	Apprentissage supervisé.....	51
5.2.2.	Apprentissage non supervisé	53
5.2.3.	Apprentissage par renforcement	53
5.2.4.	Algorithmes d'apprentissage supervisé	54
5.3.	Objectif du module IA	54
5.4.	Conception du module IA.....	55
5.4.1.	Récupération des logs SLURM	55
5.4.2.	Données de production issues de l'ENAGEO	56
5.4.3.	Prétraitement des données	57
5.4.4.	Sélection des variables pertinentes (features)	58
5.4.5.	Objectifs de prédiction : NCPUS et ElapsedTime.....	59
5.5.	Entraînement et sélection des modèles	61
5.5.1.	Modèles de régression utilisés	61
5.5.2.	Méthodologie d'entraînement	61
5.5.3.	Sélection automatique du meilleur modèle	62
5.5.4.	Évaluation des performances	62
5.6.	Déploiement dans l'application « Prédicteur SLURM Intelligent »	64
5.6.1.	Interface utilisateur avec Streamlit	64
5.6.2.	Processus de prédiction automatisé	64
5.6.3.	Affichage des performances du modèle.....	65
5.7.	Résultats et cas de test.....	65
5.8.	Analyse des écarts et erreurs.....	66
5.9.	Interface utilisateur : Streamlit.....	66
5.9.1.	Objectifs de Streamlit	66
5.9.2.	Caractéristiques techniques de Streamlit.....	66
5.9.3.	Avantages de Streamlit.....	67
5.9.4.	Cas d'usage typiques	67
5.10.	Déploiement du module IA via une interface Streamlit	67
5.10.1.	Prédiction interactive pour l'utilisateur	68
5.10.2.	Retour visuel et indicateurs de performance	68
5.10.3.	Limites et axes d'amélioration	68
5.11.	Conclusion	69
Chapitre 6.	Evaluation et discussion.....	70
6.1.	Introduction	70
6.2.	Cas d'étude : Traitement de données sismiques.....	70
6.3.	Conception du Workflow	71
6.4.	Protocole de Test	71
6.4.1.	Données de Test	72
6.4.2.	Infrastructure Matérielle des Tests	72

6.4.3.	Les Objectifs De Test	73
6.5.	Evaluation	73
6.5.1.	Métrique D'évaluation	73
6.5.2.	Résultats	75
6.5.3.	Estimation de La L'Accélération Théorique	76
6.5.4.	Speed-up (Accélération)	77
6.5.5.	Efficacité	78
6.5.6.	Test De HPL.....	80
6.6.	Discussion.....	81
Conclusion Générale		83
Bibliographie.....		85

II. Table De Figures

Figure 1-1 Organigramme de la macrostructure de Sonatrach	4
Figure 2-1 Architecteur D'un Cluster HPC	11
Figure 2-2 Comparaison des performances entre différentes générations de CPU et de GPU	13
Figure 2-3 le flux de travail typique	16
Figure 2-4 Les quatre classes de la taxonomie de Flynn	20
Figure 2-5 La fonction de l'architecture et du mode d'accès mémoire	21
Figure 2-6 Image 2D en contexte Sismique.....	24
Figure 2-7 traitement d'image sismique sur Geovation.....	26
Figure 3-1 Architecture Globale du Plateforme.....	42
Figure 3-2 Diagramme de séquence	43
Figure 5-1 Processus de Machine Learning.....	50
Figure 5-2 Apprentissage supervisé.....	51
Figure 5-3 Exemple de régression linéaire avec données annotées.	52
Figure 5-4 Apprentissage non supervisé.....	53
Figure 5-5 Apprentissage par renforcement.	53
Figure 5-6 Exemple de logs SLURM réels de l'utilisateur a-adjou à l'ENAGEO.....	57
Figure 5-7 Évaluation comparative des modèles de Machine Learning sur des données simulées.....	63
Figure 5-8 Comparaison des Prédictions vs Valeurs Réelles : CPU et Temps.....	65
Figure 6-1 Image sismique avant et après migration	70
Figure 6-2 Analyse des performances : Calcul, Communication et Inactivité	74
Figure 6-3 Résultat de Speed Up Théorique sur les différentes configurations	77
Figure 6-4 Le Speedup real vs Speedup théorique	78
Figure 6-5 L'efficacité des différentes configurations.....	79
Figure 6-6 HPL Benchmark Résultat.....	81

III. Liste Des Tableaux

Table 2-1 Comparaison entre principales solutions de stockage HPC	14
Table 2-2 Comparaison entre Ethernet et Infiniband	15
Table 4-1 Liste des Rôles Créés	45
Table 4-2 Fonctionnalités des composants BeeGFS	46
Table 4-3 Métriques de supervision dans Grafana	47
Table 5-1 Les colonnes extraites via Sacct.....	56
Table 5-2 Description des variables d'entrée pour les jobs SLURM	59
Table 5-3 Tables De Métriques	62
Table 6-1 Matérielle de Test.....	73
Table 6-2-Temps d'exécution.....	75
Table 6-3 Résultat de Temps Parallèle	77
Table 6-4 les accélérations expérimentales aux accélérations	77

IV. Liste Des Abréviations

Abréviation	Signification
HPC	High Performance Computing (Calcul haute performance)
SLURM	Simple Linux Utility for Resource Management
CPU	Central Processing Unit
RAM	Random Access Memory
IA	Intelligence Artificielle
ML	Machine Learning (Apprentissage automatique)
XGBoost	eXtreme Gradient Boosting
SVM	Support Vector Machine
RF	Random Forest
UI	User Interface (Interface utilisateur)
CLI	Command Line Interface
SSH	Secure Shell
LDAP	Lightweight Directory Access Protocol
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
CSV	Comma-Separated Values
JSON	JavaScript Object Notation
YAML	Yet Another Markup Language
API	Application Programming Interface
NFS	Network File System
VM	Virtual Machine
GPU	Graphics Processing Unit
ETA	Estimated Time of Arrival
DSL	Domain Specific Language
PFE	Projet de Fin d'Études
OSS	Open Source Software
ENAGEO	Entreprise Nationale de Géophysique
SGWS	Scientific Workflow Management System
I/O	Input / Output (Entrée / Sortie)
UAT	User Acceptance Testing
SGY	SEG-Y (Standard des données sismiques)
FFT	Fast Fourier Transform
MPI	Message Passing Interface
SMB	Server Message Block
OOD	Open OnDemand
BEEGFS	Balanced Efficient Extensible Global File System
UI/UX	User Interface / User Experience

Abréviation	Signification
TCO	Total Cost of Ownership
R²	Coefficient de détermination (R carré)
MAE	Mean Absolute Error (Erreur absolue moyenne)
MSE	Mean Squared Error (Erreur quadratique moyenne)
RMSE	Root Mean Squared Error (Racine de l'erreur quadratique moyenne)
AUC	Area Under Curve
ROC	Receiver Operating Characteristic
NCPUS	Number of CPUs (Nombre de processeurs)
SGD	Stochastic Gradient Descent
EDA	Exploratory Data Analysis
TCO	Total Cost of Ownership

Introduction Générale

Le calcul haute performance (ou **High Performance Computing – HPC**) désigne l'ensemble des techniques, architectures et outils permettant de traiter des calculs intensifs ou de très grands volumes de données dans des délais réduits. Il s'agit de faire collaborer un grand nombre de ressources de calcul (processeurs, mémoire, stockage) pour résoudre rapidement des problèmes complexes que ne pourrait pas traiter un ordinateur classique dans des temps acceptables.

L'émergence du HPC a bouleversé de nombreux domaines scientifiques et industriels. Il est aujourd'hui un pilier incontournable dans des secteurs tels que la recherche scientifique, l'ingénierie pétrolière, la météorologie, la bio-informatique, la finance, ou encore l'intelligence artificielle. Les applications typiques vont de la simulation de fluides, à la modélisation moléculaire, en passant par l'analyse de données massives ou l'entraînement de modèles d'apprentissage automatique.

Techniquement, les systèmes HPC reposent généralement sur des clusters de calcul, c'est-à-dire un ensemble de serveurs (ou nœuds) interconnectés et pilotés par un système d'ordonnancement. Ces systèmes permettent la répartition intelligente des tâches, la gestion des files d'attente, et l'optimisation de l'utilisation des ressources matérielles.

L'un des défis majeurs en HPC réside dans la capacité à exploiter efficacement ces ressources, en assurant à la fois performance, fiabilité, accessibilité et supervision. À mesure que les architectures deviennent plus complexes, le besoin de solutions automatisées et intelligentes devient essentiel.

Dans ce contexte, l'intégration de technologies modernes telles que l'automatisation, la virtualisation d'accès, la surveillance en temps réel, ainsi que l'intelligence artificielle pour la prédiction des ressources, constitue une évolution naturelle et stratégique du domaine HPC.

Ce projet s'inscrit dans cette dynamique, avec pour ambition de mettre en place une infrastructure HPC moderne, évolutive, et intelligente, capable de répondre aux besoins opérationnels d'une entreprise telle que **SONATRACH**, tout en adoptant des solutions open source robustes et éprouvées.

Au sein de SONATRACH, la demande en calcul intensif est croissante. Cependant, l'exploitation optimale d'un cluster HPC repose sur plusieurs défis : le déploiement initial d'un environnement stable et performant, la gestion efficace des ressources (CPU, mémoire, temps), la supervision en temps réel, et l'accessibilité pour les utilisateurs non experts.

De plus, la soumission de tâches via l'ordonnanceur repose souvent sur une estimation manuelle des ressources nécessaires, ce qui peut conduire à une sous-utilisation du cluster, à des files d'attente saturées ou à l'échec des tâches. Une solution intelligente et automatisée pour prédire les ressources nécessaires représenterait une avancée significative dans l'exploitation du cluster.

L'objectif principal du projet est de mettre en place un cluster HPC complet, automatisé et intelligent, répondant aux besoins internes de calcul intensif chez SONATRACH. Plus précisément, il s'agit de :

- Concevoir une architecture HPC fonctionnelle reposant sur trois serveurs physiques.
- Déployer et configurer des outils open source.
- Automatiser le déploiement et la configuration.
- Intégrer une **intelligence artificielle** pour prédire automatiquement le nombre de CPU et le temps d'exécution en fonction du caractéristique du job.
- Offrir une interface utilisateur simple via pour accéder à ces prédictions.
- Fournir une supervision complète du cluster en temps réel.

Structure du mémoire :

Ce mémoire est structuré comme suit :

- **Chapitre 1 : Présentation du Contexte**
Présentation de SONATRACH, des enjeux du traitement sismique, de la problématique et des objectifs du projet.
- **Chapitre 2 : État de l'art et fondement théoriques**
Exploration des notions clés du HPC, des architectures de cluster et des bases de l'intelligence artificielle appliquée.
- **Chapitre 3 : Conception de l'architecture HPC**
Conception matérielle, logicielle et fonctionnelle de la plateforme en couches, adaptée aux besoins sismiques.
- **Chapitre 4 : Implémentation du cluster HPC**
Mise en place pratique du cluster, automatisation avec Ansible, intégration des composants.
- **Chapitre 5 : Intégration de l'intelligence artificielle**
Création d'un module IA prédictif des ressources, avec interface utilisateur développée en Streamlit.
- **Chapitre 6 : Évaluation et discussion**
Tests de performance sur des traitements sismiques, analyse des résultats et discussion critique.
- **Conclusion générale et perspectives**
Synthèse du projet et propositions d'améliorations futures.

Chapitre 1. Présentation du Contexte

1.1. Introduction

Ce chapitre vise à présenter le contexte général dans lequel s'inscrit notre projet de fin d'études. Nous y abordons les fondements du calcul haute performance (**HPC**), les enjeux liés à l'optimisation des ressources dans un environnement de calcul intensif, ainsi que les besoins spécifiques de l'entreprise **SONATRACH**. Ce cadre introductif nous permettra de mieux comprendre les choix technologiques effectués et les objectifs fixés pour le projet de déploiement d'un cluster HPC intelligent et automatisé.

1.2. Présentation de Sonatrach

Sonatrach est une entreprise publique algérienne créée le 31 décembre 1963, peu après l'indépendance de l'Algérie. Elle a été fondée dans le but de gérer, développer et exploiter les ressources hydrocarbures du pays, qui représentent une richesse stratégique pour l'économie nationale. À ses débuts, Sonatrach se consacrait essentiellement au transport et à la commercialisation du pétrole brut extrait en Algérie.

Avec le temps, Sonatrach a évolué pour devenir la plus grande entreprise en Afrique dans le domaine des hydrocarbures. Elle intervient aujourd'hui dans l'ensemble de la chaîne de valeur pétrolière et gazière : exploration, production, transport, transformation et commercialisation. L'entreprise est également active dans la pétrochimie et les énergies renouvelables.

Dans les années 1970, suite à la nationalisation des hydrocarbures, Sonatrach a renforcé son rôle stratégique en prenant le contrôle total des ressources naturelles du pays. Cela a permis à l'Algérie de mieux contrôler ses revenus énergétiques et d'investir davantage dans son développement économique et social.

Sonatrach entretient de nombreuses collaborations internationales avec des compagnies pétrolières et gazières étrangères. Elle est impliquée dans de grands projets de partenariat à travers des joint-ventures et accords technologiques, notamment avec des entreprises européennes, américaines, asiatiques et africaines. Ces coopérations ont permis un transfert de savoir-faire et de technologie vers l'Algérie.

Sonatrach joue également un rôle clé dans la stratégie énergétique de l'Algérie en matière de sécurité énergétique, de transition vers une économie plus durable, et de développement de nouveaux secteurs tels que le gaz naturel liquéfié (GNL), le gaz de schiste et les énergies renouvelables.

Aujourd'hui, Sonatrach est un acteur majeur sur la scène énergétique mondiale, avec une présence dans plusieurs pays et une forte contribution aux exportations algériennes. Elle continue d'innover et de s'adapter aux enjeux géopolitiques et environnementaux afin de maintenir sa place dans un marché énergétique en constante évolution. [1]

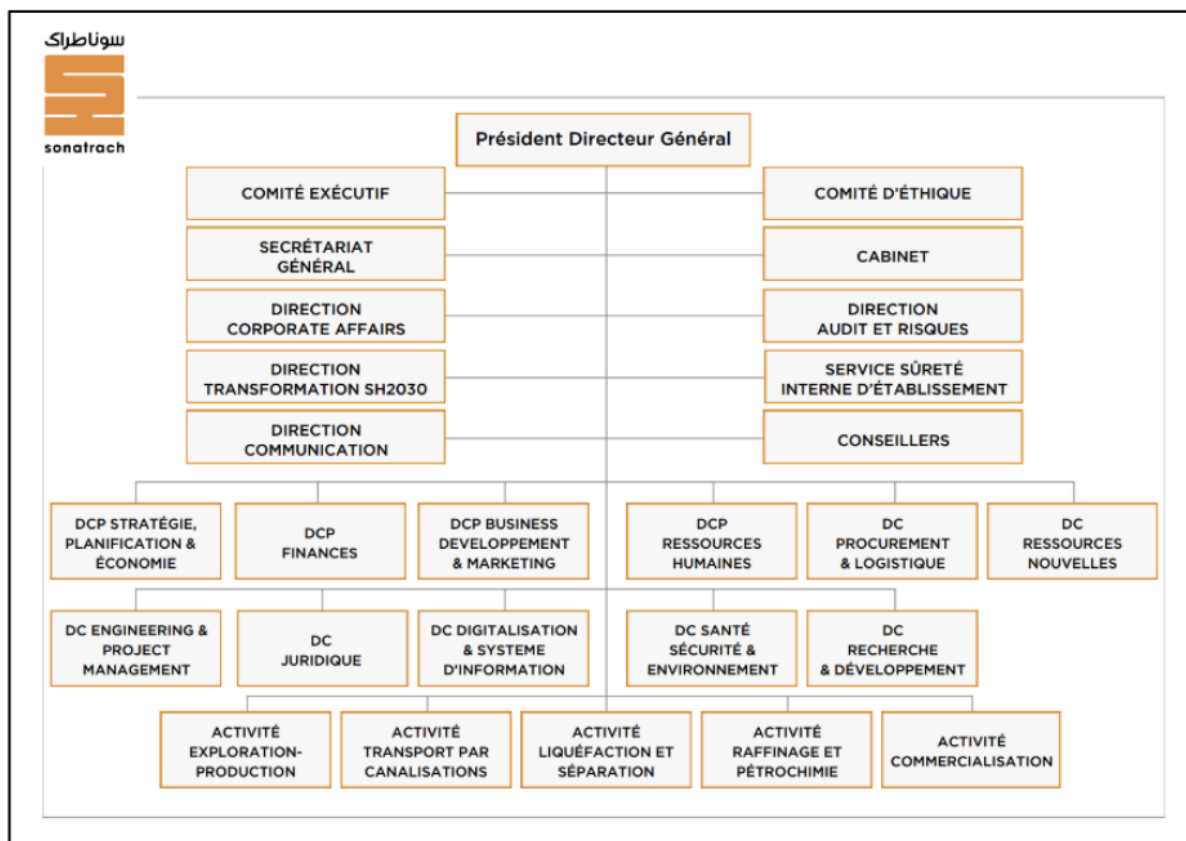


Figure 1-1 Organigramme de la macrostructure de Sonatrach

Le projet est réalisé au sein du Data Center de la Direction Centrale Recherche et Développement (DC R&D) de Sonatrach. Cette direction joue un rôle stratégique dans l'innovation technologique et l'optimisation des processus liés à l'exploration et la production. Elle dispose d'infrastructures informatiques avancées, notamment des clusters de calcul et des solutions de stockage haute capacité, destinées à soutenir les travaux de simulation et de traitement sismique. C'est dans ce cadre que s'inscrit notre projet, avec pour objectif de renforcer les capacités de calcul et d'améliorer l'efficacité du traitement sismique grâce à une architecture HPC performante.

1.3. Introduction à La Sismique et Aux Besoins de Calcul Intensif

La donnée sismique, utilisée dans l'exploration pétrolière, joue un rôle central tout au long du processus qui s'étend de l'acquisition sur le terrain jusqu'à l'interprétation des résultats, en passant par des étapes de traitement particulièrement complexes. [2]

Dès sa collecte, cette donnée nécessite des calculs intensifs, notamment lors des phases de prétraitement et de migration, afin de produire des images précises du sous-sol. Plus ces images sont fidèles à la réalité géologique, plus elles permettent d'identifier avec justesse les zones potentiellement riches en hydrocarbures. Cette recherche de précision implique l'utilisation d'algorithmes reposant sur des modèles mathématiques complexes, simulant le

comportement des ondes sismiques dans les différentes couches géologiques. Elle engendre donc une demande considérable en puissance de calcul.

Ce besoin devient encore plus marqué à l'étape de l'interprétation, où des traitements supplémentaires, tels que le calcul d'attributs sismiques (cohérence, lissage, analyse spectrale, etc.), sont appliqués pour assister les géologues dans l'analyse des données. Ces calculs sont généralement réalisés de manière itérative, chaque itération visant à ajuster les paramètres pour améliorer les résultats obtenus. Ce processus, bien qu'indispensable, peut rapidement devenir chronophage en raison de la complexité des traitements et du volume de données à analyser. Il devient donc essentiel d'exploiter efficacement les ressources de calcul disponibles afin d'accélérer les cycles d'interprétation et de permettre l'utilisation d'algorithmes de traitement plus sophistiqués. [3]

1.3.1. Enjeux Du Traitement Sismique Chez Sonatrach

Chez Sonatrach, le traitement sismique constitue une activité stratégique au cœur de l'exploration pétrolière et gazière. Compte tenu de l'immensité du territoire algérien et de la diversité géologique de ses bassins sédimentaires, les enjeux associés au traitement sismique sont multiples et cruciaux :

1. Précision des données et modélisation géophysique

L'obtention d'images sismiques précises est cruciale pour identifier les réservoirs d'hydrocarbures. Des erreurs de traitement peuvent compromettre l'exploration et les décisions d'exploitation.

2. Optimisation des ressources de calcul

Le traitement sismique nécessite une grande puissance de calcul. Sonatrach doit réduire les temps de traitement tout en maintenant la précision, en optimisant l'utilisation des ressources et en explorant les technologies de calcul accéléré comme les GPU.

3. Adaptation aux nouvelles technologies

Sonatrach doit intégrer les avancées technologiques comme l'intelligence artificielle et le machine learning, tout en assurant leur compatibilité avec les systèmes existants et en maximisant leur efficacité.

4. Gestion de données massives et stockage

Le traitement sismique génère de grandes quantités de données, nécessitant des solutions de stockage performantes et une gestion optimale des ressources pour éviter les goulots d'étranglement.

5. Réduction des coûts et amélioration de la rentabilité

Optimiser les processus de traitement et réduire le temps de calcul permettent de diminuer les coûts et d'améliorer la rentabilité des projets d'exploration.

La sécurité des opérations et la fiabilité des résultats sont essentielles, car des erreurs d'interprétation peuvent avoir de lourdes conséquences économiques et environnementales

1.3.2. Besoins en calcul haute performance pour la sismique

Les traitements sismiques modernes, de plus en plus complexes et exigeants, impliquent une manipulation intensive de données volumineuses. Ces opérations nécessitent non seulement une grande puissance de calcul, mais aussi une infrastructure capable de traiter efficacement ces données dans des délais raisonnables.

Dans ce contexte, le recours à des solutions de calcul haute performance (HPC) devient incontournable. Le HPC permet de réduire considérablement les temps de traitement tout en assurant un niveau de précision compatible avec les exigences de l'exploration géophysique.

Face à ces enjeux, il devient essentiel pour une entreprise comme Sonatrach de disposer d'une infrastructure adaptée, à la fois performante, évolutive et capable de répondre aux nouveaux défis du traitement sismique.

1.4. Problématique

Dans le contexte actuel de la transition numérique et de l'explosion des volumes de données, les entreprises et centres de recherche ont de plus en plus recours aux infrastructures de calcul haute performance (HPC) pour exécuter des simulations complexes, des traitements scientifiques intensifs ou des calculs d'ingénierie exigeants.

La problématique centrale de ce projet est donc de :

Concevoir et déployer une infrastructure HPC open source, automatisée et scalable, capable de répondre efficacement aux besoins de traitement intensif de données sismiques dans un environnement industriel.

Cependant, au-delà de l'infrastructure matérielle et logicielle, l'optimisation de l'allocation des ressources au sein d'un cluster HPC représente un défi majeur. Actuellement, les utilisateurs souvent peu familiers avec les détails techniques de l'environnement HPC doivent définir manuellement des paramètres tels que le nombre de cœurs CPU ou la durée estimée d'un job lors de sa soumission. Cette démarche, souvent approximative, peut entraîner des conséquences importantes.

Pour répondre à cette sous-problématique, nous avons proposé une approche innovante, fondée sur l'intégration de techniques **d'intelligence artificielle**, afin de prédire automatiquement les paramètres essentiels (nombre de CPU, durée d'exécution), à partir de l'analyse des historiques de jobs.

1.5. Objectifs du projet de cluster HPC

L'objectif principal de ce projet est la conception, la mise en œuvre et l'optimisation d'un cluster **HPC** basé sur des technologies open source, adapté aux besoins internes de **SONATRACH** ou d'autres institutions scientifiques, et capable de réduire les délais de traitement des applications intensives.

Ce projet ne se limite pas à une installation classique : il intègre des technologies modernes d'automatisation, d'intelligence artificielle, de supervision et de facilité d'accès, dans le but de créer un environnement performant, intelligent et accessible.

Objectifs généraux

- Concevoir une infrastructure HPC distribuée, évolutive et performante, reposant sur trois nœuds physiques.
- Utiliser des logiciels open source fiables et robustes.
- Garantir un accès centralisé, une gestion simplifiée et une haute disponibilité des ressources.

Objectifs spécifiques

1. Automatisation du déploiement :

Mettre en place une automatisation complète du déploiement des nœuds, des services et des configurations, afin de réduire les erreurs humaines, garantir l'homogénéité et faciliter la maintenance.

2. Orchestration et gestion de jobs :

Déployer le gestionnaire de ressource pour permettre une planification efficace des tâches sur les nœuds du cluster.

3. Surveillance et visualisation :

Intégrer un système de monitoring temps réel pour visualiser la performance et la santé du cluster.

4. Accès simplifié pour les utilisateurs :

Offrir une interface web conviviale permettant de soumettre, surveiller et gérer les jobs sans connaissance avancée de la ligne de commande.

5. Prédiction intelligente des ressources :

Développer une application IA qui :

- Prédit automatiquement le nombre optimal de CPU à utiliser selon le nom du job.
- Estime le temps d'exécution probable d'un job avant son lancement.
- S'appuie sur les logs historiques pour l'entraînement du modèle.

6. Stockage performant et distribué :

Utiliser le système de fichiers pour assurer un accès rapide, simultané et distribué aux fichiers des utilisateurs et des jobs.

7. Centralisation des utilisateurs et de la sécurité :

Mettre en place un serveur LDAP pour gérer de manière centralisée l'authentification et l'autorisation des utilisateurs dans le cluster.

Objectif final

Créer une solution clé en main, intelligente, automatisée et efficace, capable de démontrer comment les technologies modernes peuvent transformer l'approche classique du HPC en une solution :

- Plus **efficace** dans l'exploitation des ressources,
- Plus **prédictive** grâce à l'intelligence artificielle,
- Plus **accessible** aux utilisateurs non experts.

1.6. Périmètre et limitations du projet

Ce projet vise la mise en place d'un cluster HPC open source pour une utilisation interne, par une équipe de 1 à 10 utilisateurs spécialisés dans le traitement sismique chez Sonatrach. L'infrastructure est déployée sur un réseau local, avec des serveurs physiques appartenant à l'entreprise.

Même si les technologies utilisées sont entièrement open source et portables, le déploiement dans ce projet est limité à un usage local et sécurisé, sans accès distant ni recours au cloud. Ce choix permet de mieux protéger les données sensibles et de garder un contrôle total sur l'environnement de calcul.

Limitations du projet :

- **Ressources matérielles restreintes**
Le cluster est déployé sur un nombre limité de serveurs physiques (3 nœuds), ce qui limite la capacité de calcul globale et la scalabilité du système face à des besoins futurs plus importants.
- **Données sismiques partiellement exploitées**
Les jeux de données utilisés pour les tests sont représentatifs mais limités en volume et diversité. Par conséquent, les performances observées peuvent différer en conditions réelles avec des données plus volumineuses et variées.
- **Limitation concernant les logiciels sismiques**
Bien que le projet ait intégré l'utilisation de **SeisUnix**, un outil open source pour le traitement sismique, l'intégration complète des logiciels commerciaux majeurs tels que **ProMAX** et **Geovation** n'a pas été possible. En effet, ces logiciels nécessitent des licences spécifiques qui n'ont pas été fournies dans le cadre du projet.

- **Maintenance du cluster :**
La gestion de la maintenance à long terme (mises à jour, sauvegarde des données, gestion des pannes) n'est pas couverte par ce projet.
- **Contraintes temporelles et humaines**
Le projet a été réalisé dans un cadre temporel limité avec des ressources humaines restreintes.

1.7. Conclusion

Dans ce chapitre, nous avons exposé le contexte général du calcul haute performance (HPC), les besoins croissants en puissance de calcul dans des secteurs critiques comme l'énergie, et les motivations ayant conduit à la réalisation de ce projet. Nous avons également défini les objectifs, le périmètre et la problématique centrale autour de la gestion intelligente des ressources dans un cluster HPC. Ce cadre nous permettra d'aborder, dans les chapitres suivants, les choix techniques, la mise en œuvre et les innovations apportées.

Chapitre 2. État de l’art et concepts théoriques

2.1. Introduction au calcul intensif et au HPC

Dans un contexte de transformation numérique accélérée, les organisations scientifiques et industrielles doivent traiter des volumes de données toujours plus importants et exécuter des calculs de plus en plus complexes. Cette tendance est particulièrement marquée dans des domaines comme la recherche pétrolière, la météorologie, la modélisation physique, ou encore l’intelligence artificielle. Pour répondre à ces exigences, le recours à des infrastructures de calcul haute performance (High Performance Computing – HPC) s’est imposée comme une nécessité stratégique. [4]

Le HPC désigne l’ensemble des techniques, architectures et logiciels permettant de résoudre des problèmes nécessitant une puissance de calcul bien supérieure à celle des ordinateurs traditionnels. Ces systèmes sont capables d’exécuter simultanément des millions, voire des milliards d’opérations en un temps réduit, grâce à la parallélisation des tâches sur un grand nombre de processeurs interconnectés. [5]

Les infrastructures HPC sont composées de **clusters de serveurs** (ou nœuds de calcul), fonctionnant de manière coordonnée, où chaque nœud peut disposer de plusieurs cœurs CPU, de grandes capacités de mémoire RAM, voire de GPU spécialisés pour certains traitements massifs. L’efficacité du HPC repose ainsi non seulement sur la performance matérielle, mais aussi sur une gestion fine de l’allocation des ressources, des files d’attente et des communications entre les processus. [6]

Dans le domaine du traitement sismique, et plus généralement dans les workflows de traitement de données scientifiques, le calcul haute performance permet d’accélérer les temps de traitement, d’améliorer la précision des modèles, et de réduire les cycles d’analyse. Il devient alors un levier crucial pour optimiser la productivité, la prise de décision et la compétitivité.

2.2. Définition d’un cluster HPC

Un **cluster HPC** (High Performance Computing) désigne une infrastructure informatique regroupant plusieurs machines interconnectées, qui travaillent ensemble pour exécuter des traitements nécessitant une grande puissance de calcul. Contrairement à un ordinateur individuel, un cluster permet de répartir les tâches complexes entre plusieurs unités de traitement, ce qui le rend particulièrement adapté aux domaines où les volumes de données sont importants et les calculs intensifs. [7]

Le fonctionnement d’un cluster repose sur une coordination entre les différentes machines, appelées **nœuds**, qui coopèrent pour traiter des tâches de manière parallèle. Cette organisation permet d’accélérer le temps d’exécution des programmes et d’optimiser l’utilisation des ressources disponibles. Grâce à cette approche collaborative, le cluster devient un système puissant et efficace, capable de répondre aux besoins de calcul d’environnements scientifiques, industriels ou techniques. [8]

Un cluster HPC ne se limite pas à une simple mise en réseau d'ordinateurs. Il intègre une structure logicielle spécifique qui permet d'organiser le travail, de gérer les priorités et de répartir automatiquement les ressources entre les différents utilisateurs. Cette approche permet d'exploiter pleinement les capacités de l'infrastructure tout en assurant une certaine équité d'accès.[4]

2.3. Domaines d'application du HPC

Le calcul haute performance (HPC) constitue aujourd'hui un pilier essentiel dans de nombreux secteurs d'activité, tant industriels que scientifiques. Il permet d'exécuter des calculs complexes, de traiter des volumes massifs de données, et de simuler des phénomènes difficiles, voire impossibles, à reproduire expérimentalement.

Dans le secteur du **pétrole et du gaz**, notamment chez **Sonatrach**, le HPC est un outil stratégique pour la modélisation sismique, la simulation de réservoirs ou encore l'optimisation des opérations de forage. En **recherche scientifique**, il est largement mobilisé dans des domaines comme la physique fondamentale, l'astrophysique, le calcul quantique ou encore la modélisation climatique.

Dans le **secteur médical** et en **bio-informatique**, le HPC est utilisé pour le séquençage de l'ADN, la simulation de structures protéiques ou l'analyse des interactions moléculaires dans la mise au point de nouveaux médicaments. Les **bureaux d'études en ingénierie** exploitent ses capacités pour des calculs de mécanique des fluides, de résistance des matériaux ou encore d'aérodynamisme. Même dans le domaine de la **finance**, il est devenu incontournable pour la modélisation de risques, l'analyse prédictive et le trading à haute fréquence. [4]

2.4. Architecture des clusters HPC

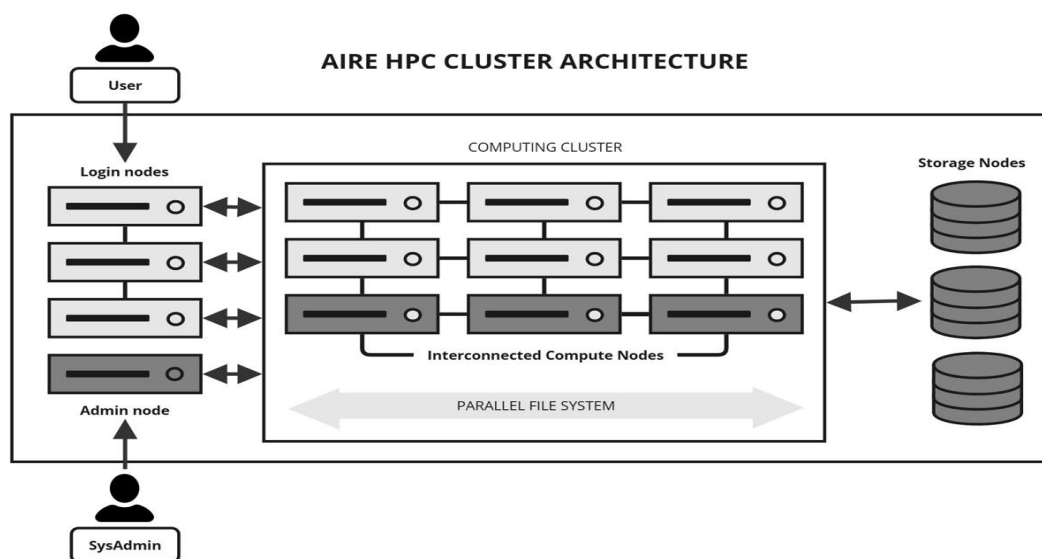


Figure 2-1 Architecteur D'un Cluster HPC

La figure 2-1 illustre une architecture typique de cluster HPC (High Performance Computing), dans laquelle les composants sont organisés de manière modulaire pour assurer performance, fiabilité et évolutivité. L'utilisateur accède au système via des nœuds de connexion (login nodes), qui servent d'interface interactive pour soumettre des jobs, compiler du code ou gérer des fichiers, sans interférer avec les calculs.

Ces demandes sont ensuite prises en charge par un nœud d'administration central, piloté par l'administrateur système, qui coordonne les ressources du cluster à l'aide d'un planificateur de tâches et d'outils de gestion. Le cœur du cluster, représenté par les nœuds de calcul, assure l'exécution parallèle des jobs. Ces nœuds, interconnectés par un réseau à haut débit, accèdent à un système de fichiers parallèle partagé, permettant une lecture et une écriture rapide des données. Ce système repose sur des nœuds de stockage capables de supporter un grand volume de données avec un accès simultané par de multiples nœuds. Une telle architecture assure une répartition claire des rôles, optimisant ainsi les performances et la gestion des ressources, tout en facilitant l'extensibilité du cluster en fonction des besoins applicatifs. [9]

2.4.1. Rôle des nœuds dans un cluster HPC

Dans un cluster HPC, les différents nœuds sont organisés selon des rôles bien définis afin d'assurer une séparation des fonctions, une meilleure gestion des ressources et une exécution efficace des tâches. Cette organisation permet de garantir la stabilité du système, d'optimiser les performances de calcul et de simplifier l'administration. On distingue généralement les types de nœuds suivants :

Nœuds de connexion (Login Nodes) :

- Servent de point d'entrée pour les utilisateurs.
- Permettent la connexion au cluster via SSH ou une interface web.
- Utilisés pour :
 - Compiler les codes.
 - Préparer les scripts de soumission.
 - Lancer les tâches via un planificateur.
 - Manipuler les fichiers.
- Ne participent **pas** aux calculs pour éviter de perturber les utilisateurs actifs.

Nœud d'administration (Admin Node ou Master Node) :

- Gère l'ensemble du cluster.
- Héberge les services critiques :
 - Planificateur de tâches.
 - Supervision et monitoring.
 - Annuaire d'utilisateurs.
 - Déploiement automatisé.
- Point central de configuration, d'audit et de gestion des mises à jour.
- Peut héberger le système de fichiers partagé.

Nœuds de calcul (Compute Nodes) :

- Exécutent les tâches de calcul soumises par les utilisateurs.
- Equipés de :
 - Processeurs multi-cœurs (CPU) ou accélérateurs (GPU).

- RAM suffisante selon les besoins applicatifs.
- Stockage local rapide.
- Connectés à un réseau rapide pour le calcul parallèle distribué.
- Peuvent être homogènes ou hétérogènes selon les charges de travail visées.

Nœuds de stockage (Storage Nodes) :

- Hébergent les données d'entrée, les résultats et le système de fichiers partagé.
- Permettent un accès simultané depuis tous les nœuds de calcul.
- Peuvent être configurés avec des systèmes parallèles (BeeGFS, Lustre) ou centralisés (NFS).
- Critiques pour la performance globale en I/O.

2.4.2. Structure matérielle d'un système HPC

Le domaine du calcul haute performance (HPC) vise à exécuter efficacement des applications sur ces environnements distribués. Cela suppose une bonne gestion de la charge de travail entre les différents nœuds, ainsi qu'une gestion optimisée des communications inter-nœuds.

Un système HPC moderne se compose généralement des cinq éléments suivants : [6]

1. Les serveurs

Chaque nœud représente un serveur doté d'un ou plusieurs processeurs connectés à la mémoire et au système de stockage. Il est également courant d'utiliser des accélérateurs tels que les GPU. Une caractéristique clé est le système de liaison entre les nœuds, qui facilite le transfert de données, la synchronisation des processus et l'utilisation partagée des ressources.

2. Les accélérateurs

Dans le cadre de diverses opérations, comme l'analyse de signaux ou la manipulation d'objets géométriques, l'emploi de pièces matérielles spécialisées, connues sous le nom d'accélérateurs, s'avère indispensable. Ils ont pour mission d'optimiser les performances de calcul tout en diminuant la consommation énergétique de l'infrastructure. [4]

● Les GPU

À l'origine destinés à la production d'images, les GPU (unités de traitement graphique) sont désormais couramment employés dans divers domaines, dont l'intelligence artificielle. Equipés de centaines de cœurs simples, ils sont particulièrement performants pour les traitements en parallèle. L'utilisation de ceux-ci dans l'apprentissage profond permet une accélération remarquable de l'entraînement des modèles. On parle alors de GP-GPU (General Purpose GPU). La figure 2.2 illustre l'évolution de leurs performances par rapport aux CPU.

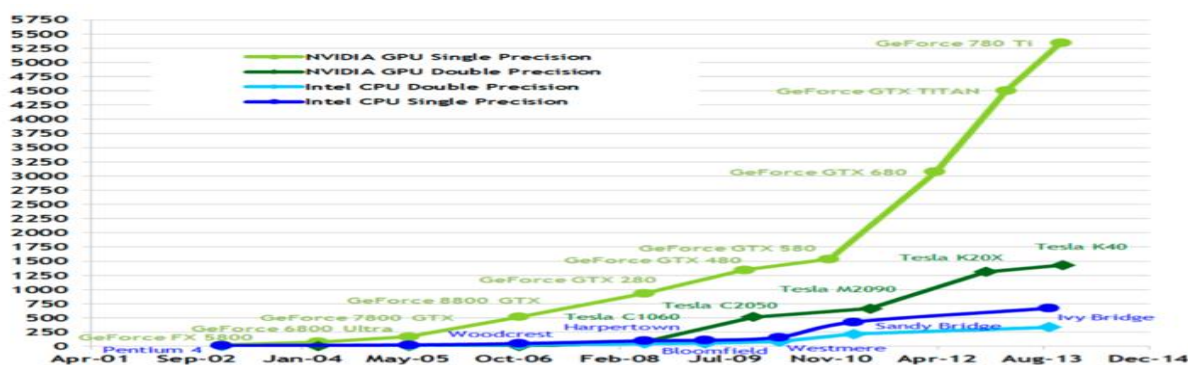


Figure 2-2 Comparaison des performances entre différentes générations de CPU et de GPU

Dans la figure 2-2 on remarque l'écart significatif séparant leur performance. Cependant, la figure représente la performance théorique des architectures et il est rare que les applications s'en approchent.

- **Les FPGA**

Les FPGA sont des circuits intégrés programmables qui offrent la possibilité de concevoir des architectures matérielles spécifiques. Malgré leur grande efficacité énergétique et la capacité à être reprogrammés pour diverses fonctions, leur élaboration s'avère complexe et se fait généralement avec des langages dédiés comme le HDL. Bien que des outils plus modernes tels qu'OpenCL existent, la complexité de la programmation et le prix élevé entravent leur utilisation généralisée. [4]

- **Les DSP**

Les processeurs de signal numérique (DSP) sont élaborés pour des tâches telles que la convolution ou le filtrage, fréquemment appliquées dans l'analyse du signal. Ils consomment moins d'énergie, fonctionnant à une fréquence réduite, mais sont optimisés pour ce genre d'opérations. En raison de leur structure Harvard (qui dispose de deux bus séparés pour les données et les instructions), ils surpassent les processeurs traditionnels en termes d'efficacité dans ce domaine. [4]

3. Le stockage

Les applications de calcul haute performance requièrent un espace de stockage considérable pour conserver les données d'entrée ainsi que les résultats générés. Quand la quantité de données excède la capacité d'un serveur, celles-ci sont déplacées vers des rangées de stockage externes. En fonction de l'application, les priorités peuvent changer entre la vitesse de transmission des données, la latence ou la fiabilité. Cependant, un système informatique efficace doit toujours reposer sur une infrastructure de stockage appropriée. [4]

Critère	Systèmes parallèles (BeeGFS, Lustre, GPFS)	Systèmes centralisés (NFS, ext4)
Accès simultané	Oui	Limité
Débit I/O	Très élevé	Faible à moyen
Scalabilité	Excellente	Très limitée
Complexité d'installation	Moyenne à élevée	Faible
Coût matériel	Variable	Faible
Utilisation typique	HPC, IA, Big Data	Petits clusters ou serveurs

Table 2-1 Comparaison entre principales solutions de stockage HPC

4. L'interconnexion

L'interconnectivité entre les serveurs, nécessaire pour le partage de données et la synchronisation des applications d'un supercalculateur, a un impact direct sur la performance générale. Des technologies telles que le 10GbE (10 Gigabit Ethernet), qui s'appuie sur le protocole TCP, permettent des vitesses de transmission atteignant jusqu'à 10 Gb/s et peuvent être déployées avec des câbles en cuivre ou en fibre optique. Même s'il est performant pour des besoins courants, l'Infiniband est généralement favorisé pour des vitesses de transfert supérieures et une fiabilité accrue. La sélection du type de connexion, qu'il s'agisse d'Ethernet ou d'Infiniband, ainsi que les protocoles et algorithmes de routage employés, est cruciale pour améliorer la performance des applications. [4] [7]

Caractéristique	Ethernet	InfiniBand
Vitesse	1 Gbps à 400 Gbps	10 Gbps à 400 Gbps
Latence	Moyenne à élevée	Faible
Bande passante	Modérée à élevée	Très élevée
Efficacité pour HPC	Moyenne	Très bonne
RDMA support	Non	Oui
Coût	Moindre	Plus élevé
Utilisation courante	Réseaux standards	HPC et supercalculateurs

Table 2-2 Comparaison entre Ethernet et Infiniband

5. Le système de refroidissement

La majeure partie de l'énergie utilisée par les dispositifs informatiques se convertit en chaleur. Un rack unique pourrait consommer plus de 50 kW, ce qui exige un système de refroidissement efficace pour prévenir tout risque de surchauffe.

Sans un refroidissement adéquat, les composants risquent de subir une diminution de leur fréquence ou même d'être détériorés. Afin d'améliorer l'efficacité thermique, diverses technologies ont été mises au point, comme le système de refroidissement par liquide ou l'immersion des pièces dans des réservoirs d'huile. [4]

2.5. Gestionnaires de tâches en environnement HPC

Dans un cluster HPC, les performances globales ne dépendent pas uniquement de la puissance matérielle, mais également de la manière dont les tâches sont organisées, planifiées et exécutées. Le bon fonctionnement de l'infrastructure repose sur des mécanismes sophistiqués de gestion des ressources, qui permettent de maximiser l'utilisation des nœuds de calcul tout en assurant une répartition équilibrée des charges de travail. Ces mécanismes sont généralement orchestrés par un **gestionnaire de tâches**, qui joue un rôle central dans le pilotage du cluster. [10]

2.5.1. Rôle et importance des gestionnaires de tâches

Le système de gestion de tâches permet :

- Planifier les jobs soumis par les utilisateurs
- Allouer dynamiquement les ressources (CPU, RAM, temps, GPU)
- Prioriser les tâches selon leur criticité ou le type d'utilisateur
- Superviser l'exécution pour garantir l'équité et l'efficacité.

Grâce à ce système, les utilisateurs n'ont pas besoin de connaître l'état de chaque nœud. Ils soumettent simplement leur tâche, et le gestionnaire se charge de son placement optimal, tout en respectant les contraintes du cluster et les politiques d'utilisation définies. [11]

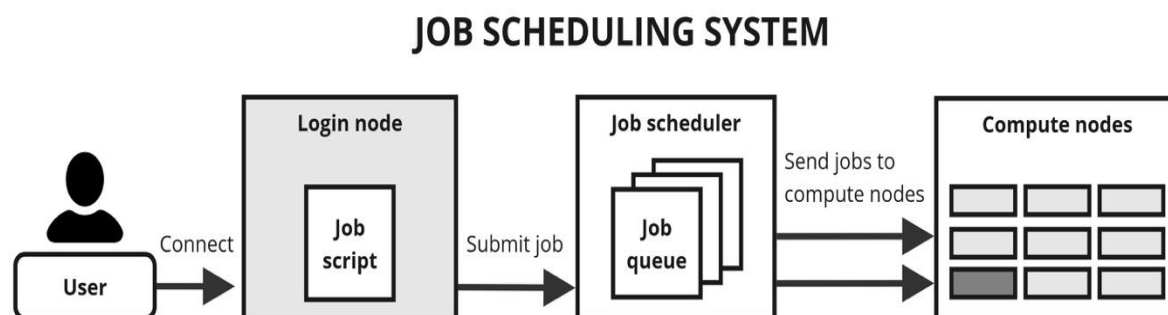


Figure 2-3 le flux de travail typique

La figure 2-3 illustre pour l'exécution d'un job dans un cluster HPC à travers un système d'ordonnancement (comme SLURM). Elle montre comment une tâche passe de l'utilisateur aux nœuds de calcul via différents composants du cluster.

2.5.2. Exemples de gestionnaires de tâches

Parmi les gestionnaires de tâches les plus utilisés dans le monde du calcul haute performance (HPC), on retrouve :

- **SLURM (Simple Linux Utility for Resource Management):**

Une solution open source extrêmement répandue dans les centres de calcul académiques et industriels. Elle offre une grande souplesse dans la définition des files d'attente, la réservation des ressources (CPU, mémoire, GPU), et l'intégration avec les systèmes de fichiers parallèles. SLURM permet également une scalabilité efficace sur des milliers de nœuds, une interface en ligne de commande riche (avec des outils comme `srun`, `sbatch`, `squeue`).

- **PBS Pro (Portable Batch System):**

Utilisé dans de nombreux environnements scientifiques et institutionnels. Il propose une interface robuste pour la gestion de jobs, avec des politiques avancées de planification (scheduling) basées sur les priorités, les quotas utilisateurs ou la consommation historique. [10] PBS Pro est particulièrement apprécié pour sa stabilité et son interopérabilité avec divers systèmes Unix/Linux, ainsi que pour sa compatibilité avec des outils de soumission distribuée.

- **IBM LSF (Load Sharing Facility):**

Une solution **commerciale** développée à l'origine par Platform Computing (rachetée par IBM), largement utilisée dans les **environnements critiques** de l'industrie, notamment dans les secteurs pharmaceutique, financier, automobile et pétrolier. [12] IBM LSF se distingue par les éléments suivants :

- **Planification avancée** : prise en charge de politiques complexes, telles que les files hiérarchiques, les priorités dynamiques, et la réservation anticipée de ressources. [13]
- **Haute disponibilité et tolérance aux pannes** : grâce à des mécanismes redondants et des fonctions de relance automatique des jobs en cas d'échec matériel.
- **Monitoring et reporting intégrés** : avec des outils comme IBM Spectrum LSF RTM, qui permet d'avoir une vision en temps réel de l'utilisation des ressources et des métriques de performance.

- **Support multi-cluster et cloud hybride** : LSF permet la fédération de plusieurs clusters, y compris dans des environnements cloud (IBM Cloud, AWS, etc.), et propose des fonctionnalités de bursting vers le cloud en cas de surcharge locale.
- **Intégration applicative poussée** : il offre des connecteurs et APIs compatibles avec les logiciels métiers (ex. : applications de CAO, de simulation numérique, d’analyse financière ou de traitement sismique). [13]

Ainsi, le choix d’un gestionnaire de tâches dépend des contraintes spécifiques du centre de calcul (budget, type de charge de travail, niveau de criticité, flexibilité souhaitée), mais tous remplissent des fonctions fondamentales : file d’attente, ordonnancement, allocation des ressources, et supervision des tâches. [9]

2.5.3. Fonctionnalités clés des gestionnaires de jobs modernes

Dans un cluster HPC, la coordination des ressources de calcul joue un rôle central dans la performance globale du système. L’ordonnancement des tâches est assuré par un gestionnaire de workload, qui prend en charge l’organisation des files d’attente, l’allocation des ressources (CPU, mémoire, temps), ainsi que la distribution des tâches sur les nœuds de calcul disponibles. Ce mécanisme permet de maximiser le taux d’utilisation du cluster tout en évitant les conflits d’accès ou les surcharges. En parallèle, des outils de supervision assurent une observation en temps réel des performances, de la charge système et de l’état des nœuds, permettant ainsi aux administrateurs et utilisateurs d’ajuster dynamiquement l’utilisation des ressources et de prévenir d’éventuelles défaillances :

- **Gestion des ressources** :
Le planificateur attribue automatiquement les ressources demandées (CPU, RAM, GPU, durée) en fonction des contraintes du job et des ressources disponibles.
- **File d’attente (Job queue)** :
Les jobs sont mis en file d’attente et exécutés selon des politiques définies (priorité, équité, partition, réservation, etc.).
- **Ordonnancement intelligent** :
Des algorithmes internes garantissent un équilibre entre performances, équité d’accès aux ressources et efficacité énergétique.
- **Monitoring en temps réel** :
Des outils comme **Grafana**, **Prometheus** ou **Node Exporter** permettent de visualiser la charge des nœuds, l’utilisation des ressources, le statut des jobs et les alertes systèmes.
- **Traçabilité et analyse** :
Les historiques d’exécution permettent d’analyser les performances, d’identifier les goulots d’étranglement et de prévoir les besoins futurs.

2.6. Stockage haute performance

Le stockage constitue un élément fondamental dans tout environnement HPC (High Performance Computing). Lorsque les performances de lecture/écriture deviennent un facteur limitant, les systèmes de fichiers classiques comme NFS ou ext4 ne suffisent plus. C’est dans ce contexte qu’interviennent les systèmes de fichiers parallèles, qui permettent d’exploiter simultanément plusieurs disques ou serveurs de stockage afin d’augmenter le débit d’I/O

(input/output). BeeGFS (anciennement FhGFS) est l'un des systèmes de fichiers parallèles open source les plus populaires, particulièrement adapté aux environnements HPC et aux charges de travail intensives en données.

2.6.1. Objectifs du stockage HPC

Dans un environnement de calcul haute performance, le système de stockage joue un rôle central dans la fluidité et l'efficacité des traitements. Il a pour objectif principal de fournir un débit élevé en lecture et en écriture, capable de soutenir les flux massifs de données générés par les applications scientifiques ou industrielles.

Un bon système de stockage doit également permettre un accès simultané aux fichiers par plusieurs nœuds de calcul, sans provoquer de congestion ni de ralentissement. Cette capacité d'accès concurrent est essentielle pour maintenir une exécution parallèle fluide.

La cohérence et l'intégrité des données doivent aussi être assurées, même lorsque les opérations sont parallélisées à grande échelle. Cela nécessite des mécanismes robustes de gestion des métadonnées et de synchronisation.

La latence, quant à elle, doit rester minimale, surtout dans le cas d'applications sensibles au temps ou interactives, où le moindre retard peut compromettre les résultats.

Un système de stockage HPC doit être conçu pour évoluer facilement. À mesure que le cluster grandit et que les volumes de données augmentent, la solution de stockage doit pouvoir s'adapter sans nécessiter de refonte complète, garantissant ainsi la pérennité de l'infrastructure.

2.6.2. Architecture du stockage HPC

L'architecture de stockage HPC repose généralement sur les composants suivants :
Systèmes de fichiers parallèles : ils permettent une lecture/écriture simultanée sur plusieurs serveurs et disques. Ils sont conçus pour fragmenter les fichiers en blocs distribués sur différents nœuds de stockage. Les plus connus sont BeeGFS, Lustre, GPFS (IBM Spectrum Scale) ou OrangeFS [13].

Nœuds de stockage :

Machines dédiées au stockage de données. Ils hébergent les fichiers réels (data nodes) et, dans certains cas, les métadonnées (nom, emplacement, permissions, etc.).

Stockage en couches (storage tiering) :

- **SSD** : utilisés pour le cache ou les données "chaudes" nécessitant un accès ultra-rapide.
- **HDD classiques** : utilisés pour le stockage massif à plus faible coût.
- **Archivage sur bandes magnétiques** : utilisé pour la conservation de long terme
-

Réseaux de stockage :

Pour maintenir les performances, les systèmes HPC utilisent souvent des réseaux spécialisés comme **Infiniband** ou **Ethernet 100 GbE**, bien plus performants que les réseaux traditionnels.

Serveurs de métadonnées (Metadata Servers) :

Ils gèrent la structure logique du système de fichiers (hiérarchie, noms de fichiers, droits, etc.), essentielle pour la performance globale.

2.7. Modèles de calcul dans un cluster HPC

Dans une infrastructure HPC, les modèles de calcul sont conçus pour exploiter au mieux les ressources distribuées afin d’exécuter des traitements intensifs dans un temps réduit. Deux approches principales sont distinguées : le **calcul parallèle** et le **calcul distribué**. Le calcul parallèle repose sur l’exécution simultanée de plusieurs parties d’un même programme sur différents cœurs ou nœuds, tandis que le calcul distribué consiste à exécuter plusieurs processus indépendants ou faiblement couplés sur des machines distinctes, souvent dans des workflows complexes.

En pratique, les tâches sont soumises via différents modes d’exécution. Le **mode batch** est le plus couramment utilisé : les jobs sont placés dans une file d’attente, puis exécutés selon les politiques de priorités et la disponibilité des ressources. À l’inverse, le **mode interactif** permet aux utilisateurs de lancer directement des calculs ou des tests depuis une session de terminal sur un nœud de calcul. Enfin, pour les traitements complexes et récurrents, les utilisateurs peuvent mettre en œuvre des workflows, c’est-à-dire des chaînes automatisées de traitements successifs, souvent décrites dans des scripts ou orchestrées via des outils comme Snakemake ou Nextflow.

2.7.1. Le Calcul parallèle

Le Calcul parallèle est une méthode d’exécution dans laquelle un problème est divisé en plusieurs sous-problèmes pouvant être traités simultanément sur des ressources distinctes. Cette approche vise à réduire considérablement le temps de traitement des applications complexes, en mobilisant plusieurs processeurs, cœurs ou accélérateurs de calcul. Elle est au cœur du fonctionnement des supercalculateurs modernes, qui combinent des milliers d’unités de traitement pour exécuter des calculs que ne pourrait réaliser un seul processeur dans un temps raisonnable. [8]

Le principe du calcul parallèle :

- Le programmeur décompose le problème principal en tâches plus petites, autonomes et parallélisables.
- Les tâches sont distribuées entre les différentes ressources du cluster (processeurs, GPU, etc.).
- Une fois exécutées, les sous-tâches produisent des résultats partiels qui sont ensuite agrégés.
- Cette exécution simultanée permet un gain de performance significatif.

Les différentes formes de parallélisme ont été regroupées dans la taxonomie de Flynn, qui distingue les architectures en fonction de l’indépendance des instructions et des données : [12]

- **SISD** (Single Instruction Single Data) : une seule instruction est appliquée à une seule donnée à la fois (architecture classique).
- **SIMD** (Single Instruction Multiple Data) : une même instruction est exécutée sur plusieurs données (architecture vectorielle).

- **MISD** (Multiple Instruction Single Data) : plusieurs instructions sont exécutées sur une même donnée. Exemple : les processeurs FMA ou certains ASIC utilisés dans les voitures connectées.
- **MIMD** (Multiple Instruction Multiple Data) : plusieurs instructions sont exécutées sur plusieurs données en parallèle. Exemple : un serveur multi-processeur.

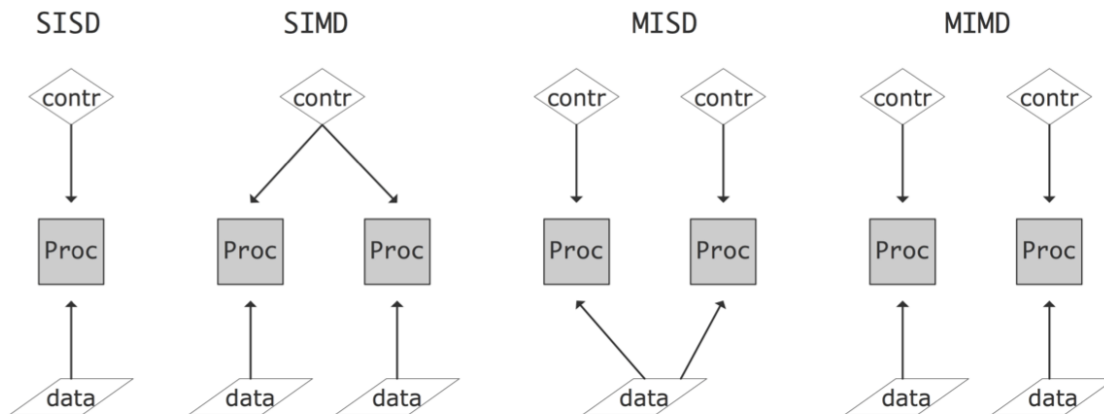


Figure 2-4 Les quatre classes de la taxonomie de Flynn

Niveaux de parallélisme dans un système HPC

Le parallélisme s'exprime à plusieurs niveaux dans une architecture HPC :

1. **Entre les nœuds** : chaque nœud du cluster exécute une tâche indépendante (MIMD).
2. **Entre-les processeurs/accélérateurs** : les ressources internes à un nœud sont exploitées.
3. **Entre les cœurs** : chaque processeur contient plusieurs cœurs pouvant exécuter des threads.
4. **Dans les pipelines** : les processeurs superscalaires traitent plusieurs instructions en parallèle.
5. **Parallélisme vectoriel** : une seule instruction traite plusieurs données (SIMD).

2.7.2. Méthodes de parallélisation

2.7.2.1. Parallélisation avec OpenMP (mémoire partagée, multi-threading)

- **OpenMP** (Open Multi-Processing) est une API permettant d'ajouter facilement du parallélisme dans des programmes séquentiels en C, C++ ou Fortran.
- Adaptée aux architectures **SMP** (Symmetric Multi-Processing) où les cœurs partagent une même mémoire.
- Il suffit d'annoter certaines boucles ou blocs de code avec des directives pour paralléliser leur exécution.
- Idéal pour les systèmes à mémoire partagée et pour les développeurs souhaitant paralléliser rapidement des portions de code.

2.7.2.2. Parallélisation avec MPI (mémoire distribuée, multiprocessus)

- **MPI** (Message Passing Interface) est une bibliothèque permettant la communication entre processus répartis sur différents nœuds.
- Adaptée aux architectures **MIMD** avec **mémoire distribuée**, typiques des clusters HPC.
- Chaque processus s'exécute indépendamment et communique avec les autres via des messages.
- L'intégration est plus complexe que OpenMP, mais elle est indispensable pour les grands clusters.

2.7.2.3. Cycle d'un processus MPI :

1. Réalisation locale des instructions
2. Échange de données avec d'autres processus
3. Synchronisation avant l'étape suivante [12]

Niveaux de parallélisme dans un supercalculateur [8]

- **Niveau 1** : *Parallélisme entre les nœuds* — Chaque serveur (ou nœud), connecté via un réseau d'interconnexion, exécute des tâches distinctes pouvant échanger des informations.
- **Niveau 2** : *Parallélisme au niveau des processeurs/accélérateurs* — Les tâches sont distribuées entre les processeurs et accélérateurs disponibles dans un même serveur.
- **Niveau 3** : *Parallélisme au niveau des cœurs* — Chaque processeur dispose de plusieurs cœurs pouvant exécuter des tâches simultanément.
- **Niveau 4** : *Parallélisme au niveau des pipelines* — Les processeurs superscalaires possèdent plusieurs pipelines d'exécution permettant un traitement simultané d'instructions.
- **Niveau 5** : *Parallélisme vectoriel* — Les unités vectorielles peuvent exécuter une instruction sur plusieurs données en parallèle (SIMD). [4]

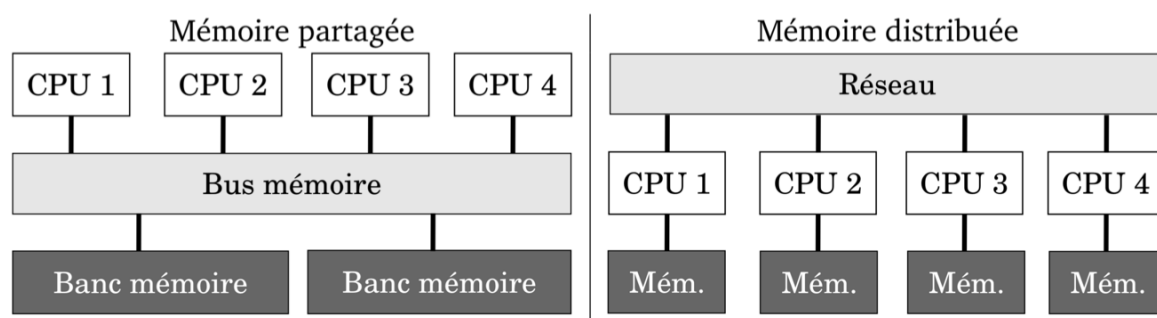


Figure 2-5 La fonction de l'architecture et du mode d'accès mémoire

2.7.3. Exemples simplifiés de traitements parallèles

Le calcul parallèle est utilisé dans de nombreux domaines pour accélérer des traitements exigeants. Voici quelques exemples concrets et simplifiés :

- **Traitement d’image**

Lorsqu’on applique un filtre à une grande image (par exemple, pour la netteté ou la détection de contours), chaque portion de l’image peut être traitée simultanément par un cœur différent du processeur. Cela permet de traiter l’image entière beaucoup plus rapidement qu’en séquentiel.

- **Analyse sismique**

Un grand volume de données sismiques peut être divisé en segments géographiques ou temporels, chacun étant analysé en parallèle par des nœuds différents. Cette approche est couramment utilisée dans l’industrie pétrolière pour réduire les délais d’interprétation.

- **Simulation numérique**

Dans les calculs de mécanique des fluides ou de prévisions météo, l’espace de simulation est divisé en zones (maillage), et chaque zone est simulée en parallèle. Cela permet de couvrir de très grandes surfaces en peu de temps.

- **Apprentissage automatique**

L’entraînement d’un modèle d’intelligence artificielle sur un grand jeu de données peut être réparti entre plusieurs GPU. Chaque processeur calcule une partie de la solution, ce qui réduit considérablement le temps d’apprentissage.

- **Recherche dans un grand ensemble de données**

Lorsqu’un programme doit chercher une information précise dans des millions de fichiers ou de lignes de texte, cette tâche peut être répartie entre plusieurs cœurs qui cherchent chacun dans une partie du corpus.

2.8. Données sismiques et contraintes applicatives chez

Sonatrach

L’analyse des données sismiques est essentielle dans les études géophysiques visant à comprendre la structure de la Terre et à rechercher des hydrocarbures. Il est nécessaire d’effectuer des calculs sophistiqués sur ces données généralement de grande taille pour obtenir des informations exactes. Les algorithmes employés, comme la migration, l’inversion ou la modélisation, nécessitent une forte capacité de traitement et un espace de stockage conséquent. Ces restrictions rendent le traitement sismique un secteur particulièrement approprié pour les environnements de calcul haute performance (HPC), où l’efficacité des ressources et la vitesse d’exécution sont cruciales pour satisfaire les exigences des professionnels de l’industrie et des chercheurs. [14]

2.8.1. Le traitement des données sismiques

La sismique réflexion est une méthode géophysique largement utilisée en exploration pétrolière. Elle permet de représenter l’image du sous-sol en exploitant les réflexions des ondes sismiques sur les différentes interfaces géologiques [14].

Cette méthode repose sur le principe de réflexion des ondes sismiques lorsqu'elles rencontrent des contrastes d'impédance acoustique (produit de la densité et de la vitesse des ondes) dans le sous-sol. Ces réflexions sont enregistrées à la surface après avoir été générées par une source sismique [15].

Processus d'acquisition :

1. Une onde sismique est générée à la surface (via une source mécanique ou explosive).
2. Elle se propage dans le sous-sol, se réfléchit sur les interfaces géologiques.
3. Les ondes réfléchies remontent et sont captées par des récepteurs (géophones ou hydrophones) [3].

Objectifs :

- Reconstituer une image 2D ou 3D du sous-sol.
- Identifier la géométrie, la stratigraphie et les propriétés élastiques des formations géologiques.

Types d'acquisition :

- **Sismique 2D** : les sources et récepteurs sont alignés, produisant un profil linéaire.
- **Sismique 3D** : les dispositifs sont répartis sur une surface, générant un cube sismique (représentation volumique) [16].

Étude de faisabilité préalable :

Avant toute acquisition, une phase de planification est nécessaire pour définir les paramètres géométriques de l'enregistrement :

1. **Offset** : Influence l'angle de réflexion, et donc la profondeur atteinte. Une gamme d'offsets bien choisie permet d'imager efficacement les structures profondes.
2. **Inter-trace** : Espacement entre deux récepteurs. Un inter-trace trop large peut nuire à l'échantillonnage, alors qu'un inter-trace trop réduit alourdit inutilement la logistique.
3. **Temps d'écoute** : Durée d'enregistrement après le tir. Il doit être suffisamment long pour capter les réflexions profondes, mais sans introduire de bruit parasite.
4. **Densité de tir** : Nombre de points de tir par unité de surface. Une densité élevée améliore la résolution, mais augmente le coût global. [3]

Facteurs à prendre en compte :

- **Relief du terrain** : Des topographies complexes peuvent nuire à la propagation des ondes et rendre l'installation difficile.
- **Accessibilité** : L'absence de routes ou de chemins peut ralentir ou empêcher les opérations.
- **Connaissances géologiques préalables** : Les données existantes (carottes, anciennes campagnes sismiques, imagerie satellite) aident à optimiser les paramètres d'acquisition.
- **Moyens logistiques** : Disponibilité du matériel, du personnel et des véhicules.
- **Contraintes économiques** : Un compromis doit souvent être trouvé entre la qualité des données souhaitée et les ressources disponibles. [3]

Types d'environnements :

- **Sismique terrestre** : Réalisée à l'aide de camions vibreurs ou d'explosifs et de géophones.
- **Sismique marine** : Utilise des sources et des récepteurs immergés. [3]

2.8.2. Acquisition terrestre

En sismique terrestre, les sources sismiques sont généralement des camions vibreurs (méthode *vibroiseis*) ou des charges explosives. Les ondes générées se propagent dans le sous-sol, se réfléchissent, puis sont captées à la surface par des capteurs appelés **géophones**.

Types de géophones :

- **Mono-composante** : Mesurent uniquement les mouvements verticaux du sol. Ils sont surtout sensibles aux ondes de compression (ondes P).
- **Multi-composantes** : Capables de mesurer le mouvement dans plusieurs directions (verticale et horizontales). Ils permettent d'exploiter les ondes de cisaillement (ondes S), en complément des ondes P, pour une meilleure caractérisation du sous-sol.

Disposition des équipements :

- En acquisition **2D ou 3D mono-azimutale**, les sources et récepteurs sont alignés.
- En acquisition **multi-azimutale**, la disposition devient plus complexe afin d'assurer une couverture spatiale optimale. Des configurations comme le "cross-spread" (lignes de sources perpendiculaires aux lignes de récepteurs) ou des grilles 3D sont alors utilisées.

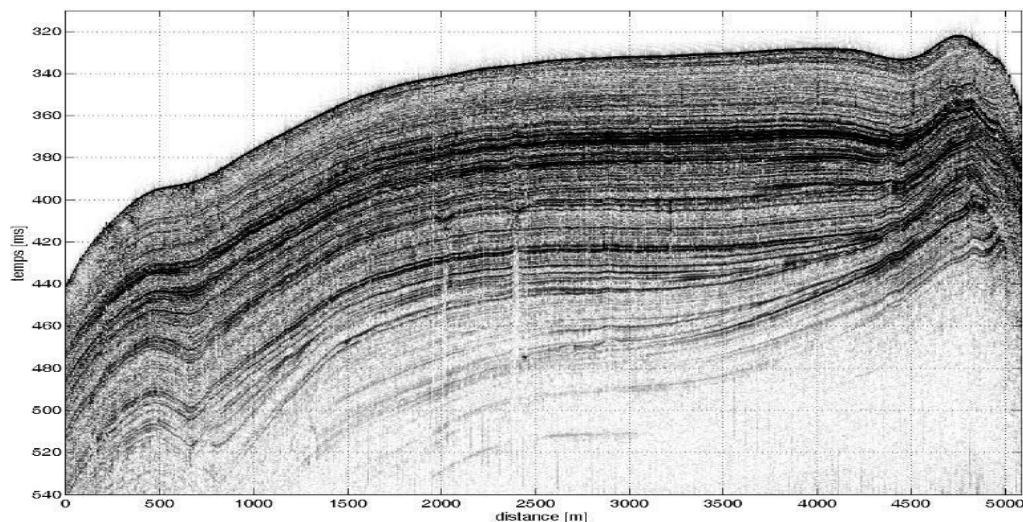


Figure 2-6 Image 2D en contexte Sismique

L'un des principaux défis en sismique terrestre réside dans la variabilité de la topographie du terrain. Lorsque des différences significatives d'altitude sont présentes, cela complique à la fois les **opérations logistiques** d'acquisition et le traitement des données. Il devient alors nécessaire d'appliquer des **corrections statiques** afin de compenser les effets liés à ces variations d'élévation, pour assurer une restitution fidèle de la géométrie du sous-sol.

2.8.3. Méthodes de traitement sismique et infrastructures HPC

On trouve les méthodes suivantes parmi les principales techniques de traitement sismique :

- **La migration sismique**, qui permet de repositionner les réflexions sismiques à leur emplacement réel dans le sous-sol, améliorant ainsi la résolution des images géologiques.
- **L'inversion sismique**, utilisée pour estimer les propriétés physiques des roches à partir des données sismiques.
- **La modélisation numérique**, qui simule la propagation des ondes sismiques dans le sous-sol pour mieux comprendre la géométrie et la composition des structures.

2.8.4. Outils et logiciels sismiques

SeisSpace/ProMAX est un logiciel propriétaire de traitement sismique développé par **Landmark**, filiale de **Halliburton**. Il s'agit de l'un des outils les plus largement utilisés dans l'industrie pétrolière et gazière pour le traitement sismique 2D, 3D et 4D. Chez Sonatrach, il constitue une brique essentielle du workflow sismique, notamment pour la prospection, l'imagerie et l'interprétation géophysique.

2.8.5. Optimisation et intelligence artificielle dans le traitement sismique

L'incorporation de l'intelligence artificielle (IA) dans le traitement sismique permet d'améliorer considérablement l'efficacité des calculs et la précision des résultats. On utilise les méthodes d'apprentissage automatique pour anticiper les ressources requises, optimiser les paramètres des algorithmes ou encore accélérer certaines phases du traitement. L'IA, associée au calcul haute performance (HPC), optimise la gestion judicieuse des ressources et permet l'automatisation de tâches complexes, contribuant de ce fait à la diminution des coûts et des délais dans le secteur sismique. [15]

2.9. Études de cas existantes

Pour évaluer la pertinence de l'architecture du cluster HPC dans le contexte de traitement des données sismiques, il est important d'examiner des études de cas existantes où des infrastructures similaires ont été mises en place. En particulier, il est essentiel de se concentrer sur des applications industrielles ou des recherches académiques qui utilisent des outils de traitement comme SeisSpace (anciennement ProMAX) ou des infrastructures HPC pour le traitement massif de données sismiques.

2.9.1. Utilisation de ProMAX et de Geovation à l'ENAGEO (cas concret)

Lors de notre visite à l'ENAGEO, nous avons observé que les équipes de Sonatrach utilisent ProMAX et Geovation pour traiter des volumes de données sismiques importants.

Ces logiciels sont installés sur des serveurs dédiés, offrant ainsi des performances optimales pour le traitement en temps réel des données.

L'infrastructure sur laquelle ces logiciels sont déployés repose également sur des clusters de calcul, permettant une gestion parallèle des tâches. Cela nous a permis de mesurer les avantages d'une telle architecture, en termes de réduction des temps de traitement et de fiabilité des résultats.

Ces outils puissants, bien que performants, restent coûteux et nécessitent des licences payantes, ce qui constitue une contrainte notable dans le cadre de projets de R&D ou d'expérimentation. C'est pour cette raison que notre projet se distingue, en optant pour une architecture HPC open source tout en utilisant ProMAX comme outil propriétaire principal pour le traitement des données sismiques.

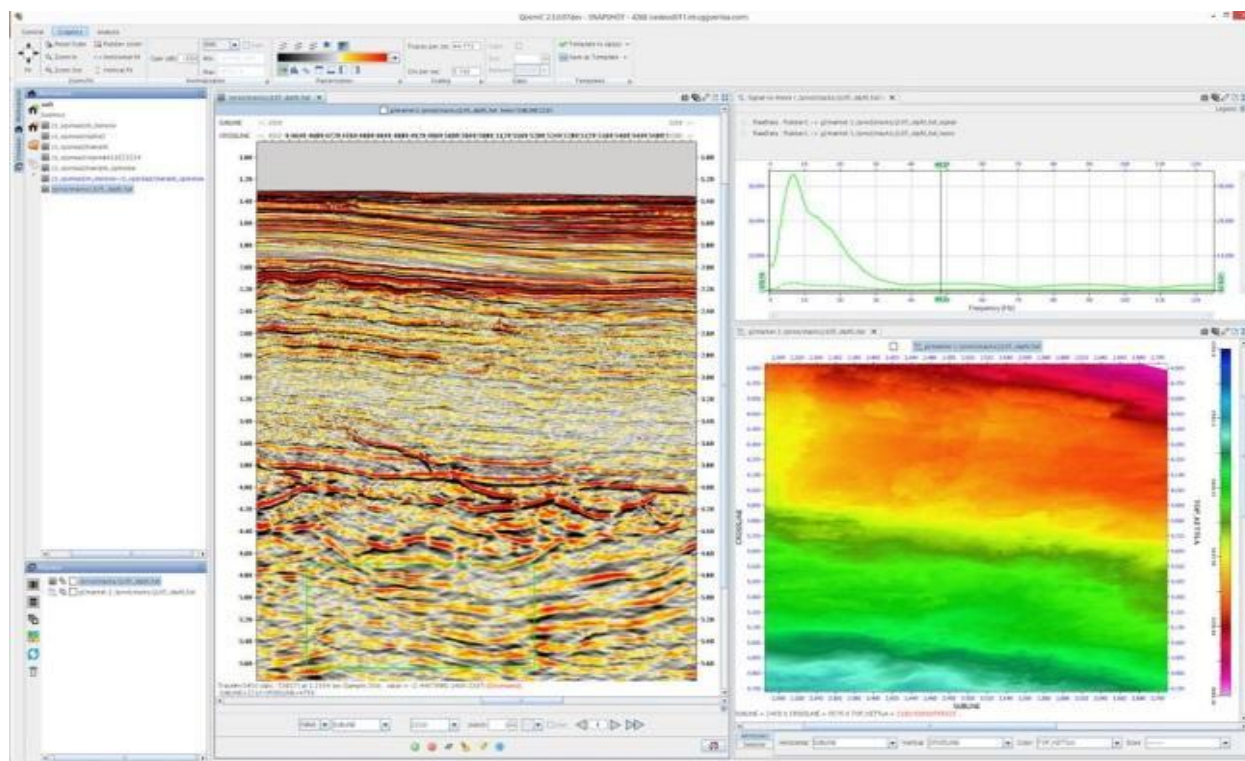


Figure 2-7 traitement d'image sismique sur Geovation

2.9.2. Applications internes chez Sonatrach

À Sonatrach, plusieurs départements, notamment ceux de la Direction Exploration, ont mis en œuvre des campagnes sismiques importantes pour étudier la structure géologique du sous-sol et surveiller les risques industriels liés aux mouvements du sous-sol. Cependant, le traitement de ces données repose principalement sur des logiciels propriétaires comme Geovation, SeisSpace et **Seismic Unix**, qui sont installés sur des serveurs spécialisés. Ces logiciels ont prouvé leur efficacité mais présentent des limitations liées à leurs coûts, leurs

licences, et leur manque de flexibilité dans un contexte d'innovation ou de développement de nouvelles méthodes de traitement.

Le projet que nous proposons se positionne donc comme une alternative stratégique à cette approche, en introduisant une solution open source avec un cluster HPC.

2.10.Conclusion

Ce chapitre a permis d’explorer les fondements théoriques du calcul haute performance (HPC) et les principales technologies qui y sont associées. Il a également mis en lumière le rôle croissant de l’intelligence artificielle dans l’optimisation des ressources HPC. Cet état de l’art constitue une base essentielle pour comprendre les choix techniques et les orientations prises lors de la conception et de la mise en œuvre du cluster, présentées dans les chapitres suivants

Chapitre 3. Conception de l'architecture HPC

3.1. Introduction

Ce chapitre présente la conception de l'architecture matérielle et logicielle du cluster HPC développé dans le cadre de ce projet. Il détaille les choix techniques effectués en matière de serveurs, de réseau, de stockage distribué et de rôles des nœuds. La structure logicielle est également expliquée, avec un focus sur les outils open source retenus pour assurer la performance, la sécurité, l'automatisation et la supervision du cluster. Cette architecture est conçue pour répondre efficacement aux besoins de calcul intensif identifiés chez SONATRACH.

3.2. Architecture matérielle du cluster HPC

Dans cette section, nous décrivons les composants physiques ayant servi à la mise en œuvre du cluster HPC, en détaillant les serveurs, le réseau, le stockage et les interconnexions entre les différentes unités.

3.2.1. Description des serveurs utilisés

Les serveurs constituant le cluster HPC déployé dans le cadre de ce projet sont au nombre de trois. Chacun d'eux joue un rôle défini dans l'architecture globale nœud de contrôle, nœuds de calcul. Ces machines physiques disposent de caractéristiques matérielles homogènes afin de garantir une exécution équilibrée des tâches parallèles. Chaque serveur est équipé de plusieurs processeurs multi-cœurs, d'une quantité de mémoire RAM suffisante pour les traitements intensifs, ainsi que d'interfaces réseau à haut débit permettant des communications rapides entre les nœuds.

Voici un aperçu des principales caractéristiques techniques des serveurs :

- **Processeurs** : processeurs multi-cœurs (ex. : Intel Xeon ou AMD EPYC), offrant de hautes performances pour les charges HPC.
- **Mémoire RAM** : de 64 à 256 Go par serveur, pour répondre aux besoins des traitements sismiques gourmands en mémoire.
- **Stockage local** : disques SSD ou NVMe pour les performances d'entrée/sortie rapides.
- **Accélérateurs** : possibilité d'intégration de GPU pour les traitements parallèles massifs ou l'intelligence artificielle.
- **Cartes réseau** : interfaces 1 GbE ou 10 GbE, connectées à un switch central, assurant une faible latence pour la communication inter-nœuds.

3.2.2. Réseau et interconnexions

Un réseau performant et bien structuré est un composant fondamental dans une infrastructure HPC, car il conditionne la rapidité des échanges entre nœuds, la réactivité du

système et l'efficacité des traitements parallèles. Dans ce projet, une **architecture réseau double** a été mise en place pour séparer les flux d'administration des flux de calcul, assurant à la fois performance et sécurité.

a) Architecture réseau

Le cluster repose sur deux réseaux physiquement séparés, chacun dédié à un usage spécifique

- **Réseau de gestion**

Utilisé pour :

- L'administration des nœuds ;
- La synchronisation système ;
- Les connexions utilisateurs ;
- Les échanges liés à la supervision.

- **Réseau de calcul**

Dédié aux communications haute performance entre nœuds, notamment pour :

- Le transfert de données massives entre jobs parallèles ;
- L'accès rapide et simultané au système de fichiers distribué ;
- La coordination efficace entre les démons SLURM (slurmctld/slurmd).

Un **switch 10 Gbit/s dédié** est mis en place pour ce réseau de calcul, garantissant une bande passante élevée et une latence réduite, deux critères essentiels dans le traitement sismique ou toute autre application HPC à forte intensité de données.

b) Protocoles et services de communication

Divers protocoles et services assurent la communication entre les composants du cluster :

- **SSH**

Utilisé pour la connexion sécurisée entre les nœuds, la gestion distante, et l'exécution des tâches Ansible.

- **SLURM**

`slurmctld` (sur le nœud maître) et `slurmd` (sur les nœuds de calcul) communiquent en continu pour gérer la soumission, l'ordonnancement et l'exécution des jobs.

- **BeeGFS**

Fournit un accès distribué, rapide et concurrent au système de fichiers partagé via le réseau 10 Gbit/s, indispensable pour les jobs manipulant de gros volumes de données.

- **Prometheus**

Collecte les métriques système via le protocole HTTP (scraping), à partir des nœuds de calcul et du nœud maître, pour une visualisation et une alerte via Grafana.

3.2.3. Stockage partagé

L'architecture de stockage HPC repose généralement sur les composants suivants :

- **Systèmes de fichiers parallèles :**

Ils permettent une lecture/écriture simultanée sur plusieurs serveurs et disques. Ils sont conçus pour fragmenter les fichiers en blocs distribués sur différents nœuds de stockage. Les plus connus sont BeeGFS, Lustre, GPFS.

- **Nœuds de stockage :**

Machines dédiées au stockage de données. Ils hébergent les fichiers réels (data Nodes) et, dans certains cas, les métadonnées (nom, emplacement, permissions, etc.). [17]

- **Stockage en couches (storage tiering) :**
 - **SSD** : utilisés pour le cache ou les données "chaudes" nécessitant un accès ultra-rapide.
 - **HDD classiques** : utilisés pour le stockage massif à plus faible coût.
 - **Archivage sur bandes magnétiques** : utilisé pour la conservation de long terme.
- **Réseaux de stockage :**

Pour maintenir les performances, les systèmes HPC utilisent souvent des réseaux spécialisés comme InfiniBand ou Ethernet 100 GbE, bien plus performants que les réseaux traditionnels.
- **Serveurs de métadonnées (Metadata Servers) :**

Ils gèrent la structure logique du système de fichiers (hiérarchie, noms de fichiers, droits, etc.), essentielle pour la performance globale. [17]

3.3. Architecture logique du cluster HPC

Après avoir présenté les aspects matériels de l'infrastructure, cette section décrit l'organisation logique du cluster, c'est-à-dire la manière dont les différents composants logiciels interagissent, les rôles attribués à chaque nœud, ainsi que la distribution des services. Cette organisation vise à garantir efficacité, performance et maintenabilité du système.

3.3.1. Rôles des nœuds

L'architecture logique repose sur une répartition fonctionnelle claire entre les trois nœuds physiques, chacun jouant un rôle spécifique dans l'écosystème du cluster.

Nœud maître (Master Node) :

Ce nœud centralise l'administration et l'orchestration du cluster. Il assure la gestion des ressources, la supervision, la sécurité et le déploiement automatisé.

Services déployés :

- **Contrôleur SLURM (slurmctld)** : planification et gestion des jobs soumis
- **Serveur LDAP** : gestion centralisée des comptes utilisateurs et authentification
- **Prometheus & Grafana** : collecte, surveillance et visualisation des métriques système et applications
- **BeeGFS Management** : gestion du système de fichiers distribué et point de montage partagé
- **Open OnDemand** : portail web d'accès utilisateur simplifié ;
- **Serveur Ansible** : automatisation du déploiement et configuration ;
- **Point de montage BeeGFS** : accès partagé au stockage.
- **Accès** : réservé aux administrateurs avec des droits restreints pour les utilisateurs

Nœuds de calcul (Compute Nodes)

Ces deux nœuds sont dédiés à l'exécution des tâches de calcul intensif, parallèles ou distribuées, gérées via SLURM.

Services déployés :

- Exécution des jobs alloués ;
- Remontée des métriques système (CPU, mémoire, réseau, etc.) ;
- Accès aux données partagées nécessaires à l'exécution des tâches.

3.3.2. Communication et intégration des services

Pour assurer une interaction harmonieuse entre les nœuds et les services, plusieurs mécanismes et protocoles sont utilisés :

- Authentification centralisée pour garantir une gestion cohérente des utilisateurs et leurs droits à travers tout le cluster.
- Accès aux données partagées qui offre un système de fichiers distribué performant, accessible de manière transparente sur tous les nœuds.
- Surveillance unifiée fournissant des métriques temps réel et historiques pour le matériel, les jobs, et les services critiques.
- Gestion et déploiement automatisés permettant une configuration cohérente et reproductible des nœuds.

3.3.3. L'architecture logique du cluster

L'architecture logique du cluster est structurée de façon modulaire et cohérente :

- Le nœud maître centralise les fonctions critiques (planification, surveillance, annuaire LDAP, portail web).
- Les nœuds de calcul se concentrent sur l'exécution des jobs de manière distribuée.
- L'ensemble repose sur une infrastructure unifiée, automatisée et supervisée avec un stockage distribué performant.

Cette organisation permet un déploiement efficace, une gestion simplifiée et une haute disponibilité pour les utilisateurs et administrateurs du cluster.

3.4. Environnement technologique

L'environnement technologique constitue le socle sur lequel repose l'ensemble de l'infrastructure HPC. Il comprend les systèmes d'exploitation, les outils de gestion des ressources, les mécanismes de supervision, les solutions de stockage, ainsi que les technologies d'automatisation et d'intelligence artificielle. Le choix de chaque composant a été guidé par des critères de stabilité, de performance, de compatibilité avec les logiciels scientifiques et de support à long terme. Parmi ces composants, le système d'exploitation AlmaLinux joue un rôle central en tant que base système pour l'ensemble des nœuds du cluster.

3.4.1. AlmaLinux comme base système

AlmaLinux a été retenu comme système d'exploitation de référence pour les serveurs du cluster HPC. Ce choix s'explique par plusieurs facteurs techniques et stratégiques : [18]

- **Remplaçant fiable de CentOS :**

Après la fin du support de CentOS en tant que distribution stable, AlmaLinux est apparu comme l'une des alternatives communautaires les plus crédibles. Entièrement binaire-compatible avec Red Hat Enterprise Linux (RHEL), AlmaLinux garantit une stabilité et

une compatibilité optimales avec les logiciels scientifiques et les outils utilisés dans les environnements HPC.

- **Support à long terme et sécurité :**

AlmaLinux est maintenu par la communauté AlmaLinux OS Foundation, avec des engagements de support de sécurité sur le long terme. Cela permet d'assurer un fonctionnement fiable et sécurisé du cluster, un élément crucial dans un contexte de production industrielle comme celui de Sonatrach.

- **Compatibilité avec les outils HPC :**

La distribution AlmaLinux offre une compatibilité native avec les outils de gestion de ressources tels que SLURM, ainsi qu'avec les systèmes de supervision (Prometheus, Grafana), les systèmes de fichiers parallèles (BeeGFS), et les outils d'automatisation (Ansible). De nombreux paquets logiciels pour le calcul scientifique et parallèle sont disponibles directement ou via EPEL (Extra Packages for Enterprise Linux).

- **Facilité de déploiement et d'administration :**

Grâce à son modèle de distribution RPM et à la disponibilité d'outils d'automatisation tels qu'Ansible, AlmaLinux permet un déploiement rapide et homogène de l'environnement système sur l'ensemble des nœuds du cluster, facilitant ainsi la maintenance et les mises à jour. [19]

3.4.2. SLURM comme gestionnaire de ressources

Dans un environnement HPC (High Performance Computing), la gestion efficace des ressources de calcul est cruciale pour optimiser l'utilisation des nœuds, réduire les temps d'attente et garantir une allocation équitable entre les utilisateurs et les projets. Le gestionnaire de ressources choisi dans ce projet est **SLURM** (Simple Linux Utility for Resource Management), une solution open source largement utilisée dans les centres de calcul scientifique et industriel à travers le monde. [9] [20]

SLURM est un ordonnanceur de tâches (job scheduler) et un gestionnaire de ressources conçu pour les environnements distribués. Il permet d'allouer dynamiquement les ressources (CPU, mémoire, GPU, etc.) aux utilisateurs, de planifier l'exécution des tâches, de surveiller leur état et de gérer les files d'attente. Il se compose de plusieurs composants modulaires, dont les principaux sont :

- **Slurmetld** : le contrôleur principal qui gère l'ensemble du cluster.
- **Slurmd** : le démon installé sur chaque nœud de calcul, chargé d'exécuter les tâches.
- **Slurmdbd** : le service de base de données pour la persistance des informations historiques des jobs.
- **Sacct, Squeue, Sinfo, Sbatch, Srun, Scancel** : des outils en ligne de commande pour interagir avec le système. [20]

Ces avantages peuvent être regroupés selon plusieurs axes :

1. Open Source, gratuit et largement adopté

SLURM est un logiciel open source sous licence GPL, ce qui permet une utilisation sans frais de licence, une personnalisation poussée du code source, ainsi qu'un large soutien

communautaire. Il est utilisé par de nombreux centres HPC dans le monde, dont plusieurs supercalculateurs du classement TOP500, ce qui témoigne de sa fiabilité et de sa maturité. [9]

2. Scalabilité et performance

SLURM est conçu pour fonctionner aussi bien sur de petits clusters que sur des supercalculateurs de plusieurs milliers de nœuds. Il maintient des performances élevées même sous de fortes charges grâce à une architecture distribuée et modulaire. Cela garantit une montée en charge fluide si le cluster de Sonatrach venait à être étendu. [21]

3. Flexibilité et modularité

SLURM offre une grande flexibilité dans la configuration des politiques de gestion des ressources :

- Files d'attente multiples avec règles personnalisées ;
- Réservation de nœuds ;
- Priorités basées sur des politiques FAIR SHARE, des comptes et des QOS (Quality of Service) ;
- Prise en charge des dépendances entre jobs.

Ces fonctionnalités permettent d'adapter précisément le système à l'organisation des workflows métiers. [21]

4. Richesse fonctionnelle

SLURM propose une large gamme de fonctionnalités avancées :

- Job arrays pour exécuter des centaines de tâches similaires avec une seule soumission
- Gestion des tâches parallèles (MPI, Open MP) avec `srun`
- Support natif des GPU
- Monitoring en temps réel et historique des jobs (`sacct`, `sstat`, `squeue`, etc.)
- Plugins pour étendre ou modifier son comportement (ordonnanceurs, accounting, logs...). [22]

5. Intégration avec d'autres outils

SLURM s'intègre naturellement avec d'autres composants utilisés dans l'infrastructure du cluster :

- **Prometheus/Grafana :**
Via des exporter comme `slurm_exporter` ou `node_exporter`, pour collecter et visualiser les métriques système et de jobs.
- **Open OnDemand :**
Permet aux utilisateurs de soumettre des jobs SLURM depuis une interface web.
- **Modules IA :**
Les logs SLURM servent de données d'entraînement à l'algorithme de prédiction (Random Forest) intégré dans l'application Streamlit.

6. Robustesse et fiabilité

Grâce à sa conception robuste, SLURM peut continuer à exécuter les jobs en cours même en cas de défaillance temporaire du contrôleur (`slurmctld`). Il prend aussi en charge des mécanismes de reprise automatique et d'archivage, ce qui le rend adapté à un contexte de production critique. [21]

3.4.3. Système de fichiers distribué

Le système de fichiers est un composant critique dans une infrastructure HPC, en particulier pour les applications sismiques qui génèrent et traitent de très grands volumes de données. Le choix d'un système de fichiers performant, évolutif et hautement disponible est donc essentiel. Dans ce projet, BeeGFS (Beyond Extensible Enterprise File System) a été retenu comme solution de stockage parallèle distribué. [17]

BeeGFS est un système de fichiers distribué développé initialement par Fraunhofer ITWM, spécifiquement conçu pour les environnements HPC et les applications nécessitant un accès rapide et simultané à de grandes quantités de données. Contrairement aux systèmes de fichiers traditionnels, BeeGFS répartit les fichiers et les métadonnées sur plusieurs serveurs, ce qui permet d'augmenter la bande passante et de réduire les goulots d'étranglement. [17]

BeeGFS repose sur une architecture modulaire composée de plusieurs services : [23]

- **Metadata Server (MDS)** : gère les opérations sur les fichiers (création, suppression, droits, etc.).
- **Storage Server (OSS)** : stocke physiquement les fichiers, découpés en blocs (chunked).
- **Client** : un module installé sur les nœuds de calcul, permettant un accès transparent au système distribué.
- **Management Server** : surveille l'état du système et coordonne les services.

Avantages de BeeGFS

BeeGFS a été choisi dans ce projet pour les raisons suivantes :

- **Haute performance** :
BeeGFS permet une agrégation de la bande passante grâce au parallélisme sur les serveurs de stockage. Les données sont distribuées en plusieurs morceaux (chunks) et peuvent être lues ou écrites simultanément par différents nœuds.
- **Scalabilité horizontale** :
Il est possible d'ajouter dynamiquement des serveurs de stockage ou de métadonnées sans interruption de service, ce qui permet une adaptation progressive aux besoins croissants en capacité et en performance.
- **Facilité d'installation et de configuration** :
Comparé à d'autres systèmes comme Lustre ou GPFS, BeeGFS est plus simple à déployer, à administrer, et dispose d'une documentation claire et d'outils de gestion conviviaux.
- **Compatibilité avec les charges HPC** :
BeeGFS est optimisé pour les accès simultanés à de nombreux fichiers, ce qui le rend parfaitement adapté aux workflows de traitement sismique.
- **Intégration avec Prometheus/Grafana** :
Le système expose des métriques exploitables pour la supervision en temps réel, ce qui s'inscrit dans la stratégie de monitoring du cluster. [17]

3.4.4. Open OnDemand : Interface utilisateur web

Dans un environnement HPC, l'accessibilité et la simplicité d'usage sont des critères importants, notamment pour les utilisateurs non spécialistes de la ligne de commande. Pour améliorer l'expérience utilisateur et faciliter l'interaction avec le cluster, le projet intègre Open OnDemand, une interface web open source dédiée aux infrastructures de calcul intensif.

Open OnDemand est un portail web développé par le Ohio Supercomputer Center (OSC). Il permet aux utilisateurs d'interagir avec un cluster HPC à travers une interface graphique intuitive, sans avoir besoin d'utiliser directement le terminal ou des connexions SSH complexes. Il s'agit d'une surcouche web qui expose les principales fonctionnalités du système HPC, tout en respectant les politiques de sécurité et de gestion de ressources déjà en place. [24]

Les fonctionnalités offertes par Open OnDemand incluent notamment :

- Explorateur de fichiers avec possibilité de transfert, d'édition, de suppression ou de compression de fichiers
- Terminal interactif web (web Shell)
- Soumission de jobs SLURM via des formulaires web personnalisables
- Visualisation des jobs en cours ou terminés

Avantages d'Open OnDemand

L'intégration d'Open OnDemand dans le cluster HPC apporte plusieurs bénéfices [24]:

- Accessibilité facilitée : les utilisateurs peuvent accéder à l'environnement de calcul via un simple navigateur web, sans installation locale de logiciels supplémentaires.
- Apprentissage accéléré : la courbe d'apprentissage pour les nouveaux utilisateurs est fortement réduite, ce qui encourage une adoption rapide de l'infrastructure.
- Support natif de SLURM : Open OnDemand est compatible avec SLURM et permet une soumission directe de jobs, la visualisation des files d'attente (*squeue*) et des ressources allouées.
- Personnalisation et extensibilité : il est possible de configurer des interfaces de soumission de jobs adaptées aux différents workflows métiers (traitement sismique, post-traitement, IA...).
- Sécurité centralisée : l'authentification peut être intégrée à un annuaire LDAP, comme dans ce projet, assurant ainsi une gestion des utilisateurs unifiée et sécurisée.

3.4.5. Prometheus & Grafana : Supervision

La supervision d'un cluster HPC est essentielle pour garantir sa disponibilité, identifier rapidement les anomalies, optimiser l'utilisation des ressources et assurer la qualité de service. Dans ce projet, la solution de supervision repose sur deux outils open source complémentaires : Prometheus pour la collecte de métriques, et Grafana pour leur visualisation graphique.

Présentation de Prometheus

Prometheus est une solution de monitoring et d'alerte orientée métriques, développée initialement par **SoundCloud**. Elle repose sur une architecture de type pull, dans laquelle le serveur Prometheus interroge régulièrement les nœuds et services du cluster à travers des endpoints HTTP exposant des données au format `/metrics`.

Les principales caractéristiques de Prometheus sont :

- Langage de requête **PromQL** pour interroger et analyser les séries temporelles
- Stockage intégré en base de données time-series

- Support natif des alertes via **Alertmanager**
- Extensibilité via des **exporters** (Node Exporter, Slurm Exporter, BeeGFS Exporter, etc.).

Présentation de Grafana

Grafana est un outil de **visualisation de données** capable de se connecter à plusieurs sources, dont Prometheus. Il permet de créer des **tableaux de bord interactifs et dynamiques**, offrant une vision en temps réel et historique de l'état du cluster.

Grafana est utilisé pour :

- Créer des Dashboard personnalisés ;
- Visualiser des métriques système (CPU, RAM, réseau, I/O disque) ;
- Suivre l'activité des jobs SLURM (durée, ressources, statut) ;
- Surveiller les performances du système de fichiers BeeGFS.

Avantages de cette solution de supervision

- **Vue centralisée** : tous les composants critiques du cluster sont surveillés depuis un point unique.
- **Réactivité** : les alertes configurées dans Prometheus permettent de réagir rapidement en cas de surcharge, défaillance matérielle ou anomalie logicielle.
- **Analyse des performances** : l'historique des métriques permet d'identifier les périodes de pic d'activité, de dimensionner les ressources, et d'optimiser les politiques de scheduling.
- **Facilité d'utilisation** : Grafana offre une interface conviviale pour les administrateurs, sans nécessiter de compétences avancées en ligne de commande.

3.4.6. LDAP : Gestion centralisée des utilisateurs

La gestion des utilisateurs est un aspect crucial dans une infrastructure HPC, notamment pour des raisons de sécurité, de traçabilité, de facilité d'administration et de cohérence entre les nœuds. Pour répondre à ces exigences, ce projet s'appuie sur LDAP (Lightweight Directory Access Protocol) pour la gestion centralisée des comptes utilisateurs.

LDAP est un protocole standard ouvert utilisé pour interroger et modifier des services d'annuaire. Il permet de centraliser les informations d'identification (identifiants, mots de passe, groupes, droits) des utilisateurs dans une base unique, accessible depuis l'ensemble des nœuds du cluster.

Dans le contexte HPC, un annuaire LDAP évite la duplication et la gestion manuelle des comptes locaux sur chaque machine. Il agit comme référentiel unique d'authentification, garantissant une gestion cohérente et sécurisée des accès.

L'utilisation de LDAP offre de nombreux bénéfices :

- **Unicité de l'authentification** : chaque utilisateur dispose d'un seul identifiant et mot de passe pour tous les services (Shell, SLURM, OnDemand, etc.).
- **Simplification de l'administration** : la création, la modification ou la suppression des utilisateurs se fait une seule fois dans l'annuaire.
- **Synchronisation automatique** : les informations utilisateurs sont immédiatement visibles sur tous les nœuds.

- **Sécurité accrue** : gestion fine des droits d'accès via les groupes LDAP, audibilité centralisée.
- **Compatibilité avec les outils standards** : LDAP s'intègre nativement avec les systèmes Linux, les applications web, les portails de supervision et de soumission.

3.4.7. Ansible : Automatisation du déploiement

Le déploiement et la configuration manuelle d'un cluster HPC représentent une tâche complexe, sujette aux erreurs humaines, chronophage, et difficilement reproductible. Afin de répondre à ces défis, ce projet intègre Ansible, un outil d'automatisation open source, pour assurer un déploiement reproductible, cohérent et maintenable de l'ensemble de l'infrastructure. [25]

Ansible est un outil d'automatisation de la configuration, du déploiement et de l'orchestration. Il fonctionne sans agent, en se basant sur SSH pour se connecter aux machines cibles, et utilise des playbooks (fichiers YAML) pour décrire les actions à exécuter. [25]

Rôle d'Ansible dans le projet

Dans le cadre de ce projet HPC chez Sonatrach, Ansible a été utilisé pour automatiser :

- L'installation du système de base AlmaLinux et des dépendances
- La configuration réseau des nœuds (IP statiques, DNS, NTP, etc.)
- Le déploiement de SLURM sur tous les nœuds avec leurs rôles spécifiques (contrôleur, compute)
- L'installation et la configuration de BeeGFS, Prometheus, Grafana et Open OnDemand
- La mise en place du serveur LDAP et la configuration des clients (SSSD, PAM)
- La synchronisation des utilisateurs et des permissions
- La gestion des mises à jour et des redémarrages planifiés.

Avantages de l'automatisation avec Ansible

- **Gain de temps** : un déploiement complet, qui nécessiterait plusieurs heures voire jours manuellement, peut être réalisé en quelques minutes.
- **Cohérence** : tous les nœuds sont configurés de manière identique, évitant les erreurs humaines ou les divergences de configuration.
- **Reproductibilité** : le cluster peut être recréé à l'identique à tout moment, ce qui est essentiel pour la résilience et les tests.
- **Évolutivité** : l'ajout d'un nouveau nœud ne nécessite que son inclusion dans l'inventaire Ansible ; la configuration est alors automatiquement appliquée.
- **Documentable** : les playbooks Ansible servent de **documentation vivante** de l'infrastructure. [25]

3.5. Conception de l'interface graphique pour Seismic Unix

Dans le cadre de la conception de notre solution HPC dédiée au traitement sismique, une attention particulière a été portée à l'accessibilité de l'environnement Seismic Unix, qui repose exclusivement sur une interface en ligne de commande. Bien que cette approche convienne aux utilisateurs avancés, elle peut représenter une difficulté pour les ingénieurs et géophysiciens souhaitant soumettre rapidement leurs traitements sans passer par des scripts complexes.

Pour répondre à cette problématique, nous avons conçu et développé une interface graphique légère permettant à l'utilisateur de soumettre et exécuter ses jobs sismiques de manière simple et intuitive. Cette interface communique directement avec le système SLURM pour transmettre les paramètres des jobs et récupérer les résultats d'exécution.

.

3.5.1. Objectifs de conception

Les objectifs visés par cette interface sont les suivants :

- Réduire la complexité de soumission des jobs **Seismic Unix**
- Offrir un environnement ergonomique pour les utilisateurs non experts en ligne de commande
- Faciliter l'intégration dans les workflows existants sur le cluster
- Garantir une solution 100 % open source, librement déployable et adaptable.

Cette interface représente un pont essentiel entre l'utilisateur final et l'infrastructure HPC, en démocratisant l'accès aux puissances de calcul pour le traitement sismique, tout en respectant les contraintes de robustesse, de flexibilité et de sécurité de l'environnement de production de SONATRACH.

3.6. Architecture fonctionnelle en couches

Dans le cadre de ce projet, l'infrastructure HPC a été conçue selon une architecture modulaire répartie sur trois couches principales : la couche d'exécution, la couche middleware, et la couche utilisateur. Cette séparation logique permet de clarifier les rôles des composants, de faciliter la maintenance, et d'optimiser les flux entre utilisateurs, ordonnanceur et nœuds de calcul.

3.6.1. Couche d'exécution (Exécution Layer)

La couche d'exécution constitue le socle opérationnel de l'infrastructure HPC. Elle est responsable de l'allocation et de la gestion des ressources matérielles pour l'exécution des applications et traitements scientifiques. Dans notre projet, cette couche repose principalement sur une combinaison de systèmes d'exploitation, de gestionnaire de ressources et de systèmes de fichiers distribués, garantissant performance, scalabilité et fiabilité.

3.6.1.1. Système d'exploitation : AlmaLinux

Les nœuds du cluster HPC sont équipés d'AlmaLinux, une distribution Linux open source. Ce choix garantit une stabilité et une compatibilité optimales avec les outils HPC et les middlewares utilisés. AlmaLinux permet également une gestion fine des ressources système et une intégration aisée avec les outils d'administration. [19]

3.6.1.2. Gestionnaire de ressources : SLURM

La gestion des tâches et la planification des ressources sont assurées par SLURM (Simple Linux Utility for Resource Management), un gestionnaire open source largement utilisé dans les environnements HPC. SLURM offre plusieurs fonctionnalités essentielles : [21]

- Allocation dynamique des ressources CPU, mémoire et GPU.
- Planification des jobs selon des politiques configurables (priorités, partitions).
- Surveillance et gestion des files d'attente.
- Interface de soumission de jobs via commandes ou API.

Dans notre projet, SLURM est configuré pour optimiser l'utilisation des ressources selon les besoins spécifiques des traitements sismiques, en intégrant une prédiction intelligente des ressources nécessaires via un modèle d'apprentissage automatique (voir section dédiée à l'IA).

3.6.1.3. Système de fichiers distribué : BeeGFS

Pour répondre aux exigences élevées en termes d'IO et de stockage partagé, le cluster utilise BeeGFS, un système de fichiers parallèle et distribué. BeeGFS permet de :

- Fournir un accès simultané et performant aux données sur tous les nœuds.
 - Assurer une haute disponibilité et scalabilité du stockage.
 - Faciliter la gestion des gros volumes de données générées par les traitements sismiques.
- [17]

Cette solution assure que les jobs lancés par SLURM accèdent efficacement aux données, limitant ainsi les goulots d'étranglement liés au stockage.

3.6.1.4. Optimisation via Intelligence Artificielle

Une innovation majeure de la couche d'exécution est l'intégration d'un module d'intelligence artificielle basé sur une Random Forest. Ce module prédit, en temps réel, le nombre optimal de CPU nécessaires pour chaque job SLURM, ainsi que son temps d'exécution estimé. Cette prédiction permet :

- Une meilleure allocation des ressources, évitant la sur- ou sous-utilisation.
- Une réduction du temps global d'exécution des jobs.
- Une planification plus fine des charges dans le cluster.

3.6.2. Couche Middleware

La couche middleware joue un rôle fondamental dans l'architecture du cluster HPC en assurant l'interface entre la couche d'exécution (infrastructure physique et gestionnaire de ressources) et la couche utilisateur. Elle facilite la gestion, l'orchestration et la sécurisation des services tout en offrant des outils d'administration et d'interaction performants.

3.6.2.1. Gestion des ressources et planification avancée

Le middleware intègre SLURM en tant que gestionnaire de ressources principal, mais apporte également des extensions pour optimiser la planification des tâches. Par exemple :

- **Plugins personnalisés SLURM** : Pour adapter la planification aux besoins spécifiques des traitements sismiques.
- **Intégration du module d'intelligence artificielle** qui prédit la charge des jobs pour améliorer la planification et éviter la surcharge des nœuds.

3.6.2.2. Gestion centralisée des utilisateurs : LDAP

Le middleware intègre un service LDAP (Lightweight Directory Access Protocol) pour centraliser la gestion des identités et des accès. Cette solution permet :

- Un contrôle d'accès unifié et sécurisé à l'ensemble du cluster.
- La gestion fine des droits utilisateurs selon les rôles (administrateurs, chercheurs, opérateurs).
- La simplification de l'administration des comptes et la conformité avec les politiques de sécurité de Sonatrach.

3.6.2.3. Interface web Open OnDemand

Pour faciliter l'usage du cluster par les utilisateurs finaux, la couche middleware propose une interface web via Open OnDemand. Cette interface offre :

- Un portail web simple et intuitif pour soumettre, suivre et gérer les jobs SLURM.

- L'accès aux fichiers et à l'environnement de travail via un navigateur, sans besoin d'un accès SSH direct.
- La visualisation des résultats et des logs en temps réel.

3.6.2.4. Surveillance et métriques : Prometheus & Grafana

La couche middleware assure la collecte, l'analyse et la visualisation des métriques système et applicatives grâce à l'intégration de Prometheus et Grafana :

- **Prometheus** collecte en continu les données de performance des nœuds, du réseau et des jobs.
- **Grafana** fournit des tableaux de bord personnalisés pour une supervision en temps réel.
- Ces outils permettent d'anticiper les problèmes, d'optimiser les ressources et de garantir la disponibilité du service. [26]

3.6.2.5. Automatisation avec Ansible

Enfin, la couche middleware intègre Ansible pour l'automatisation des déploiements, mises à jour et configurations du cluster :

- Déploiement automatisé des composants middleware et d'exécution.
- Gestion centralisée des configurations pour assurer la cohérence et la rapidité des interventions.
- Réduction des erreurs humaines et amélioration de la maintenabilité [25]

3.6.3. Couche Utilisateur (User Layer)

La couche utilisateur représente l'interface directe entre les chercheurs, ingénieurs et opérateurs et l'infrastructure HPC. Elle est conçue pour offrir une expérience fluide, sécurisée et performante, tout en masquant la complexité technique de la couche d'exécution et middleware. Cette couche vise à maximiser la productivité des utilisateurs en leur fournissant des outils adaptés à leurs besoins métier.

3.6.3.1. Interface d'accès simplifiée

Les utilisateurs accèdent au cluster principalement via l'interface web Open OnDemand, qui propose :

- Un environnement de travail accessible depuis un navigateur web, évitant la nécessité d'un accès SSH direct.
- Des modules pour soumettre, gérer et suivre leurs jobs SLURM.
- L'accès aux données stockées sur BeeGFS et aux résultats des traitements, avec des outils de visualisation intégrés.
- Un espace personnel sécurisé avec gestion des droits, en lien avec le service LDAP.

3.6.3.2. Outils de soumission et gestion des jobs

Pour simplifier l'utilisation des ressources HPC, la couche utilisateur intègre plusieurs outils :

- Des modèles d'interface de soumission préconfigurés adaptés aux traitements sismiques.
- Une application Streamlit développée dans le cadre du projet, permettant de prédire le nombre optimal de CPU et le temps d'exécution d'un job via un modèle de Random Forest, ce qui aide les utilisateurs à dimensionner précisément leurs requêtes.
- Un système de notifications et alertes sur l'état des jobs (exécution, succès, erreurs).

3.6.3.3. Environnements de développement et bibliothèques

Afin de favoriser la productivité et la réutilisation des codes, la couche utilisateur met à disposition :

- Des environnements logiciels modulaires et personnalisables (modules Lmod) pour charger les bibliothèques spécifiques aux traitements sismiques et calcul HPC.
- Des accès aux notebooks Jupyter via Open OnDemand pour des analyses interactives.
- Un référentiel de scripts et workflows validés, favorisant le partage des bonnes pratiques.

3.6.3.4. Sécurité et gestion des accès

La sécurité est assurée par une authentification centralisée via LDAP, combinée à des contrôles d'accès finement configurés, garantissant que chaque utilisateur dispose uniquement des droits nécessaires à ses tâches. Les données sensibles sont protégées via des mécanismes de chiffrement et de sauvegarde régulière.

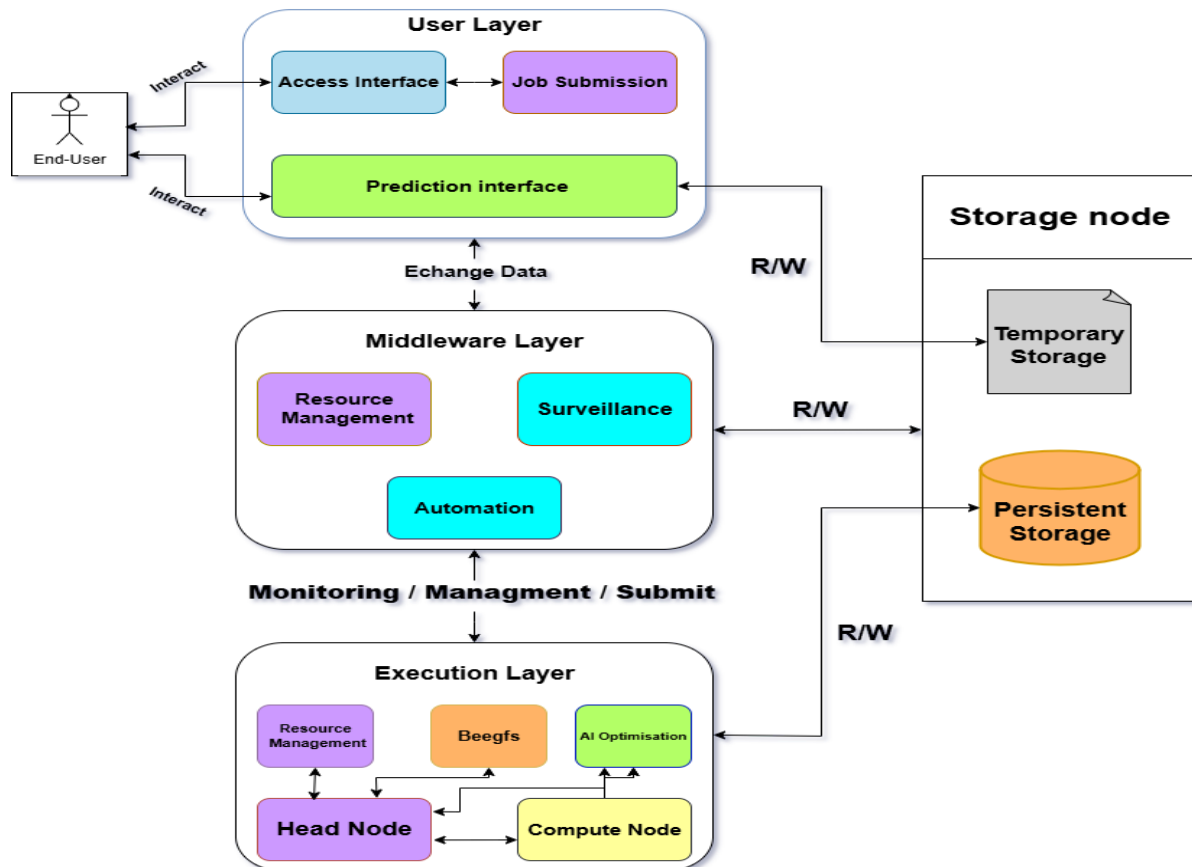


Figure 3-1 Architecture Globale du Plateforme

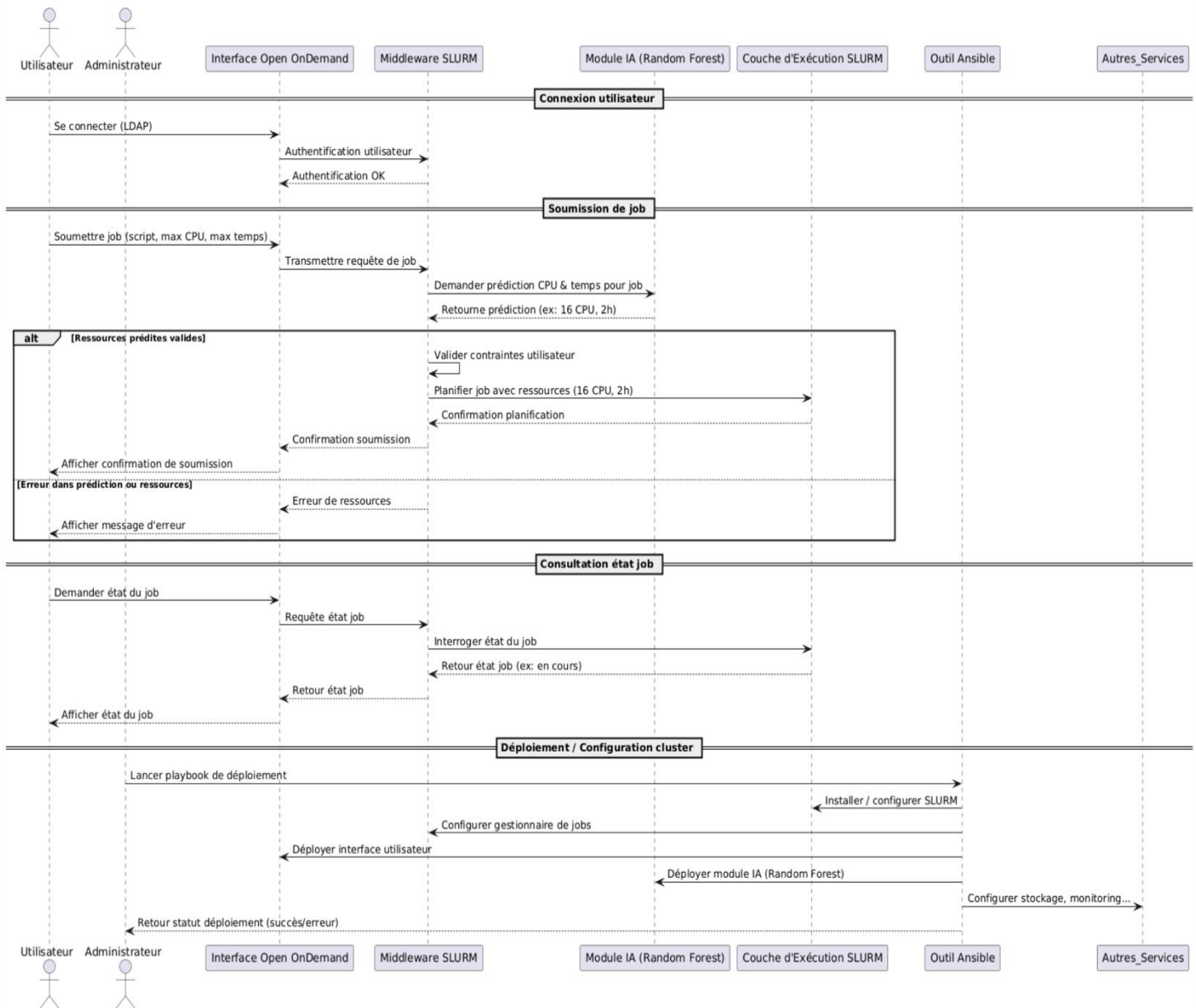


Figure 3-2 Diagramme de séquence

3.7 Conclusion

La conception de l'architecture HPC a permis de définir une infrastructure équilibrée entre performance, simplicité et évolutivité. Le choix du matériel, l'organisation logique des nœuds, ainsi que les technologies open source sélectionnées assurent une base solide pour le déploiement du cluster. Cette architecture répond aux besoins de calcul intensif de manière fiable et constitue le socle technique de la mise en œuvre détaillée dans le chapitre suivant

Chapitre 4. Implémentation d'un cluster HPC

4.1. Introduction

Ce chapitre décrit en détail la mise en œuvre concrète du cluster HPC conçu dans le cadre de ce projet. Il présente les différentes étapes techniques réalisées, depuis l'installation manuelle initiale jusqu'à l'automatisation complète à l'aide d'Ansible. L'ensemble des composants du système sont progressivement intégrés pour offrir une interface utilisateur conviviale. Chaque outil est configuré pour fonctionner de manière cohérente et efficace dans une architecture distribuée sur trois nœuds physiques.

4.2. Déploiement manuel initial

Avant d'automatiser le déploiement à l'aide d'Ansible, une phase de **mise en place manuelle** a été nécessaire pour valider les étapes critiques d'installation, comprendre les dépendances entre services et assurer le bon fonctionnement du cluster dans sa configuration minimale. Cette phase a permis d'identifier les points de blocage potentiels et de construire une base solide pour l'automatisation future.

4.2.1. Configuration de base des nœuds

Chaque serveur (nœud maître et nœuds de calcul) a été installé avec **AlmaLinux**, une distribution stable, compatible avec les outils HPC et bénéficiant de mises à jour à long terme.

- Installation AlmaLinux 9.5 via clé USB bootable.
- Mise à jour du système.
- Configuration des utilisateurs système (root + comptes administrateurs).
- Paramétrage du hostname :
 - node-master, node-1, node-2
- Modification du fichier `/etc/hosts` pour assurer la résolution locale des nœuds.

```
192.168.20.250 node-master
192.168.20.216 node-1
192.168.20.18 node-2
```

- Installation des paquets de base.
- Génération de clés SSH et échange sans mot de passe pour la communication inter-nœuds.

4.2.2. Mise en place du réseau et synchronisation

Un cluster HPC nécessite un réseau rapide, stable et bien configuré pour permettre la communication entre les nœuds, ainsi qu'une synchronisation temporelle rigoureuse pour éviter les incohérences lors des soumissions de jobs.

4.3. Automatisation avec Ansible

Après la configuration manuelle initiale, une solution d'automatisation a été mise en place pour faciliter le déploiement, la reconfiguration et la gestion du cluster. **Ansible**, un outil d'Infrastructure as Code (IaC), a été choisi pour sa simplicité, son approche agentless (aucun logiciel à installer sur les nœuds), et sa compatibilité avec les environnements Linux.

Il est également possible d'ajouter de nouveaux serveurs au cluster de manière très simple, il suffit d'ajouter leurs adresses IP dans les fichiers d'inventaire d'**Ansible** pour qu'ils soient automatiquement intégrés et configurés selon les mêmes règles

L'objectif de cette phase est d'automatiser l'installation et la configuration de tous les composants du cluster. [25]

4.3.1. Rôles Ansible développés

L'automatisation repose sur des **rôles Ansible** séparés, organisés par fonction. Chaque rôle contient des tâches (*tasks*), fichiers de configuration (*templates*), et variables (*defaults*) pour déployer un service de façon modulaire. [25]

Rôle	Description
Common	Configuration de base système (utilisateurs, SSH...)
Network	Configuration IP statique et des interfaces
Ntp	Installation et configuration de Chrony (NTP)
Slurm_master	Installation et configuration de slurmctld
Slurm_node	Installation et configuration de slurmd
Beegfs_mgmt	Déploiement du serveur de gestion BeeGFS
Beegfs_client	Montage BeeGFS sur les nœuds de calcul
Monitoring	Déploiement de Prometheus et Grafana
Ood	Installation de Open OnDemand

Table 4-1 Liste des Rôles Créés

4.4. Configuration de SLURM

Architecture adoptée

- **Slurmctld** : déployé sur le **nœud maître** – c'est le démon central chargé de planifier et d'allouer les ressources.
- **Slurmd** : installé sur **chaque nœud de calcul** – il exécute les jobs assignés par le contrôleur.

- **Slurmdbd** : non utilisé dans cette version (pas de base de données de comptabilité activée, pour simplification). [21]

Fichier de configuration

Le fichier de configuration `/etc/slurm/slurm.conf` a été généré avec l'outil en ligne Slurm Configurator, puis adapté aux spécificités du cluster.

Autres éléments configurés

- **MUNGE** pour l'authentification sécurisée entre nœuds.
- Répertoires de logs et états persistants.
- Utilisation du **système de partitions** SLURM pour organiser les tâches. [21]

4.4.1. Test de soumission de jobs

Après le démarrage des services SLURM (`slurmctld` sur le maître, `slurmd` sur les nœuds de calcul), des tests fonctionnels ont été réalisés pour valider le bon fonctionnement du cluster.

Résultats observés

- Les jobs ont été acceptés et exécutés correctement.
- La charge a été répartie entre les deux nœuds de calcul.
- Les logs ont confirmé la bonne communication entre `slurmctld` et `slurmd`. [21] [25]
- Les métriques système ont été correctement remontées dans Grafana.

4.5. Déploiement de BeeGFS :

Dans un environnement HPC, les performances du système de fichiers sont critiques pour garantir un accès rapide et concurrent aux données par les jobs. Pour répondre à ces exigences, le système de fichiers parallèle **BeeGFS** (Beyond Extensible Global File System) a été choisi pour ce projet. Conçu pour les charges de travail scientifiques, BeeGFS offre une architecture modulaire, performante et facilement extensible [17].

Le déploiement de BeeGFS s'est appuyé sur trois composants essentiels :

Composant	Rôle	Nœuds concernés
Management Server	Coordonne l'ensemble du système BeeGFS	Nœud maître
Metadata Server	Gère les noms de fichiers, permissions	Nœud maître
Storage Server	Stocke physiquement les blocs de données	Nœud maître
Client BeeGFS	Accède au système BeeGFS	Tous les nœuds (maître + calcul)

Table 4-2 Fonctionnalités des composants BeeGFS

4.6. Supervision avec Grafana / Prometheus

Un système HPC nécessite une surveillance en temps réel pour garantir son bon fonctionnement, détecter rapidement les anomalies, évaluer les performances, et anticiper les besoins en maintenance. Pour répondre à ces besoins, une solution de supervision basée sur **Prometheus** (collecte de métriques) et **Grafana** (visualisation graphique) a été mise en place. [26]

Métriques collectées :

Type de métrique	Exemple de données
CPU	Charge CPU, usage par cœur
Mémoire	RAM totale, utilisée, libre
Disque	I/O disque, capacité, usage IOPS
Réseau	Débit entrant/sortant, erreurs, paquets
Disponibilité des nœuds	UP/DOWN, disponibilité, latence de réponse

Table 4-3 Métriques de supervision dans Grafana

4.7. Dashboards développés

Les tableaux de bord ont été créés dans Grafana afin de visualiser les performances du cluster de manière claire et exploitable par les administrateurs et les utilisateurs avancés. [26]

Dashboard 1 : Vue globale du cluster

- Charge CPU moyenne par nœud
- Utilisation de la mémoire
- État du disque (lecture/écriture, occupation)
- Disponibilité des nœuds
- Avoir une vision d'ensemble de l'état du cluster en temps réel.

Résultats obtenus

- La supervision s'est révélée **légère, réactive et peu intrusive**.
- Les tableaux de bord sont **accessibles via l'interface web Grafana**, avec des filtres par période, nœud ou utilisateur.
- Des **alertes peuvent être définies** (ex. : CPU > 90%, nœud inactif).

4.8. Interface utilisateur avec Open OnDemand

Dans un cluster HPC, l'accessibilité est souvent un frein pour les utilisateurs non familiers avec les interfaces en ligne de commande. Pour démocratiser l'usage du cluster et faciliter la soumission de jobs, l'interface web **Open OnDemand** a été déployée. Cette solution open source permet d'interagir avec le cluster à travers un **navigateur web**, sans configuration côté client.

Open OnDemand est un portail web conçu pour les environnements HPC, développé par l'**Ohio Supercomputing Center**. Il offre une interface intuitive pour :

- Lancer des sessions interactives ou distantes.
- Soumettre des jobs SLURM via formulaire.
- Naviguer dans les fichiers et dossiers du cluster.
- Visualiser les résultats graphiques.
- Utiliser des IDE et notebooks scientifiques (ex. : Jupyter).
- Accessible via un **navigateur web** à l'adresse : `https://<IP_CLUSTER>:443`
- Interface intuitive pour soumettre des jobs SLURM à partir de modèles prédéfinis
- Affiche les jobs en cours, en file d'attente ou terminés
- Informations sur les ressources consommées
- Les utilisateurs peuvent soumettre des jobs sans connaître la ligne de commande.

4.9. Implémentation de l'interface utilisateur pour Seismic Unix

L'interface graphique développée pour Seismic Linux a été implémentée dans le but de simplifier l'interaction des utilisateurs avec l'environnement de traitement sismique. L'approche adoptée repose sur une application web légère permettant de soumettre des jobs, de spécifier les paramètres nécessaires et de suivre l'état de l'exécution.

4.9.1. Architecture de l'interface

L'interface repose sur une architecture web classique en trois couches :

- **Frontend** : développé en HTML, CSS et JavaScript, ce composant gère l'affichage, les formulaires de saisie des paramètres (nom du job, type de traitement, nombre de CPU, fichier d'entrée, etc.) ainsi que les retours utilisateur.
- **Backend (bridge)** : un script (par exemple en Python, Bash ou Node.js) agit comme passerelle entre l'interface et le système de gestion de jobs SLURM. Il génère dynamiquement les scripts de soumission (.sbatch) à partir des paramètres saisis, puis les soumet via la commande sbatch.
- **Système SLURM** : exécute le job sur les nœuds du cluster, gère l'allocation des ressources et fournit les fichiers de sortie (logs, erreurs, résultats).

4.9.2. Fonctionnalités principales

- L'interface permet à l'utilisateur :
- D'indiquer les ressources nécessaires (nombre de CPU, mémoire, durée maximale, etc.)
- D'importer un fichier sismique d'entrée

4.9.3. Technologies utilisées

- **HTML** pour la structure des pages,
- **CSS** pour la mise en forme et l'ergonomie visuelle,
- **JavaScript** pour la logique d'interaction côté client (validation des formulaires, interactions dynamiques, etc.).
- **Flask (Python)** pour récupérer les valeurs depuis l'interface, les charger dans un script, puis lancer l'exécution.

Cette approche permet une compatibilité universelle via un navigateur web, sans nécessiter l'installation d'un logiciel client, et s'intègre facilement avec les autres outils de la couche utilisateur, notamment Open OnDemand

4.10. Conclusion

La mise en œuvre du cluster HPC a permis de construire une infrastructure complète, performante et évolutive, intégrant les standards actuels du calcul haute performance. Grâce à l'automatisation via Ansible, le déploiement est devenu reproductible et fiable. La combinaison de SLURM, BeeGFS, Prometheus, Grafana et Open OnDemand permet une exploitation optimisée des ressources et un accès simplifié pour les utilisateurs. Cette base solide constitue le socle technique sur lequel a été intégrée la couche d'intelligence artificielle, objet du chapitre suivant

Chapitre 5. Intégration d'intelligence artificiel

5.1. Introduction

L'intelligence artificielle (IA) est un terme générique qui désigne différentes stratégies et techniques que vous pouvez utiliser pour rendre les machines plus humaines. L'IA englobe tout, des assistants intelligents comme Alexa aux aspirateurs robotisés en passant par les voitures à conduite autonome. Le machine Learning (ML) est l'une des nombreuses autres branches de l'IA. Le ML est la science qui consiste à développer des algorithmes et des modèles statistiques que les systèmes informatiques utilisent pour effectuer des tâches complexes sans instructions explicites. Les systèmes s'appuient plutôt sur des motifs et des inférences. Les systèmes informatiques utilisent des algorithmes de ML pour traiter de grandes quantités de données historiques et identifier des motifs de données. Bien que le machine Learning soit de l'IA, toutes les activités de l'IA ne relèvent pas forcément du machine Learning.

5.1.1. Définition

L'apprentissage automatique ou Machine Learning (ML) est l'étude scientifique des algorithmes et des modèles statistiques. Il est utilisé pour apprendre aux machines à traiter les données plus efficacement. L'objectif de l'apprentissage automatique est de laisser les machines apprendre par elles-mêmes à partir des données c'est à dire apprendre quel calcul effectuer [27], contrairement à la programmation où elle se contente d'exécuter à la lettre des règles prédéterminées. « L'apprentissage automatique est la discipline donnant aux ordinateurs la capacité d'apprendre sans qu'ils soient explicitement programmés. » Arthur Samuel, 1959 [28]. Et voici une autre plus technique : « Étant donné une tâche T et une mesure de performance P, on dit qu'un programme informatique apprend à partir d'une expérience E si les résultats obtenus sur T, mesurés par P, s'améliorent avec l'expérience E ». [28].

5.1.2. Les étapes clés pour une méthode de Machine Learning

Une méthode de ML comprend principalement les quatre étapes suivantes [29]:

- Ingénierie des caractéristiques.
- Choix de l'algorithme d'apprentissage automatique approprié.
- Former et évaluer les performances du modèle.
- Utiliser le modèle formé pour classer ou prédire les données.

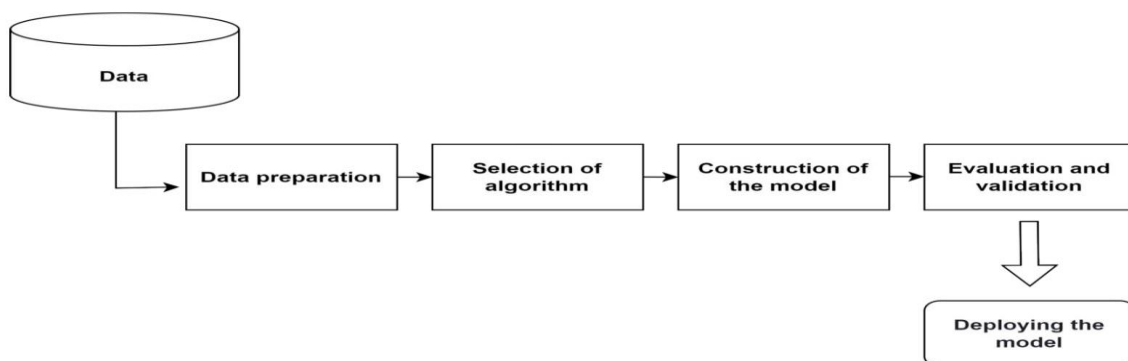


Figure 5-1 Processus de Machine Learning.

5.2. Types d'apprentissage automatique

Pour donner à une machine la capacité d'apprendre, on utilise des méthodes d'apprentissage. Parmi ces méthodes, il existe trois catégories majeures :

5.2.1. Apprentissage supervisé

L'apprentissage supervisé est la méthode la plus courante en Machine Learning et en Deep Learning. Il constitue d'un ensemble des données (des exemples) d'apprentissage. Les données d'entraînement sont composées d'une entrée (typiquement, une instance du problème) et d'une sortie (typiquement, la réponse réputée bonne). Lors de l'apprentissage, on fournit les paires (observation, étiquette), et l'algorithme génère une fonction qui consiste à apprendre à faire correspondre des données d'entrée à des cibles [30]. Avec l'apprentissage supervisé, la machine peut apprendre à faire de nombreuses applications comme : la reconnaissance optique des caractères, la reconnaissance vocale, la classification d'images, la traduction de langues [31].

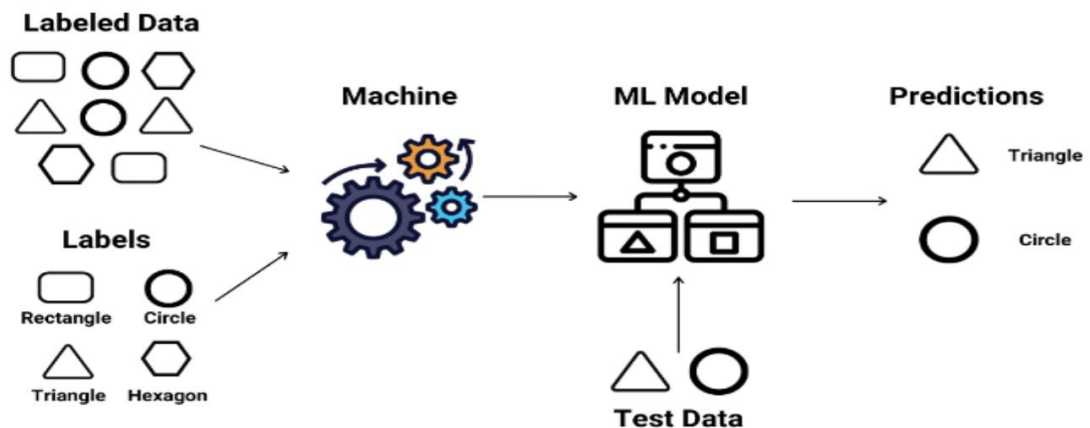


Figure 5-2 Apprentissage supervisé.

5.2.1.1. Classification

Dans ce type, on utilise des variables discrètes, c'est-à-dire des variables qui ne peuvent prendre que certaines valeurs. L'algorithme doit déterminer l'une des sorties possibles pour ce problème. Il peut soit identifier directement la catégorie, soit produire une distribution de probabilités pour chaque catégorie. La classe ayant la probabilité la plus élevée est alors retenue [27]. Par exemple, on peut identifier si un message spam ou pas en fonction du nombre de lien, du nombre de fautes d'orthographe, etc.).

On peut distinguer deux types de classification : la classification binaire, qui consiste à séparer les données en deux classes distinctes, et la classification multi-classe, qui permet de classer les données en plusieurs classes.

5.2.1.2. Régression

Dans ce type, on utilise des variables continues c'est-à-dire des variables qui peuvent prendre une infinité de valeurs. L'algorithme fournit une valeur numérique en sortie au lieu de prédire une étiquette. Il est aussi possible de prédire plusieurs valeurs numériques en sortie

[27]. Le but de la régression est d'estimer une valeur de sortie à partir des valeurs d'un ensemble des observations. Par exemple, on peut prédire le prix d'un appartement en fonction de sa surface, sa localisation, sa qualité, ...)

Dans un contexte HPC, la régression est particulièrement utile pour **prédire des métriques de performance** telles que :

- Le **temps d'exécution** d'un job,
- Le **nombre optimal de cœurs CPU** nécessaires,

L'algorithme apprend à partir d'un ensemble de paires $\{X, y\}$, où :

- X représente les caractéristiques du job (nombre de cœurs, taille des données, type de traitement, etc.),
 - Y est la variable continue à prédire (temps d'exécution, CPU optimal, etc.).
- Le modèle tente de minimiser une fonction de perte, souvent l'erreur quadratique moyenne, entre les prédictions $\hat{y}$ et les valeurs réelles y . [32]

Étapes clés de l'approche supervisée pour la régression

- **Collecte des données** : Extraction de logs incluant des paramètres (features) et leurs résultats associés (target). [33]
- **Prétraitement** : Nettoyage, normalisation, encodage des variables catégorielles, suppression des valeurs aberrantes.
- **Séparation du jeu de données** : En général, les données sont divisées en : Entraînement (80 %), Test (20 %) pour l'évaluation du modèle [33]
- **Choix de l'algorithme** : Plusieurs modèles sont possibles, tels que : **Régression linéaire**
- **Arbres de décision / Random Forest XGBoost**, etc.
- **Apprentissage** : Ajustement des paramètres du modèle pour minimiser l'erreur sur le jeu d'entraînement.
- **Évaluation** : Mesure des performances sur le jeu de test via des métriques comme **MAE** (Mean Absolute Error) **R² Score** (coefficient de détermination)

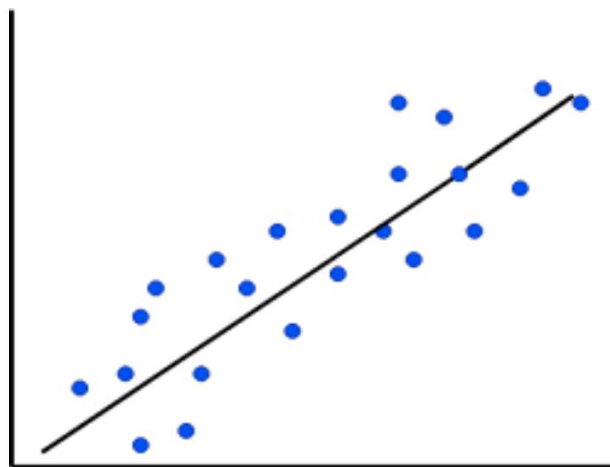


Figure 5-3 Exemple de régression linéaire avec données annotées.

5.2.2. Apprentissage non supervisé

Ce type d'apprentissage doit être utilisé lorsque la catégorie de données acquises n'est pas connue à l'avance c'est-à-dire il n'y'a pas de résultat prédéterminé (on fournit à la machine un ensemble de données qui ne sont pas étiqueté) [33]. L'objectif est de regrouper avec succès les observations disponibles dans leurs propres classes par lui-même pour des catégories d'objets basées sur des mesures de similarité entre les objets.

L'apprentissage non supervisé peut être classé en deux catégories d'algorithmes :

- Regroupement (Clustering).
- Association.

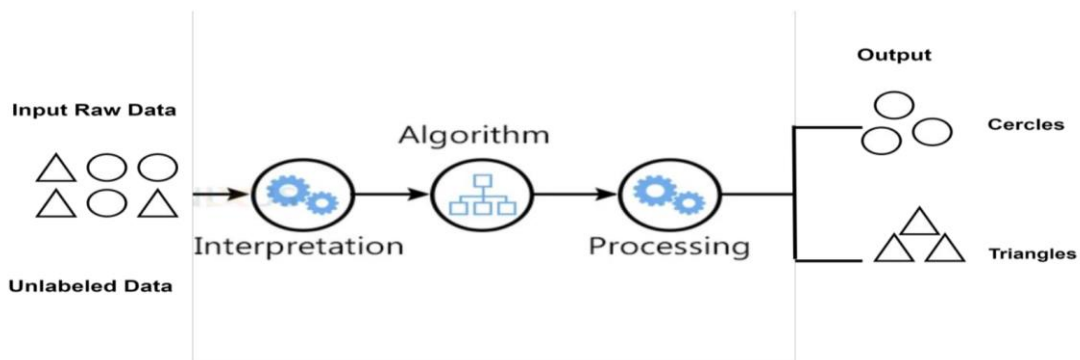


Figure 5-4 Apprentissage non supervisé

5.2.3. Apprentissage par renforcement

L'apprentissage par renforcement est une méthode qui utilise un processus d'essais et d'erreurs pour trouver la meilleure action à effectuer pour chaque situation qu'un robot percevra afin de maximiser la récompense [33]. Le système ajuste ses paramètres en fonction des commentaires qu'il reçoit de l'environnement :

On cite comme exemple un système qui simule un joueur d'échecs en utilisant les résultats des étapes précédentes pour améliorer ses performances est un système qui apprend par renforcement.

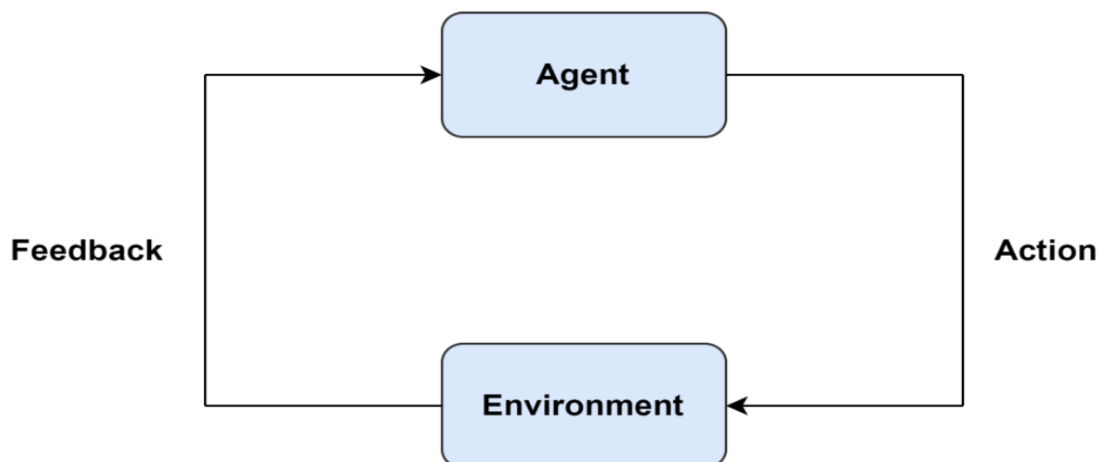


Figure 5-5 Apprentissage par renforcement.

5.2.4. Algorithmes d'apprentissage supervisé

Les algorithmes d'apprentissage supervisé les plus importants sont :

- Régression linéaire (Linear Regression).
- Régression logistique (Logistic Regression).
- Naïve Bayes gaussien (Gaussian Naïve Bayes).
- Machine à vecteur de support (Support Vector Machine SVM).
- Arbres de décision (Decision Trees).
- K le plus proche (K Nearest Neighbor KNN).
- Forêt aléatoire (Random Forest).

Après avoir conçu et déployé l'architecture du cluster HPC, nous avons cherché à améliorer l'efficacité de l'utilisation des ressources via une approche basée sur l'intelligence artificielle. Ce chapitre présente l'intégration d'un module prédictif permettant d'estimer le nombre optimal de CPU et le temps d'exécution d'un job SLURM, à l'aide d'un modèle de régression entraîné sur les logs du système.

5.3. Objectif du module IA

Le fonctionnement d'un cluster HPC repose sur une gestion fine et efficace des ressources partagées : CPU, mémoire, stockage, etc. Les utilisateurs doivent estimer les ressources nécessaires à leurs jobs au moment de la soumission. Ces estimations sont souvent approximatives, ce qui entraîne plusieurs problèmes majeurs :

- **Des surallocations** : les utilisateurs réservent plus de ressources que nécessaire par précaution. Cela conduit à un gaspillage de ressources, une sous-utilisation du cluster et des délais accrus pour les autres utilisateurs.
- **Des sous-allocations** : les jobs échouent ou s'exécutent lentement, nécessitant parfois d'être relancés avec plus de ressources, ce qui accroît la charge globale sur le système.

Au-delà de la prédiction des ressources, la **prédiction du temps d'exécution des jobs** joue un rôle clé dans l'optimisation du fonctionnement d'un cluster HPC. En effet, connaître de manière fiable la durée estimée d'un job apporte plusieurs avantages essentiels :

1. Optimisation de l'ordonnancement des jobs

Les ordonnanceurs comme SLURM peuvent tirer parti de la prédiction du temps d'exécution pour mieux planifier le lancement des jobs. Une estimation précise permet d'allouer les ressources au moment optimal, réduisant les conflits entre jobs et améliorant la fluidité d'exécution dans la file d'attente.

2. Réduction des temps d'attente

En anticipant la durée probable des jobs en cours et à venir, le système peut mieux gérer la charge, ce qui diminue les temps d'attente en file. Cela est particulièrement critique dans les environnements HPC très sollicités où la demande dépasse souvent la capacité disponible.

3. Amélioration de la qualité de service et de l'expérience utilisateur

Informé des utilisateurs du temps d'exécution prévu leur permet d'organiser leur travail plus efficacement, de prendre des décisions éclairées sur la planification et la gestion de leurs calculs. Cette transparence accroît la satisfaction et réduit la frustration liée aux imprévus.

4. Diminution des erreurs de planification et des relances de jobs

Les sous-estimations du temps d'exécution entraînent souvent des interruptions prématurées ou des relances, ce qui génère une perte de ressources et de temps. Une prédiction précise limite ces événements, stabilisant le flux de calculs et optimisant l'utilisation des ressources.

5. Planification stratégique des ressources à long terme

En agrégeant les prédictions de temps d'exécution sur plusieurs jobs et utilisateurs, les administrateurs peuvent mieux prévoir les besoins futurs du cluster, adapter la capacité matérielle, et optimiser la maintenance ou l'extension du système.

Cette double prédiction augmente l'efficacité du cluster HPC, réduit les gaspillages et les délais, tout en facilitant l'utilisation par des personnes non experts. Ainsi, elle participe à un usage plus intelligent, plus durable et plus performant des infrastructures HPC.

Les bénéfices pour l'utilisateur et le cluster

Pour l'utilisateur :

- Assistance intelligente : plus besoin de maîtriser les détails techniques.
- Soumission simplifiée via une interface conviviale (Streamlit).
- Moins d'erreurs et de jobs annulés ou redémarrés.
- Visualisation des performances estimées avant lancement.

Pour le cluster :

- Optimisation de l'allocation des ressources.
- Réduction des temps d'attente dans la file d'exécution.
- Meilleure efficacité globale du système.
- Base solide pour une future intégration MLOps ou auto-scaling.

5.4. Conception du module IA

Pour intégrer efficacement l'intelligence artificielle dans la gestion des ressources du cluster HPC, une étape fondamentale consiste à constituer un jeu de données fiable et représentatif des comportements réels des jobs soumis. Cette donnée provient directement du système de gestion SLURM via les outils natifs qu'il offre.

5.4.1. Récupération des logs SLURM

a. Mise en place de la base de données SLURM avec MariaDB

Pour que la commande `sacct` puisse interroger l'historique des jobs exécutés, SLURM doit être configuré avec un système de gestion de base de données relationnelle (SGBDR) pour stocker les données d'accounting. Dans ce projet, le choix s'est porté sur MariaDB, une solution open source légère et performante, compatible avec MySQL.

b. Avantage de l'intégration MariaDB

Cette architecture permet :

- La centralisation de l'historique des jobs ;
- Une consultation fine des métriques d'utilisation des ressources ;
- L'extraction automatisée des données via sacct pour alimenter le modèle d'intelligence artificielle ;
- La pérennité des données même après redémarrage du contrôleur SLURM.

Ainsi, la base MariaDB constitue le socle fondamental de la collecte des données nécessaires à la prédiction des ressources SLURM par l'IA.

Les colonnes extraites via sacct sont les suivantes :

Colonne	Description
JobID	Identifiant unique du job
JobName	Nom du job défini par l'utilisateur
Partition	Partition dans laquelle le job a été exécuté
AllocCPUS	Nombre de cœurs CPU alloués Elapsed Durée d'exécution du job (format HH:MM:SS)
State	État final du job (COMPLETED, FAILED, etc.)
ReqMem	Quantité de mémoire demandée
Start, End	Horodatages de début et de fin d'exécution

Table 5-1 Les colonnes extraites via Sacct

Ces données sont extraites automatiquement grâce à un script Python intégré dans l'application Streamlit, qui interroge le cluster via sacct, récupère les logs, puis les enregistre dans un fichier CSV brut.

5.4.2. Données de production issues de l'ENAGEO

Dans le cadre de ce projet, une collaboration a été établie avec des ingénieurs-conseils de l'ENAGEO, filiale spécialisée dans l'acquisition et le traitement sismique. Ces experts ont fourni des fichiers extraits du cluster HPC de production, comprenant :

- Des logs SLURM réels générés via la commande sacct ;
- Des journaux d'exécution (exec9.log) montrant des traitements sismiques via le logiciel ProMAX/SeisSpace sur plus de 90 nœuds ;
- Des graphes internes illustrant la répartition des temps d'exécution.

Ces données ont permis de construire un jeu de données d'apprentissage **réaliste et représentatif**, renforçant la pertinence de l'approche IA développée.

```
[root@sws06 ~]# sacct -u a-adjou --starttime=2025-06-01 --format=JobID,JobName,State,Elapsed
JobID      JobName      State      Elapsed
-----
343122     TEST_04_L+   COMPLETED 03:01:24
343122.batch batch        COMPLETED 03:01:24
343122.0   hydra_pmi+  COMPLETED 01:06:52
343122.1   hydra_pmi+  COMPLETED 00:58:00
343122.2   hydra_pmi+  COMPLETED 00:56:11
343143     TEST_04_L+   COMPLETED 04:49:59
343143.batch batch        COMPLETED 04:49:59
343143.0   hydra_pmi+  COMPLETED 02:58:51
343143.1   hydra_pmi+  COMPLETED 01:21:47
343143.2   hydra_pmi+  COMPLETED 00:28:59
343153     TEST_04_L+   COMPLETED 04:16:49
343153.batch batch        COMPLETED 04:16:49
343153.0   hydra_pmi+  COMPLETED 02:24:10
343153.1   hydra_pmi+  COMPLETED 01:25:10
343153.2   hydra_pmi+  COMPLETED 00:27:16
343157     TEST_04_L+   FAILED     01:02:07
343157.batch batch        FAILED     01:02:07
343157.0   hydra_pmi+  FAILED     01:01:30
343177     TEST_04_L+   COMPLETED 00:17:50
343177.batch batch        COMPLETED 00:17:50
343177.0   hydra_pmi+  COMPLETED 00:17:42
343202     TEST_04_L+   COMPLETED 02:49:43
343202.batch batch        COMPLETED 02:49:43
343202.0   hydra_pmi+  COMPLETED 00:48:29
343202.1   hydra_pmi+  COMPLETED 01:35:34
343202.2   hydra_pmi+  COMPLETED 00:24:32
343268     TEST_04_L+   CANCELLED+ 00:53:42
343268.batch batch        CANCELLED 00:53:44
343268.0   hydra_pmi+  FAILED     00:53:47
343296     TEST_04_L+   COMPLETED 03:45:26
343296.batch batch        COMPLETED 03:45:26
343296.0   hydra_pmi+  COMPLETED 02:59:15
343296.1   hydra_pmi+  COMPLETED 00:40:15
343296.2   hydra_pmi+  COMPLETED 00:05:02
343321     TEST_08_S+   FAILED     00:10:59
343321.batch batch        FAILED     00:10:59
343321.0   hydra_pmi+  FAILED     00:10:46
343322     TEST_08_S+   CANCELLED+ 00:01:14
343322.batch batch        CANCELLED 00:01:45
343322.0   hydra_pmi+  CANCELLED 00:01:07
343373     TEST_08_S+   COMPLETED 01:46:15
343373.batch batch        COMPLETED 01:46:15
343373.0   hydra_pmi+  COMPLETED 01:45:58
343502     TEST_08_S+   COMPLETED 00:01:34
343502.batch batch        COMPLETED 00:01:34
343502.0   hydra_pmi+  COMPLETED 00:01:28
343503     TEST_08_S+   COMPLETED 00:01:03
343503.batch batch        COMPLETED 00:01:03
343503.0   hydra_pmi+  COMPLETED 00:00:47
343671     -SEG_Y@de+  FAILED     00:00:22
343671.batch batch        FAILED     00:00:22
```

Figure 5-6 Exemple de logs SLURM réels de l'utilisateur a-adjou à l'ENAGEO

Cette capture montre une partie des logs SLURM extraits du cluster HPC de l'ENAGEO pour l'utilisateur a-adjou. On y observe plusieurs exécutions du job TEST 04 L+, avec des durées comprises entre **17 minutes** et **près de 5 heures**, et des états variés (COMPLETED, FAILED, CANCELLED).

Cette hétérogénéité met en évidence la complexité de la planification des ressources dans un environnement HPC réel. Elle justifie le recours à des techniques de **prédiction intelligente** afin d'anticiper le temps d'exécution et les ressources à allouer, en fonction de caractéristiques spécifiques du job (nom, taille des données, partition, etc.).

5.4.3. Prétraitement des données

Les logs bruts contiennent parfois des données inutilisables, redondantes ou mal formatées. Un traitement automatique est donc appliqué pour garantir la qualité et la fiabilité du jeu de données. Ce processus de nettoyage suit les étapes suivantes :

a. Suppression des sous-jobs

Certains jobs SLURM sont composés de plusieurs sous-jobs (étapes internes), identifiables par des JobID contenant un point (ex : 12345.1). Ces sous-jobs sont supprimés pour ne conserver que les entrées principales (12345), pertinentes pour l'analyse globale.

b. Filtrage des états

Seuls les jobs dont l'état est COMPLETED ou FAILED sont conservés, car ce sont les seuls états offrant une information utile pour l'apprentissage (durée d'exécution, consommation de ressources, succès ou échec).

c. Conversion du temps

Le champ Elapsed, initialement en format texte (HH:MM:SS), est converti en secondes via une fonction Python. Cela permet d'obtenir une variable numérique continue, nommée Elapsed_sec, directement utilisable dans les modèles de régression.

d. Transformation de la mémoire

Le champ ReqMem est souvent exprimé en unités diverses (ex : 32G, 1024M). Une normalisation est appliquée pour convertir toutes les valeurs en mégaoctets (MB), facilitant l'analyse. Le champ est renommé ReqMem_MB.

e. Nettoyage des lignes invalides

Les lignes comportant des champs vides, des erreurs de format ou des valeurs manquantes sont automatiquement éliminées pour garantir la qualité du dataset.

f. Détection des anomalies

Des règles simples sont appliquées pour détecter les valeurs aberrantes :

- Jobs avec Elapsed_sec = 0 ou des durées excessives ;
- AllocCPUS = 0 ou supérieur à une limite logique ;
- ReqMem_MB incohérente avec la partition (gpu avec très peu de mémoire, par exemple).

g. Suppression des doublons

Certaines lignes peuvent apparaître plusieurs fois à l'identique dans les logs historiques. Ces doublons sont supprimés automatiquement pour éviter un biais dans les modèles.

Une fois nettoyés, les logs sont enregistrés dans un fichier structuré nommé slurm_logs_clean.csv, qui constitue la base d'apprentissage pour les modèles de régression du module IA.

5.4.4. Sélection des variables pertinentes (features)

Une fois les logs SLURM nettoyés et structurés, une phase d'analyse exploratoire a permis d'identifier les variables (features) les plus significatives pour la prédiction des ressources et du temps d'exécution. Cette sélection est cruciale car elle détermine la capacité du modèle à généraliser sur de nouveaux jobs.

a. Variables d'entrée sélectionnées

Les variables retenues pour l'entraînement du modèle sont à la fois des caractéristiques techniques du job, des choix utilisateur, et des données calculées à partir des champs bruts. Elles comprennent :

Variable	Description
ReqMem_MB	Mémoire demandée en mégaoctets
InputSize_GB	Taille des données d'entrée (ajoutée via saisie ou déduction)
Partition	Partition SLURM utilisée (cpu, gpu, etc.)
JobName	Type de traitement (seismic, reservoir, geology, etc.)
MemPerCPU	Quantité de mémoire demandée par cœur (ReqMem_MB / AllocCPUS)
InputPerCPU	Taille des données par CPU (InputSize_GB / AllocCPUS)

Table 5-2 Description des variables d'entrée pour les jobs SLURM

Ces variables ont été choisies pour leur corrélation avec les deux cibles à prédire :

- AllocCPUS (nombre optimal de CPU)
- Elapsed_sec (durée d'exécution)

b. Encodage des variables catégorielles

Les variables Partition, JobName et ProcessingStep, de type catégoriel, sont converties via encodage One-Hot afin de les transformer en variables numériques binaires. Cela permet aux modèles de les interpréter sans leur attribuer de relation d'ordre arbitraire.

c. Importance des variables

Une fois les modèles entraînés, une analyse de l'importance des variables (feature importance) est réalisée, notamment avec les modèles **Random Forest** et **XGBoost**, qui permettent de mesurer automatiquement l'influence de chaque variable sur la prédiction.

Les résultats montrent que certaines variables comme InputSize_GB ou MemPerCPU ont un impact direct sur le nombre de CPU requis et le temps d'exécution. Cela confirme la pertinence des choix effectués.

5.4.5. Objectifs de prédiction : NCPUS et ElapsedTime

Le module d'intelligence artificielle développé dans ce projet vise à prédire deux éléments fondamentaux pour le bon fonctionnement d'un cluster SLURM :

1. Le nombre optimal de CPU (NCPUS) à allouer à un job, afin d'assurer un bon compromis entre performance et utilisation efficace des ressources.
2. Le temps d'exécution estimé (ElapsedTime) du job, ce qui permet à l'ordonnanceur SLURM d'optimiser la planification globale des tâches.

Ces deux objectifs sont abordés sous forme de **problèmes de régression**, c'est-à-dire des prédictions numériques continues, à partir des caractéristiques des jobs extraites ou déduites des logs SLURM.

a. Prédiction du nombre de CPU (NCPUS)

Le nombre de cœurs à allouer à un job influence fortement sa vitesse d'exécution, mais aussi la charge sur le cluster. Une mauvaise estimation peut entraîner :

- Des **surcharges** inutiles (gaspillage de ressources),
- Des **sous-allocations** qui ralentissent ou font échouer le job.

Le modèle IA tente de prédire automatiquement la valeur optimale de **NCPUs** en fonction de plusieurs facteurs, tels que :

- La mémoire demandée (ReqMem_MB)
- La taille des données (InputSize_GB)
- Le type de traitement (JobName, ProcessingStep)
- La partition utilisée (Partition)
- Les ratios MemPerCPU et InputPerCPU

b. Prédiction de la durée d'exécution (ElapsedTime)

Le champ ElapsedTime, converti en secondes (Elapsed_sec), représente la durée réelle d'exécution d'un job. Sa prédiction est essentielle pour :

- Anticiper la charge du cluster
- Améliorer l'ordonnancement des jobs par SLURM
- Fournir à l'utilisateur une estimation fiable de la durée

Le modèle s'appuie sur les mêmes variables d'entrée que pour la prédiction des CPU, car elles influencent également le temps nécessaire à l'exécution :

- Une taille de données plus importante
- Un traitement plus complexe
- Une mémoire insuffisante peut rallonger la durée d'un job.

c. Approche de double prédiction

Les deux objectifs sont traités **indépendamment** par des modèles de régression entraînés séparément. Cela permet :

- Optimiser la précision sur chaque cible
- Choisir éventuellement des modèles différents selon la métrique visée (par exemple : Random Forest pour NCPUSet XGBoost pour ElapsedTime).

Ces prédictions sont ensuite utilisées pour alimenter l'application Streamlit « Prédicteur SLURM Intelligent », qui fournit des recommandations claires à l'utilisateur avant la soumission de son job.

5.5. Entraînement et sélection des modèles

Après la préparation du jeu de données et la définition des objectifs de prédiction, l'étape suivante consiste à entraîner plusieurs modèles de régression supervisée pour prédire :

- Le nombre optimal de cœurs CPU (NCPUS)
- La durée d'exécution d'un job (ElapsedTime).

Cette étape repose sur une approche comparative, consistant à entraîner plusieurs modèles sur les mêmes données, à les évaluer via des métriques communes, puis à choisir automatiquement le modèle offrant la meilleure performance pour chaque cible.

5.5.1. Modèles de régression utilisés

Trois modèles de machine learning ont été sélectionnés pour leur robustesse, leur capacité à gérer les variables numériques et catégorielles, et leur performance éprouvée dans des contextes réels :

- **Random Forest Regressor**
 - Algorithme basé sur un ensemble d'arbres de décision.
 - Très efficace pour capturer des relations non linéaires.
 - Résistant au surapprentissage grâce à l'agrégation de plusieurs arbres.
- **XGBoost Regressor**
 - Implémentation performante du gradient boosting.
 - Très populaire en compétition et dans l'industrie.
 - Capable de traiter des données bruitées et hétérogènes avec une grande précision.
- **Support Vector Regressor (SVR)**
 - Méthode de régression utilisant des fonctions noyau pour modéliser des relations complexes.
 - Particulièrement utile pour capturer les tendances globales et fines dans les données.
 - Moins sensible aux valeurs extrêmes que les arbres.

5.5.2. Méthodologie d'entraînement

Chaque modèle est entraîné de manière indépendante pour les deux cibles (NCPUS, ElapsedTime), selon une procédure rigoureuse :

- **Séparation du jeu de données**
 - Le dataset est divisé en deux sous-ensembles :
 - **80 % pour l'entraînement**
 - **20 % pour le test** (validation finale)
 - La séparation est réalisée de manière aléatoire mais reproductible.
- **Prétraitement appliqué à chaque sous-ensemble**
 - Encodage One-Hot des variables catégorielles.
 - Vérification de la distribution des données (pas de fuite d'information).

- **Ajustement des hyperparamètres**
 - Des paramètres standards sont utilisés dans un premier temps.
 - Un ajustement automatique est envisageable via GridSearchCV (perspective future)

5.5.3. Sélection automatique du meilleur modèle

Pour chaque objectif (NCPUS, ElapsedTime), les trois modèles sont comparés entre eux selon la moyenne pondérée des métriques. Le modèle offrant les meilleures performances globales est retenu.

5.5.4. Évaluation des performances

Deux métriques principales sont utilisées pour évaluer la qualité des modèles :

Métrique	Description
MAE (<i>Mean Absolute Error</i>)	Erreur moyenne absolue entre la prédiction et la réalité. Faible MAE = prédictions proches. $MAE = \frac{1}{n} \sum_{i=1}^n y_i - \hat{y}_i $ • y_i : valeur réelle (observée) • $\hat{y}_i$: valeur prédite • n : nombre total d'observations
R² Score (<i>Coefficient de détermination</i>)	Représente la proportion de la variance expliquée. Proche de 1 = très bon modèle. $R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$ • y_i: valeur réelle • $\hat{y}_i$: valeur prédite • $\bar{y}$: moyenne des valeurs réelles • n : nombre d'observations

Table 5-3 Tables De Métriques

Ce modèle est ensuite :

- Affiché dans l'interface utilisateur avec son nom,
- Utilisé pour effectuer les prédictions dans l'application.

Afin de compléter l'analyse quantitative des modèles, des **graphiques de performance** ont été réalisés pour visualiser et comparer les résultats des différents algorithmes testés.

Ces visualisations permettent d'identifier rapidement le modèle le plus fiable pour chaque cible de prédiction (NCPUS et ElapsedTime), à travers deux indicateurs clés :

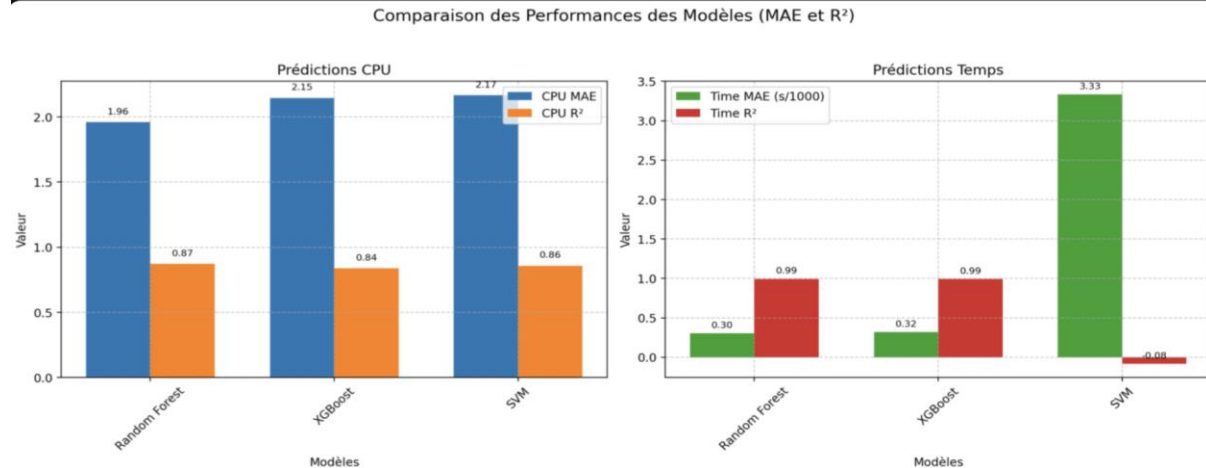


Figure 5-7 Évaluation comparative des modèles de Machine Learning sur des données simulées

Les résultats illustrés dans la figure ci-dessus permettent de comparer les performances respectives des modèles Random Forest, XGBoost et SVM sur deux objectifs de prédiction : le nombre de CPU (NCPUS) et le temps d'exécution (ElapsedTime).

Pour la prédiction du nombre de CPU, l'ensemble des modèles obtient des performances acceptables, avec un léger avantage pour Random Forest, qui affiche la plus faible erreur absolue moyenne (MAE = 1.96) et le meilleur coefficient de détermination (R² = 0.87). Cela indique une bonne capacité du modèle à approximer les valeurs réelles tout en conservant une certaine robustesse.

En ce qui concerne la prédiction du temps d'exécution, les résultats sont plus contrastés. Random Forest et XGBoost montrent une très bonne performance avec un R² proche de 0.99 et un MAE faible (respectivement 0.30 et 0.32), démontrant leur aptitude à modéliser avec précision cette variable. En revanche, le modèle SVM présente une forte erreur de prédiction (MAE = 3.33) et un R² négatif (-0.08), indiquant une incapacité à capturer correctement la structure des données dans ce contexte.

Il est important de noter que les résultats obtenus dans cette section proviennent de **données simulées** et non de **logs réels** issus du cluster.

Cette approche a été adoptée pour plusieurs raisons :

- **Indisponibilité ou accès limité aux logs SLURM** au moment de la phase expérimentale. Les données réelles de production sont souvent protégées pour des raisons de sécurité, de confidentialité ou de réglementation interne.
- **Phase exploratoire** : l'objectif initial était de tester la faisabilité du modèle et d'explorer les performances des algorithmes de manière contrôlée. La simulation permet de créer un jeu de données propre, équilibré et bien structuré, facilitant l'entraînement et l'évaluation préliminaire des modèles.

5.6. Déploiement dans l'application « Prédicteur SLURM Intelligent »

Une fois les modèles entraînés et évalués, ils sont intégrés dans une interface utilisateur accessible via une application web développée avec **Streamlit**. Cette application, baptisée « **Prédicteur SLURM Intelligent** », permet de prédire dynamiquement les ressources nécessaires à un job SLURM, tout en affichant les performances des modèles utilisés.

5.6.1. Interface utilisateur avec Streamlit

L'interface de l'application a été conçue pour être à la fois **intuitive**, **légère** et **fonctionnelle**. Elle repose sur les éléments suivants :

- **Barre latérale (sidebar) :**
 - Permet à l'utilisateur de saisir les caractéristiques de son job :
 - Taille des données d'entrée (InputSize_GB)
 - Type de traitement (JobName)
 - Partition SLURM (Partition)
 - Chaque champ est prérempli ou guidé pour minimiser les erreurs de saisie.
- **Onglets principaux :**
 - Onglet "Prédictions" :
 - Affiche le nombre recommandé de CPU (NCPUS)
 - Affiche le temps estimé d'exécution (ElapsedTime)
 - Indique le modèle utilisé pour chaque prédiction
 - Onglet "Performance des modèles" :
 - Affiche les métriques MAE, R^2 pour chaque modèle testé
 - Permet une compréhension transparente du fonctionnement du système

5.6.2. Processus de prédiction automatisé

Dès qu'un utilisateur entre ses paramètres dans la barre latérale, l'application déclenche le pipeline suivant :

1. Les paramètres sont collectés sous forme de dataframe.
2. Les transformations nécessaires (encodage One-Hot, normalisation éventuelle) sont appliquées.
3. Le meilleur modèle (sélectionné lors de l'entraînement) est chargé et utilisé pour la prédiction.
4. Les résultats sont immédiatement affichés dans l'interface.

Aucune interaction manuelle avec les logs, ni aucune connaissance préalable du ML, n'est requise de la part de l'utilisateur.

5.6.3. Affichage des performances du modèle

L'application met en valeur la transparence et la fiabilité de ses prédictions en affichant les performances de chaque modèle :

- Un tableau comparatif indique :
 - Le nom de chaque algorithme testé (Random Forest, XGBoost, SVR)
 - Les scores obtenus sur les métriques : MAE, R^2
- Le **meilleur modèle est automatiquement mis en évidence**, avec un encadré de type :
« Meilleur modèle pour la prédiction des CPUs : XGBoost (MAE = 1.23, $R^2 = 0.98$) »

Cela renforce la confiance des utilisateurs dans le système prédictif.

5.7. Résultats et cas de test

Afin d'évaluer la pertinence des prédictions générées par le module d'intelligence artificielle, plusieurs cas de test ont été simulés via l'interface utilisateur. Les résultats obtenus ont été comparés aux valeurs réelles enregistrées dans les logs SLURM, afin de mesurer la précision des prédictions dans des situations concrètes.

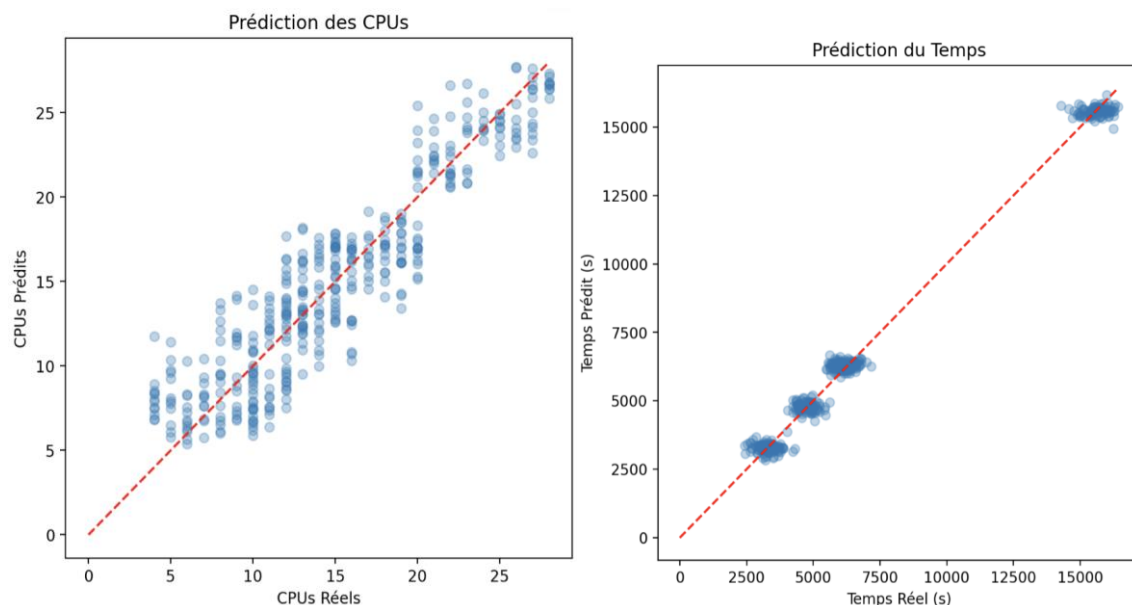


Figure 5-8 Comparaison des Prédictions vs Valeurs Réelles : CPU et Temps

Les graphiques montrent une corrélation positive entre les valeurs prédites et réelles pour les CPU et le temps, avec une ligne de régression indiquant une tendance générale correcte. Cependant, la dispersion des points, notamment pour la prédiction du temps, suggère une variabilité qui pourrait indiquer des limites dans la précision du modèle, surtout pour les valeurs élevées. Les cas de test simulés via l'interface utilisateur et la comparaison avec les logs SLURM semblent confirmer une certaine fiabilité, mais des ajustements pourraient être nécessaires pour améliorer la précision, particulièrement dans les scénarios extrêmes.

5.8. Analyse des écarts et erreurs

L'analyse des écarts met en évidence les points suivants :

- Les modèles offrent des **prédictions fiables dans 80 à 90 % des cas**, notamment lorsque les données d'entrée (taille, mémoire, partition) sont cohérentes avec celles du jeu d'entraînement.
- Les **erreurs les plus importantes** surviennent dans des cas particuliers :
 - Jobs très courts ou très longs (valeurs extrêmes),
 - Combinaisons rares (ex. : job fluid sur partition gpu),
 - Jobs ayant échoué (FAILED) avec un comportement imprévisible.

Ces erreurs restent cependant **limitées en proportion** et n'affectent pas significativement la qualité globale du système.

5.9. Interface utilisateur : Streamlit

Streamlit est un framework open source en Python prévu pour générer rapidement des interfaces web interactives, dédiées à la représentation de données, au prototypage de modèles d'apprentissage automatique, et à la conception d'outils d'analyse. Depuis son lancement en 2019, Streamlit s'est affirmé comme un outil privilégié pour les scientifiques de données, les chercheurs, les ingénieurs ou toute personne désireuse de convertir des scripts Python en applications web faciles à utiliser, sans nécessiter de compétences en développement web. [34]

5.9.1. Objectifs de Streamlit

Streamlit vise essentiellement à faciliter la mise en production d'applications axées sur les données. À l'opposé des frameworks web conventionnels tels que Flask ou Django, Streamlit s'appuie sur une approche déclarative permettant à l'utilisateur de concevoir une application comme il le ferait pour un script d'analyse. Cela offre la possibilité d'obtenir un prototype opérationnel en quelques lignes de code, avec une interactivité instantanée. [34]

Streamlit suit deux principes fondateurs :

- **Interactivité automatique** : les widgets d'entrée (boutons, sliders, menus déroulants) sont connectés directement aux variables Python.
- **Rapidité** : l'actualisation de l'interface est rapide grâce à un système intelligent de redéclenchement du script lors d'une interaction utilisateur.

5.9.2. Caractéristiques techniques de Streamlit

- Élaboré en Python pur, sans nécessité d'avoir recours au HTML, CSS ou JavaScript.
- Propose une vaste sélection de composants interactifs : zones de texte, boutons, curseurs, menus dépliables, chargement de fichiers, graphiques, et plus encore. Compatible avec les bibliothèques les plus courantes de l'univers Python : Pandas, NumPy, Matplotlib, Plotly, Scikit-learn, TensorFlow, entre autres.

- Facilite l'affichage de données en direct, ce qui est particulièrement avantageux pour le suivi d'expérimentations, la surveillance des performances ou l'interaction avec des modèles d'apprentissage automatique.
- Propose des alternatives de déploiement sur site ou dans le cloud (à travers Streamlit Cloud, Docker, ou des services tels que Heroku ou AWS). [34]

5.9.3. Avantages de Streamlit

- Vitesse de développement : il est possible de créer une application en quelques minutes à partir d'un script Python déjà existant.
- Réduction de la courbe d'apprentissage : pas besoin de maîtriser le développement web traditionnel.
- Parfait pour les démonstrations ou les prototypes interactifs.
- Représentation aisée des résultats, indispensable pour l'explicabilité des modèles d'apprentissage automatique ou l'analyse de données.
- Communauté active : Streamlit est soutenu par une communauté vaste et une documentation approfondie. [34]

5.9.4. Cas d'usage typiques

- Interfaces pour lancer des prédictions avec des modèles de machine learning entraînés.
- Tableaux de bord interactifs pour explorer des jeux de données.
- Visualisation en temps réel de résultats d'analyses scientifiques.
- Interfaces simplifiées pour accéder à des systèmes complexes (HPC, SLURM, bases de données, API). [34]

Streamlit a été utilisé pour offrir une interface utilisateur simple, intuitive et interactive, qui permet d'interagir avec le cluster HPC, d'exploiter des modèles d'apprentissage automatique, et de visualiser les performances du système en temps réel. Cette approche améliore significativement l'accessibilité du système pour les utilisateurs finaux, tout en intégrant des fonctionnalités avancées pilotées par l'IA. [34]

5.10. Déploiement du module IA via une interface

Streamlit

L'application Streamlit suit une logique séquentielle claire, découpée en quatre étapes principales :

1. **Téléchargement des logs SLURM** via la commande `sacct`.
2. **Nettoyage et transformation des données** en un format exploitable.
3. **Entraînement des modèles IA** (CPU et temps).
4. **Prédiction des ressources** à partir d'un formulaire simplifié.

Chaque étape est déclenchée par un bouton et accompagnée de messages visuels (succès, avertissement, erreur), facilitant la compréhension du processus. L'utilisateur peut

également prévisualiser les données à chaque phase (logs bruts, données nettoyées, performances des modèles).

5.10.1. Prédiction interactive pour l'utilisateur

L'utilisateur peut effectuer une prédiction en renseignant trois paramètres uniquement :

- Le **nom du job**.
- La **partition** (`cpu` ou `gpu`).
- La **mémoire demandée** (valeur ajustable via un curseur en Mo).

Le système effectue alors deux prédictions consécutives :

- Le **nombre optimal de CPU** à allouer (premier modèle),
- Le **temps estimé d'exécution** du job (second modèle, utilisant le CPU prédit).

Cette approche enchaînée reflète le processus réel de soumission SLURM et garantit une meilleure cohérence entre les paramètres.

5.10.2. Retour visuel et indicateurs de performance

L'interface intègre des retours visuels clairs pour renforcer l'interprétabilité :

- Affichage des **logs SLURM extraits** et **fichiers nettoyés**,
- Présentation des **résultats de performance** des modèles (MAE et R^2),
- Affichage des **prédictions finales** sous forme de messages explicites :
 - « CPU recommandé : 4 »
 - « Temps estimé : 12.4 minutes »

5.10.3. Limites et axes d'amélioration

Il est important de noter que, bien que le module d'intelligence artificielle soit pleinement fonctionnel, il ne sera pas encore utilisé en production dans l'immédiat. En effet, le cluster est récent et ne dispose pas encore d'un historique de logs SLURM suffisant pour entraîner les modèles sur des données réelles avec une fiabilité optimale.

Lors d'un échange avec un ingénieur HPC chez SONATRACH, il a été confirmé que le nombre de jobs exécutés varie fortement, entre 100 et 1000 jobs par jour, selon la charge de travail. Selon les bonnes pratiques en apprentissage supervisé, et en particulier dans les contextes de gestion de workloads complexes comme ceux observés en HPC, un minimum de 10 000 à 20 000 jobs nettoyés est généralement requis pour entraîner un modèle de régression capable de généraliser efficacement [Gharaibeh et al., 2021]. Cette estimation est également soutenue par des recommandations générales du domaine, indiquant qu'un nombre important d'exemples est nécessaire en présence de variabilité et de bruit [Géron, 2019].

Si le rythme moyen d'exécution est de 500 jobs/jour, un historique de 30 à 45 jours permettrait d'atteindre ce seuil de manière réaliste.

En attendant, le module IA est utilisé de manière exploratoire pour démontrer sa faisabilité et préparer son intégration dynamique, dès que le volume de données du cluster sera suffisant.

5.11. Conclusion

Ce chapitre a présenté l'intégration d'un module d'intelligence artificielle dans le cluster HPC, visant à prédire le nombre de CPU et la durée d'exécution d'un job SLURM. Grâce à l'exploitation des logs réels du cluster, un pipeline complet de traitement, d'entraînement et de déploiement a été mis en place à l'aide de modèles de régression (Random Forest, XGBoost, SVR) et d'une interface Streamlit conviviale.

Les résultats sont prometteurs, bien que le modèle ne soit pas encore utilisé en production en raison du volume limité de données historiques. Une mise en service effective est envisagée dès que le cluster aura accumulé suffisamment de jobs pour garantir une prédiction fiable.

L'un des points forts de ce projet réside dans l'utilisation de données réelles de production fournies par des ingénieurs de l'ENAGEO. Contrairement à une approche purement simulée, cette intégration renforce la **validité terrain** du modèle développé, qui pourrait être adapté et déployé dans un environnement opérationnel pour aider à la planification dynamique des ressources.

Chapitre 6. Evaluation et discussion

6.1. Introduction

Dans ce chapitre, on va expérimenter sur le système. Tout d'abord, on définit le cas de test ainsi que son implémentation, les métriques d'évaluation et la configuration matérielle utilisée. On finalise le chapitre en discutant et évaluant les résultats obtenus.

6.2. Cas d'étude : Traitement de données sismiques

Les données sismiques sont des enregistrements des ondes acoustiques réfléchies ou réfractées dans le sous-sol, obtenus à l'aide de sources sismiques et de récepteurs positionnés à la surface ou en fond marin. Ces données brutes contiennent non seulement le signal utile (réflexions des couches géologiques), mais aussi divers types de bruit (bruit de surface, bourdonnement des moteurs, réverbérations, etc.....) [16]

Pour améliorer la qualité des données sismiques, il est nécessaire d'appliquer diverses opérations permettant de réduire les signaux indésirables. L'atténuation des signaux indésirables permet d'optimiser le rapport signal/bruit, rendant les images sismiques plus lisibles et interprétables pour les géophysiciens et les géologues lors de l'analyse des structures du sous-sol.

Dans ce cas de test, on va choisir l'opération de migration en profondeur de données sismiques avec le logiciel **Seismic Unix** et le déployer sur une plateforme HPC. Cette opération sera appliquée sur les fichiers sismiques prétraités afin de replacer les événements sismiques à leur position géométrique correcte dans le sous-sol et Améliore la résolution de l'image pour que le géophysicien puisse l'analyser. La figure 6-1 illustre un exemple théorique d'une trace sismique avant et après migration.

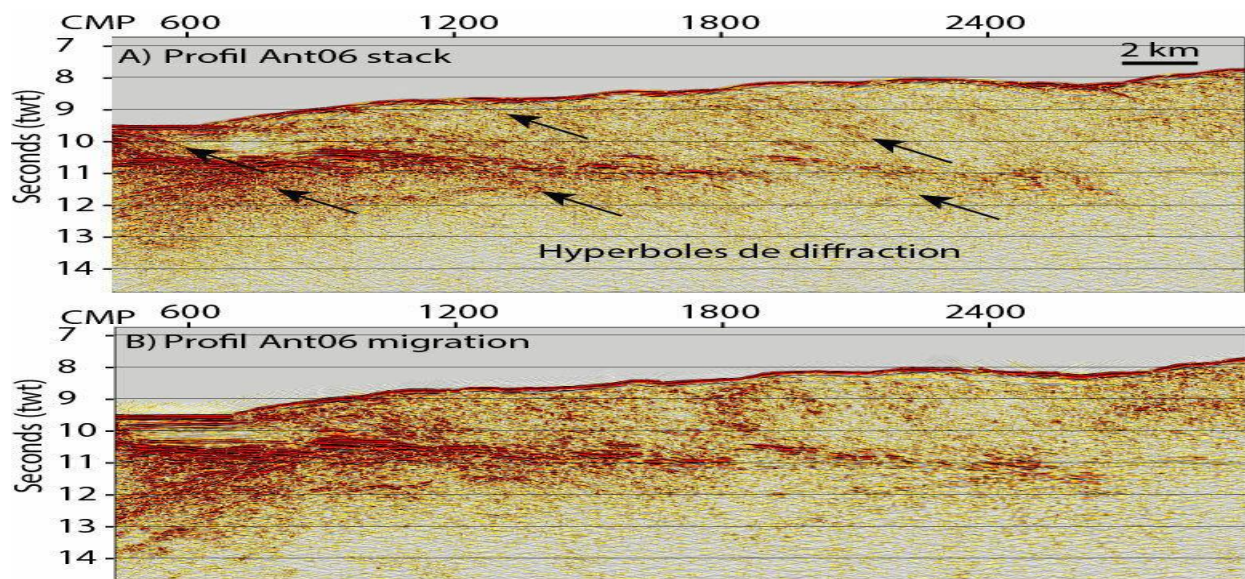


Figure 6-1 Image sismique avant et après migration

6.3. Conception du Workflow

La migration en profondeur est le processus qui consiste à repositionner les événements sismiques à leur emplacement réel en profondeur, en utilisant un modèle de vitesses. Elle permet de corriger les structures géologiques complexes, offrant ainsi une image plus précise du sous-sol [14] comme été illustré dans figure précédente.

Dans Seismic Unix l'opération de migration Fait Par la Command

sumigfd < Input.su dt= dx= dz= nz= vfile=vitesse.su > output.su

- **Sumigfd** : le nom du programme utilisé.
- **Input.su** : un fichier d'entre au format **Seismic Unix** (.su) contenant les données sismiques à migrer.
- **dt** : il s'agit du pas d'échantillonnage temporel des données en secondes (par exemple, dt=0.004 pour 4 ms).
- **dx** : il s'agit du pas d'échantillonnage spatial en X (distance entre les traces sismiques sur la surface), exprimé en mètres.
- **dz** : il s'agit du pas d'échantillonnage en profondeur, c'est-à-dire la hauteur de grille en profondeur (z), exprimé en mètres.
- **nz** : c'est le nombre total d'échantillons en profondeur (z), cela définit la profondeur maximale modélisée :

$$\text{profondeur_max} = \text{nz} * \text{dz}.$$

- **vfile** : c'est le fichier de modèle de vitesse, généralement un fichier .su contenant la vitesse du son (ondes P) en fonction de la profondeur et de la position horizontale.
- **output.su** : est le fichier de sortie contenant les données migrées (donc repositionnées correctement).

L'opération **Sumigfd** est une opération mono-thread. Pour la rendre compatible avec un environnement multi-thread, nous avons créé un script qui divise le fichier d'entrée en **n** parties, où **n** est le nombre de cœurs de processeur choisi par l'utilisateur. Ensuite, chaque partie est traitée en parallèle sur un cœur. Enfin, les résultats sont regroupés

6.4. Protocole de Test

Les tests seront réalisés sur différentes configurations en faisant varier à la fois la taille des données en entrée et le nombre d'unités de calcul (CPU). Pour établir une référence de comparaison, toutes les combinaisons de tailles d'entrée seront d'abord exécutées sur une seule unité de calcul, ce qui correspond à une exécution séquentielle sans aucun parallélisme. Ces tests seront ensuite reproduits sur plusieurs nœuds disposant de ressources variées (CPU et

mémoire) et à des dates différentes, afin d'évaluer l'impact de la configuration matérielle sur les performances.

6.4.1. Données de Test

Pour garantir la qualité et la représentativité des données utilisées dans cette étude, une sélection rigoureuse de fichiers de données sismiques a été effectuée à partir de The National Archive of Marine Seismic Surveys (NAMSS). Le NAMSS est une ressource librement accessible, spécialisée dans l'archivage et la diffusion de données sismiques marines acquises dans le cadre de campagnes géophysiques. Cette base offrant un accès à un large éventail d'enregistrements sismiques utilisés notamment pour l'exploration géologique et la recherche scientifique.

Les fichiers sélectionnés sont au format SEG-Y (SGY), un standard largement utilisé pour le stockage de données sismiques, contenant à la fois les signaux bruts et les métadonnées associées. Pour cette étude, 4 fichiers SEG-Y distincts ont été retenus, avec des tailles variant de 6 Mo à 100 Mo, représentant différents profils sismiques marins issus de diverses zones géographiques.

6.4.2. Infrastructure Matérielle des Tests

Après avoir défini le test, on va procéder à la spécification du matériel utilisé.

APPAREIL	CPU	MEMOIRE	STOCKAGE	ROLE
Master Dell PowerEdge R720	Intel Xeon E5-2620 v2 @ 2.10GHz	256 Go	1To + 256 Go	Contrôle-Node Management- Beegfs-Node Metadata-Beegfs Node Client-Beegfs Node Supervision- Grafana Node
Node1 Dell PowerEdge R720	Intel Xeon E5-2620 v2 @ 2.10GHz (4 cœurs)	64 Go	1To + 256 Go	Calcul-Node Storage-Beegfs Node Client-Beegfs Node

Node2 Dell PowerEdge R720	Intel Xeon E5-2620 v2 @ 2.10GHz (24 cœurs)	64 Go	1To + 256 Go	Calcul-Node Storage-Beegfs Node Client-Beegfs Node
---------------------------------	---	-------	--------------------	---

Table 6-1 Matérielle de Test

En raison de contraintes en ressources matérielles et dans une optique d'optimisation des performances, nous avons choisi de configurer l'ensemble des serveurs de manière qu'ils assurent également le rôle de serveur de stockage, en complément de leurs fonctions principales.

Cependant, dans un déploiement en production, il est fortement recommandé d'assigner un serveur exclusivement dédié au stockage, afin de garantir une meilleure isolation des charges de travail et une gestion plus efficace des ressources.

Concernant l'infrastructure réseau, nous avons opté pour l'utilisation d'un commutateur Huawei S5720, assurant une interconnexion de type Fast Ethernet avec un débit nominal de 1 Gb/s.

Pour améliorer notre réseau, nous avons déployé le protocole LACP (Link Aggregation Control Protocol) sur trois interfaces réseau de chaque serveur.

Le protocole LACP, défini par la norme IEEE 802.3ad, autorise l'agrégation dynamique de plusieurs liaisons physiques en une seule interface logique. Il facilite également la désagrégation automatique en cas de changement de l'état des liens.

En cas de défaillance d'un lien, LACP sélectionne automatiquement un autre lien disponible au sein du groupe, assurant ainsi la continuité du service et le maintien de l'équilibre de charge.

Cette solution permet d'augmenter la bande passante et d'améliorer la résilience du réseau, sans nécessiter de mise à niveau matérielle. [35]

6.4.3. Les Objectifs De Test

- Tester l'utilisabilité du cluster HPC.
- Évaluer les performances de calcul du cluster HPC

6.5. Evaluation

6.5.1. Métrique D'évaluation

Afin d'obtenir une interprétation solide des résultats, on va utiliser les mesures de Performance suivante :

Speed up : Lors de l'évaluation d'un système parallèle, il est essentiel de mesurer le gain de performance obtenu grâce à la parallélisation d'une application par rapport à son exécution séquentielle.

La notion d'accélération (speedup), notée S , permet de quantifier ce gain. Elle correspond au rapport entre le temps d'exécution sur une seule unité de calcul et le temps nécessaire pour résoudre le même problème sur un système parallèle comportant p unités de calcul identiques [36]

$$Speedup = Ts/Tp$$

Efficacité : Lors de l'exécution d'un algorithme parallèle, les unités de traitement ne peuvent pas consacrer 100 % de leur temps aux calculs effectifs : une partie du temps est perdue en attente (inactivité) ou en communication, comme illustré dans l'exemple de figure 6-2.

Pour évaluer l'utilisation réelle des ressources, on utilise la notion d'efficacité (*efficiency*), notée E , qui représente la proportion du temps pendant lequel une unité de traitement est activement utilisée pour effectuer des calculs utiles [36]. Elle est définie comme le rapport entre l'accélération obtenue et le nombre total d'unités de traitement :

$$E = \frac{S}{P}$$

Dans un système idéal, où l'accélération est parfaitement linéaire, l'efficacité est égale à 1. En pratique, elle est généralement inférieure à 1, et dépend du niveau d'optimisation de l'utilisation des ressources parallèles.

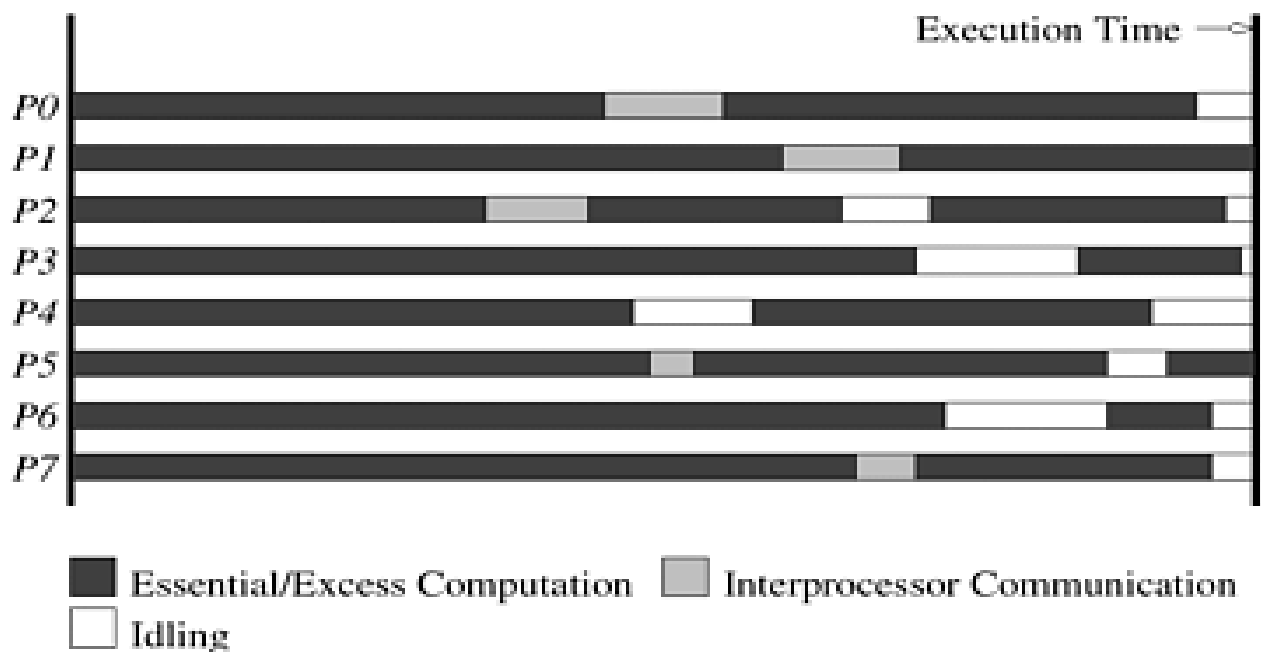


Figure 6-2 Analyse des performances : Calcul, Communication et Inactivité

La puissance de calcul d'un système informatique :

La puissance de calcul d'un système informatique désigne sa capacité à effectuer des opérations arithmétiques et logiques sur des données, notamment pour résoudre des problèmes complexes en un temps donné.

Pour quantifier cette puissance, on utilise souvent la métrique **MFLOPS** (*Méga Floating Point Operations Per Second*), qui mesure le nombre des millions d'opérations en virgule flottante qu'un système peut exécuter par seconde. [37]

De la même manière, "Gflop/s" correspond à des milliards d'opérations par seconde, et "Tflop/s" à des billions (ou milliers de milliards) d'opérations en virgule flottante par seconde.

Les opérations en virgule flottante sont l'addition et la multiplication sur des nombres réels (avec décimales).

Les opérations comme la division, racine carrée ou fonctions trigonométriques ne sont pas comptées car :

- Elles sont lentes,
- Elles ne contribuent que peu aux performances réelles dans les programmes bien optimisés.

Afin d'estimer concrètement le nombre d'opérations en virgule flottante qu'un système peut exécuter, on fait appel au **benchmark LINPACK**.

Le LINPACK Benchmark est un test standardisé de performance qui mesure la capacité d'un système informatique à résoudre un système d'équations linéaires de la forme

$$Ax = b$$

Où A est une matrice carrée dense, en utilisant une décomposition LU avec pivot partiel. Le test évalue le nombre total d'opérations en virgule flottante nécessaires, puis mesure le temps d'exécution, ce qui permet de calculer le débit de calcul exprimé en **FLOPS**.

Ce benchmark est largement utilisé dans le domaine du calcul haute performance (HPC) pour comparer la puissance de différents systèmes, notamment à travers le classement TOP500 des supercalculateurs les plus performants au monde.

Le LINPACK Benchmark repose sur le logiciel LINPACK, qui est une collection de sous-programmes Fortran conçus pour analyser et résoudre divers types de systèmes d'équations linéaires [38]

6.5.2. Résultats

Tailles de fichier	Temps d'exécution séquentielle (1 cœur)	Node 2 (7 cœurs)	Node2 (14 cœurs)	Node2 (21 cœurs)	Node2+Node1 (28 cœurs)
6.2MB	03m :51s	40s	23s	17s	16s
52MB	30m :04s	5m :18s	3m :12s	02m :21s	1m :55s
90MB	01h :04m :35s	09m :53s	05m :56s	04m :21s	3m :35s
212MB	02h :34m :46s	23m :29s	14m :26s	10m :08s	08m :09s

Table 6-2-Temps d'exécution

La Table 6-2 illustre les temps d'exécution de l'opération de migration sur différentes configurations. On a varié la taille de fichiers d'entrée et le nombre d'unités de calcul. Comme prévu, en augmentant la taille de fichiers, la différence entre le temps d'exécution séquentielle et les temps d'exécution parallèle devient plus importante.

Elle atteint un maximum de 2 heures, 34 minutes et 46 secondes pour fichier de taille 212MB sur un seul cœur.

Après la parallélisation sur 28 cœurs, on a obtenu un temps d'exécution de 08 minutes et 09 secondes, ce qui représente une différence majeure de 2 heures, 26 minutes et 37 secondes. De plus, on a observé que pour de petites tailles d'entrée, des cœurs supplémentaires ont réduit aussi le temps d'exécution.

6.5.3. Estimation de La L'Accélération Théorique

Deux lois parmi les plus utilisées pour estimer l'accélération sont la loi d'Amdahl et la loi de Gustafson. Dans la partie suivante, nous présentons ces deux lois tout en justifiant notre choix.

- **Loi d'Amdahl :**

La loi d'Amdahl stipule que l'accélération globale d'un calcul parallélisé est limitée par la portion séquentielle du programme, qui ne peut pas être parallélisée. Cette idée est résumée par l'inégalité suivante :

$$Speedup \leq \frac{1}{(1 - P) + \frac{P}{n}}$$

Où P représente la fraction parallélisable du code, et n le nombre d'unités de calcul (ou processeurs).

- **La loi de Gustafson**

La loi de Gustafson est une extension de la loi d'Amdahl qui se concentre sur la scalabilité de la solution plutôt que sur des problèmes de taille fixe. Alors que la loi d'Amdahl suppose une taille de problème fixe, la loi de Gustafson suppose que la taille du problème peut augmenter avec le nombre de processeurs, permettant de paralléliser une partie potentiellement plus grande du programme à mesure que plus de processeurs sont ajoutés. Elle est donnée par la formule suivante.

$$Speedup = n - (1 - P) \times (n - 1)$$

Où n représente le nombre des unités de calcul et P est la fraction parallélisable du programme.

Étant donné que notre approche consiste à adapter dynamiquement la charge de travail en fonction du nombre de cœurs disponibles, le temps d'exécution diminue tout en maintenant une charge de traitement proportionnelle au nombre de cœurs. Autrement dit, plus on dispose de ressources, plus on peut traiter de données en parallèle, ce qui est en parfaite adéquation avec les hypothèses de la loi de Gustafson.

Contrairement à la loi d'Amdahl, qui considère un problème de taille fixe et met en évidence les limitations dues à la partie séquentielle du code, la loi de Gustafson permet de mieux modéliser notre cas, dans lequel le parallélisme est exploité pour traiter une plus grande quantité de données dans un temps maîtrisé. Elle reflète de manière plus réaliste le gain obtenu, en tenant compte du fait que l'on adapte le traitement au nombre de cœurs.

Pour déterminer le temps parallèle, dans notre script la mesure est effectuée à partir du moment où l'opération *sumigfd* est lancée sur les cœurs. Ce temps correspond uniquement à la phase de calcul distribuée. La mesure s'arrête une fois que tous les calculs parallèles sont terminés.

Les résultats de cette analyse sont présentés dans le tableau suivant :

Tailles de fichier	Node 2 (7 cœurs)	Node2 (14 cœurs)	Node2 (21 cœurs)	Node2+Node1 (28 cœurs)
6.2MB	39s	22s	15s	15s
52MB	5m :11s	3m :02s	02m :06s	1m :36s
90MB	09m :44s	05m :37s	03m :53s	3m :03s
212MB	22m :54s	13m :42s	09m :11s	06m :54s

Table 6-3 Résultat de Temps Parallèle

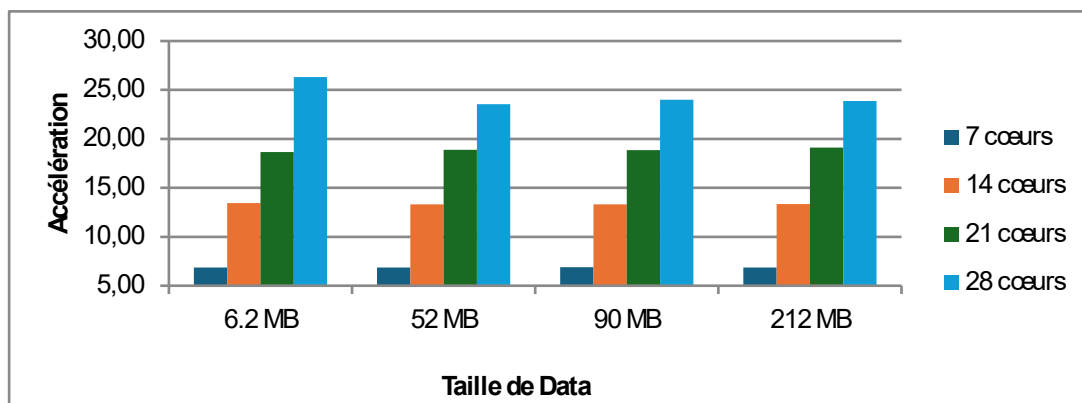


Figure 6-3 Résultat de Speed Up Théorique sur les différentes configurations

6.5.4. Speed-up (Accélération)

Tailles de fichier	Node 2 (7 cœurs)	Node2 (14 cœurs)	Node2 (21 cœurs)	Node2+Node1 (28 cœurs)
6.2MB	6,41	10,04	13,58	14,43
52MB	5,76	9,39	12,79	15,68
90MB	6,53	10,88	14,84	18,02
212MB	6,59	10,72	15,27	18,98

Table 6-4 les accélérations expérimentales aux accélérations

La figure 6-3 illustre les accélérations expérimentales (courbes pleines) aux accélérations théoriques (courbes pointillées) obtenues à l'aide de la loi de Gustafson, pour différentes tailles de fichiers d'entrée (6,2 MB, 52 MB, 90 MB, 212 MB) et divers nombres de cœurs (7, 14, 21 et 28).

On observe que pour des fichiers de petite taille, comme 6.2 MB et 52 MB, l'ajout de cœurs supplémentaires n'entraîne qu'une amélioration marginale de la performance. Par exemple, l'accélération passe de 6,41 à 14,43 pour les fichiers de 6.2 MB lorsque le nombre total de cœurs passe de 7 à 28, ce qui représente un gain limité. En revanche, lorsque la taille des fichiers augmente, l'effet de l'ajout de cœurs devient plus significatif. Pour un fichier de 90 MB, l'accélération passe de 6,53 à 18,02, et atteint même 18,98 pour un fichier de 212 MB. Ces résultats sont cohérents avec l'accélération théorique à la loi Gustafson, selon laquelle

l'impact du parallélisme est d'autant plus marqué que la part séquentielle du traitement est faible face à la charge totale. Ainsi, les performances sont nettement meilleures pour les entrées de grande taille, où le surcoût de la synchronisation entre les cœurs devient négligeable comparé au gain en parallélisme.

Ce comportement confirme l'intérêt du parallélisme massif pour les traitements de données volumineuses, alors qu'il reste peu avantageux, voire contre-productif, pour les traitements légers.

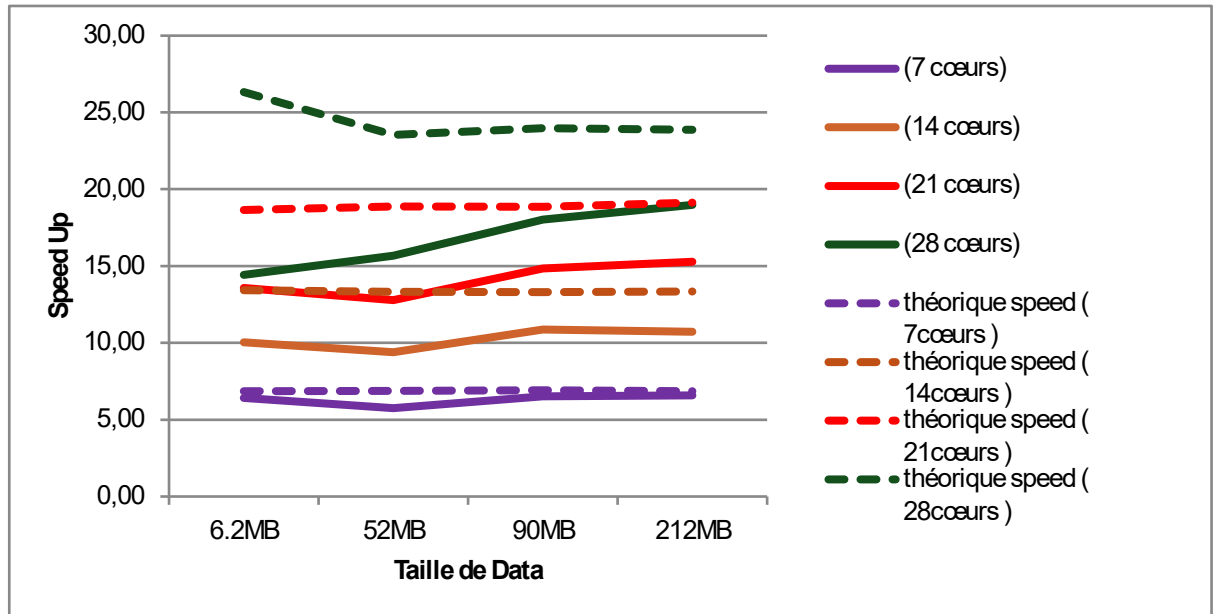


Figure 6-4 Le Speedup real vs Speedup théorique

6.5.5. Efficacité

Dans cette section, on vise à évaluer l'efficacité parallèle des différentes configurations afin d'identifier celle qui est la plus efficace.

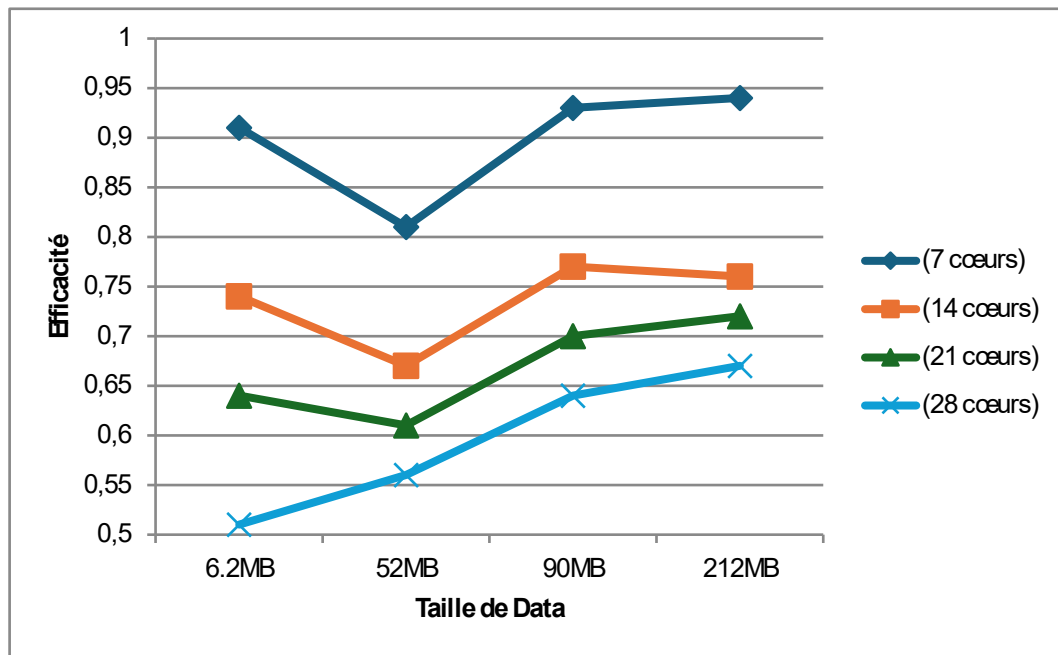


Figure 6-5 L'efficacité des différentes configurations

La figure 6-4 illustre l'évolution de l'efficacité du workflow pour différentes configurations de calcul, en fonction de la taille des données et du nombre de cœurs utilisés. On constate que les configurations à faible nombre de cœurs, notamment 7 cœurs sur une seule machine, présentent une efficacité supérieure par rapport aux configurations distribuées à plus grand nombre de cœurs.

Par exemple, pour un fichier de 212 MB, l'efficacité atteint 0,94 avec 7 cœurs, contre seulement 0,67 avec 28 cœurs. Cette tendance se vérifie également sur les autres tailles de fichier, avec une baisse progressive de l'efficacité lorsque le nombre de cœurs augmente.

Ce comportement peut s'expliquer par plusieurs facteurs :

- Dans les configurations à machine unique, la communication entre processus se fait via le bus mémoire, beaucoup plus rapide que la communication réseau utilisée dans les configurations distribuées.
- L'utilisation de plusieurs machines (dans le cas de 28 cœurs) peut introduire des déséquilibres de performance entre nœuds, rendant les plus lents un goulot d'étranglement pour l'ensemble du workflow.

Les résultats obtenus sont en accord avec les observations rapportées dans l'article « Speedup versus efficiency in parallèle systèmes », qui souligne que l'augmentation du nombre de processeurs améliore le speedup, mais entraîne souvent une diminution de l'efficacité. [39]

En résumé, plus on ajoute d'unités de calcul, plus il devient difficile de les exploiter efficacement, à moins d'augmenter proportionnellement la taille du problème. Ces résultats soulignent l'importance de trouver un équilibre entre parallélisme et efficacité, surtout dans des environnements distribués.

6.5.6. Test De HPL

HPL est un ensemble logiciel capable de résoudre un système linéaire dense aléatoire en double précision (64 bits) sur des machines à mémoire distribuée. Il constitue donc une implémentation portable et libre du **benchmark Linpack** pour le calcul haute performance (HPC). [40]

La figure 6-5 illustre la performance obtenue par l'exécution du benchmark HPL sur une architecture distribuée composée de deux nœuds totalisant 28 cœurs physiques. Le test consiste à résoudre un système linéaire dense de taille $80\,000 \times 80\,000$ en double précision, avec un schéma de distribution de données en blocs cycliques ($NB = 192$) et une topologie de processus MPI de type 2×14 .

Le résultat obtenu, soit 200.79 GFLOPS, met en évidence une accélération significative lorsque le calcul est réparti entre plusieurs cœurs. Afin d'évaluer la qualité de cette performance, il est pertinent de la comparer au **pic théorique du système**. Celui-ci peut être estimé selon l'équation suivante :

$$GFLOPS_{théorique} = Nombre\ de\ cœurs \times Fréquence\ (GHz) \times FLOP\ par\ cycle$$

En considérant 28 cœurs physiques fonctionnant à 2.10 GHz, avec une capacité de 4 opérations en virgule flottante par cycle (grâce aux instructions AVX avec FMA), le pic théorique est donné par :

$$28 \times 2.10 \times 4 = 235.2\ GFLOPS$$

Le résultat mesuré (200.79 GFLOPS) correspond donc à environ 85 % du pic théorique, ce qui témoigne d'une efficacité remarquable pour un benchmark en environnement distribué. Cela reflète un bon paramétrage du test HPL et une utilisation efficace des ressources matérielles disponibles.

Ainsi, cette exécution montre que, dans le contexte d'un problème de grande taille, l'augmentation du nombre de cœurs contribue effectivement à une accélération significative du calcul, en cohérence avec les prévisions théoriques. La performance de 200.79 GFLOPS reflète une excellente exploitation des ressources disponibles, confirmant la pertinence de HPL comme outil de référence pour l'évaluation des performances sur architectures parallèles à mémoire distribuée.

```

Open [icon] hpl_output.txt /mnt/beegfs
8 An explanation of the input/output parameters follows:
9 T/V : Wall time / encoded variant.
10 N : The order of the coefficient matrix A.
11 NB : The partitioning blocking factor.
12 P : The number of process rows.
13 Q : The number of process columns.
14 Time : Time in seconds to solve the linear system.
15 Gflops : Rate of execution for solving the linear system.
16
17 The following parameter values will be used:
18
19 N : 80000
20 NB : 192
21 PMAP : Row-major process mapping
22 P : 2
23 Q : 14
24 PFACT : Right
25 NBMIN : 4
26 NDIV : 2
27 RFACT : Crout
28 BCAST : lringM
29 DEPTH : 1
30 SWAP : Mix (threshold = 64)
31 LI : transposed form
32 U : transposed form
33 EQUIL : yes
34 ALIGN : 8 double precision words
35
36 -----
37
38 - The matrix A is randomly generated for each test.
39 - The following scaled residual check will be computed:
40 ||Ax-b||_oo / ( eps * ( ||x||_oo * ||A||_oo + ||b||_oo ) * N )
41 - The relative machine precision (eps) is taken to be 1.110223e-16
42 - Computational tests pass if scaled residuals are less than 16.0
43
44 =====
45 T/V N NB P Q Time Gflops
46 -----
47 WR11C2R4 80000 192 2 14 1700.01 2.0079e+02
48 HPL_pdgesv() start time Tue Jun 3 10:10:41 2025
49
50 HPL_pdgesv() end time Tue Jun 3 10:39:01 2025
51
52 =====
53 ||Ax-b||_oo/(eps*(||A||_oo*||x||_oo+||b||_oo)*N)= 2.25101492e-03 ..... PASSED
54 =====
55
56 Finished 1 tests with the following results:
57 1 tests completed and passed residual checks,
58 0 tests completed and failed residual checks,
59 0 tests skipped because of illegal input values.
60 -----

```

Figure 6-6 HPL Benchmark Résultat

6.6. Discussion

Les résultats précédents présentent les temps d'exécution du workflow de migration de données sur la plateforme développée. Des tests ont été réalisés sur quatre configurations matérielles :

- 7 cœurs sur une seule machine,
- 14 cœurs sur une seule machine,
- 21 cœurs répartis sur une seul machines,
- et 28 cœurs répartis sur deux machines,

En traitant plusieurs tailles de fichiers d'entrée (6.2 MB, 52 MB, 90 MB et 212 MB). À partir de ces mesures, nous avons calculé les SpeedUp et efficacités expérimentales obtenus dans chaque cas.

L'approche de parallélisation, basée sur une découpe automatique des fichiers et un traitement parallèle par cœur, a permis d'obtenir des accélérations importantes. Pour un fichier de 212 MB, le temps d'exécution est passé de 23 minutes 29 secondes avec 7 cœurs à 6 minutes 54 secondes avec 28 cœurs, soit un speed-up effectif de 3,4×. Selon la loi de Gustafson, ce gain correspond à une efficacité relativement élevée (jusqu'à 0,94 pour 7 cœurs, et 0,67 pour 28 cœurs), ce qui montre que le workflow est hautement parallélisable.

Cependant, plusieurs facteurs limitent l'efficacité maximale :

- Sur plusieurs machines, la latence réseau remplace les communications internes sur bus mémoire, ce qui augmente le coût global.
- L'hétérogénéité matérielle entre nœuds introduit des déséquilibres de traitement, où les nœuds les plus lents deviennent des goulots d'étranglement.
- Le workflow ne prend pas en compte la taille réelle des fichiers pour adapter dynamiquement le nombre de cœurs utilisés, ce qui entraîne parfois une sous-utilisation ou une surcharge sur les petits fichiers.

Conclusion Générale

À l'ère actuelle, la croissance rapide des volumes de données scientifiques, combinée à la complexité croissante des traitements numériques, a fait du calcul haute performance (HPC) une nécessité incontournable pour les entreprises et institutions de recherche. Dans ce contexte, SONATRACH, acteur majeur dans le domaine de l'exploration et de la production pétrolière en Algérie, fait face à un besoin critique : disposer d'une plateforme HPC moderne, flexible et performante, capable de répondre aux exigences de traitement sismique intensif.

Ce mémoire s'inscrit dans cette dynamique en proposant une solution complète, open source, et intégrée, alliant puissance de calcul, automatisation, supervision, et intelligence artificielle. En commençant par une analyse détaillée des besoins réels liés au traitement sismique, nous avons pu identifier les contraintes spécifiques en matière de performances, de disponibilité, de gestion des ressources et d'accessibilité utilisateur.

Nous avons ensuite défini les fondements théoriques du HPC et des systèmes de gestion de ressources comme SLURM, tout en intégrant des concepts modernes tels que la virtualisation légère, le stockage distribué, la supervision centralisée, et l'interface web multiaccès. Sur cette base, nous avons conçu une architecture modulaire à trois couches — **utilisateur, middleware, et exécution** — garantissant la scalabilité, la flexibilité et la résilience de la plateforme.

Notre infrastructure repose sur un socle exclusivement open source, incluant des outils comme Ansible pour l'automatisation, Prometheus et Grafana pour la surveillance, BeeGFS pour le stockage parallèle, Open OnDemand pour l'accès utilisateur via navigateur, et surtout SLURM pour l'orchestration des jobs. Cette approche assure une maîtrise complète du système, tout en réduisant les coûts de licence et en favorisant la transparence et l'évolution technologique.

Une innovation majeure apportée par notre travail est l'intégration d'un module d'intelligence artificielle capable de prédire dynamiquement les ressources optimales à allouer pour chaque job SLURM. À l'aide de modèles tels que XGBoost, Random Forest et SVM, entraînés sur des logs d'exécution simulés et réalistes, nous avons démontré qu'il est possible d'améliorer considérablement l'efficacité du cluster en ajustant automatiquement les paramètres (temps, CPU) avant la soumission. Cette intelligence est encapsulée dans une interface Streamlit interactive, permettant une utilisation simple, intuitive et pédagogique.

Parallèlement, une interface graphique dédiée à Seismic Linux, développée en HTML, CSS et JavaScript, a été mise en place pour rendre les outils sismiques en ligne de commande accessibles aux utilisateurs non experts. Cela permet de démocratiser l'usage du HPC pour les métiers géo scientifiques, tout en respectant les standards industriels en matière de performance et de robustesse.

Les tests réalisés, en particulier sur des cas d'usage réalistes, ont permis de valider l'ensemble de la chaîne. Par exemple, le module IA a su recommander des configurations CPU optimales avec un bon niveau de précision, tandis que l'interface graphique a simplifié la gestion des traitements. La plateforme a montré sa capacité à s'adapter à différents types de traitements sismiques, confirmant ainsi sa pertinence dans un environnement réel. Pour aller plus loin, plusieurs perspectives sont envisageables :

- Déployer la solution sur une infrastructure physique de production avec des ressources dédiées
- Intégrer des modèles d'apprentissage plus avancés (deep learning) pour la prédiction de performance ou d'énergie
- Étendre la plateforme pour prendre en charge des workflow complexes multi-étapes typiques du traitement sismique
- Mettre en place une base centralisée de partage des logs SLURM, afin d'améliorer l'apprentissage continu du système
- Ajouter un module de visualisation interactive des résultats géophysiques.

En conclusion, cette plateforme incarne une vision moderne du calcul scientifique, où la puissance du HPC rencontre l'ergonomie, l'intelligence et la souplesse de l'open source. Elle constitue une réponse concrète aux besoins stratégiques de traitement sismique dans l'industrie énergétique algérienne, tout en posant les bases d'une infrastructure évolutive, intelligente, et durable.

Bibliographie

- [1] I. N. SONATRACH, «Historique et évolution,» 13 ouat 2012. [En ligne]. Available: <https://www.institut-numerique.org/2-historique-et-evolution-5028f58cb09d3>.
- [2] C. Verney, «Modélisation du transport sédimentaire en zone côtière : application à l'estuaire de la Seine,» École Nationale des Ponts et Chaussées, 2005. [En ligne]. Available: https://pastel.hal.science/pastel-00005861/file/These_Verney.pdf.
- [3] R. Abdelkhalek, «Convey FPGA platform evaluation for seismic modeling and migra- tion. Total, 2009. 97,» Rapport technique, Total, 2009.
- [4] P. Pourroy, « Infrastructure matérielle et logicielle pour l'étude de la performance des simulations numériques HPC en interaction...,» Thèse de doctorat, Université de Strasbourg, 2021.
- [5] V. Eijkhout, Introduction to High Performance Scientific Computing,» Raleigh, NC, 2013.
- [6] «Le métacomputing au service du calcul haute performance,» ResearchGate, 2021.
- [7] «What is high-performance computing (HPC),» 2023. [En ligne]. Available: <https://cloud.google.com/discover/what-is-high-performance-computing?hl=fr>.
- [8] IBM, «What Is High-Performance Computing (HPC)?,» 2024. [En ligne]. Available: <https://www.ibm.com/think/topics/hpc>.
- [9] N. Grenèche, «Élasticité des infrastructures HPC conteneurisées : Résonance entre le HPC et le Cloud,» Université Sorbonne Paris Nord, 2023.
- [10] HPE, «What is High Performance Computing (HPC)?,» [En ligne]. Available: https://www.hpe.com/emea_africa/fr/what-is/high-performance-computing.html.
- [11] Inria, «L'essentiel sur : le calcul haute performance,» 5 avril 2025. [En ligne]. Available: <https://www.inria.fr/fr/essentiel-calcul-haute-performance-hpc>.
- [12] C. Bouillaguet, Introduction au calcul haute performance – Cours 1 : Notions de base, Laboratoire d'informatique de Paris 6 (LIP6), Sorbonne Université, Paris, France, 27 Jan. 2020.
- [13] Rescale, ««Le stockage HPC expliqué | Architecture HPC | Redimensionner,» [En ligne]. Available: <https://rescale.com/fr/hpc-storage/>.
- [14] Ö. Yilmaz, Seismic Data Analysis, 2001.

- [15] M. Laurencin, Etude de la géométrie, de la nature et des déformations de la zone de subduction des Petites Antilles du Nord, 2017.
- [16] B. Brahmi and M. Bougara, Atténuation du ground roll par le filtre surface wave atténuation: application pour le cas des données sismiques, Universidad Nacional de Colombia: Boletín de Ciencias de la Tierra, Jul–Dec. 2022.
- [17] T. GmbH, «BeeGFS Documentation 8.0.0,» 2025. [En ligne]. Available: <https://doc.beegfs.io/latest/overview/overview.html>.
- [18] R. Hat, «Red Hat Enterprise Linux 9 – Enregistrement et installation à partir du CDN à l'aide de l'interface graphique,» 2025. [En ligne]. Available: https://docs.redhat.com/fr/documentation/red_hat_enterprise_linux/9/html/performing_a_standard_rhel_9_installation/register-and-install-from-cdn-gui_assembly_installing-on-amd64-intel-64-and-64-bit-arm.
- [19] A. O. Foundation, «Guide de migration vers AlmaLinux,» 11 avril 2025. [En ligne]. Available: <https://wiki.almalinux.org/documentation/migration-guide.html>.
- [20] A. Ancel, «SLURM : Simple Linux Utility for Resource Management,» 15 mars 2016. [En ligne]. Available: <https://indico.math.cnrs.fr/event/940/contributions/2999/attachments/2120/2340/slurm.pdf>.
- [21] S. LLC, «Slurm Workload Manager – Documentation officielle (version 24.11),» 22 août 2024. [En ligne]. Available: [slurm.schedmd.com](https://slurm.schedmd.com/documentation.html)
<https://slurm.schedmd.com/documentation.html>.
- [22] M. A. Jette et T. Wickberg, «Architecture of the Slurm Workload Manager,» 19 mai 2023. [En ligne]. Available: https://jsspp.org/papers23/JSSPP_2023_keynote_SLURM.pdf.
- [23] NetApp, «Présentation de l'architecture – Services de système de fichiers BeeGFS,» 23 août 2024. [En ligne]. Available: <https://docs.netapp.com/fr-fr/beegfs/second-gen/beegfs-architecture-overview.html#services-de-système-de-fichiers>.
- [24] O. S. Center, «Open OnDemand – Documentation officielle,» 2025. [En ligne]. Available: <https://openondemand.org>.
- [25] S. LLC, «Ansible Documentation officielle,» [En ligne]. Available: <https://docs.ansible.com>.
- [26] G. Labs, «Documentation officielle de Grafana (version 12.0),» 2024. [En ligne]. Available: <https://grafana.com/docs/grafana/latest/>.
- [27] B. Mahesh, «Machine Learning Algorithms - A Review,» 2020.

- [28] A. Géron, «Machine Learning avec Scikit-Learn,» 2017.
- [29] C. E. e. .. P. Louridas, «Machine learning,» 2016, p. 110–115..
- [30] V. Martineau, «Utilisation de automatique en remplacement des simulateurs de débitage de billots de bois,» 2022.
- [31] F. Chollet, «Deep Learning with Python FRANÇOIS CHOLLET,» 2017.
- [32] C.-A. Azencott, « Introduction au Machine Learning,» 2018. [En ligne]. Available: cazencott.info.
- [33] L. F. J. O. R. S. C. Breiman, «Classification and Regression Trees,» Wadsworth, New York, 1984.
- [34] Streamlit, « Documentation officielle de Streamlit,» S.D. [En ligne]. Available: docs.streamlit.io.
- [35] «What Is Link Aggregation Control Protocol (LACP)?,» Huawei, November 2024. [En ligne]. Available: <https://info.support.huawei.com/info-finder/encyclopedia/en/LACP.html>.
- [36] Ananth Grama, Anshul Gupta, George Karypis, Vipin Kumar, Introduction to Parallel Computing, Second Edition.
- [37] Georg Hager, Gerhard Wellein, Introduction to High Performance Computing for Scientists and Engineers.
- [38] J. J. Dongarra, C. B. Moler, J. R. Bunch, and G. W. Stewart, LINPACK Users' Guide, Society for Industrial and Applied Mathematics (SIAM), 1979.
- [39] Derek L Eager, John Zahorjan, and Edward D Lazowska, Speedup versus efficiency in parallel systems., IEEE transactions on computers, 1989.
- [40] A. Petitet, R. C. Whaley, J. Dongarra, and A. Cleary, «HPL - A Portable Implementation of the High-Performance Linpack Benchmark for Distributed-Memory Computers,» Dec. 2, 2018. [En ligne]. Available: <https://www.netlib.org/benchmark/hpl/>.

Annexe :

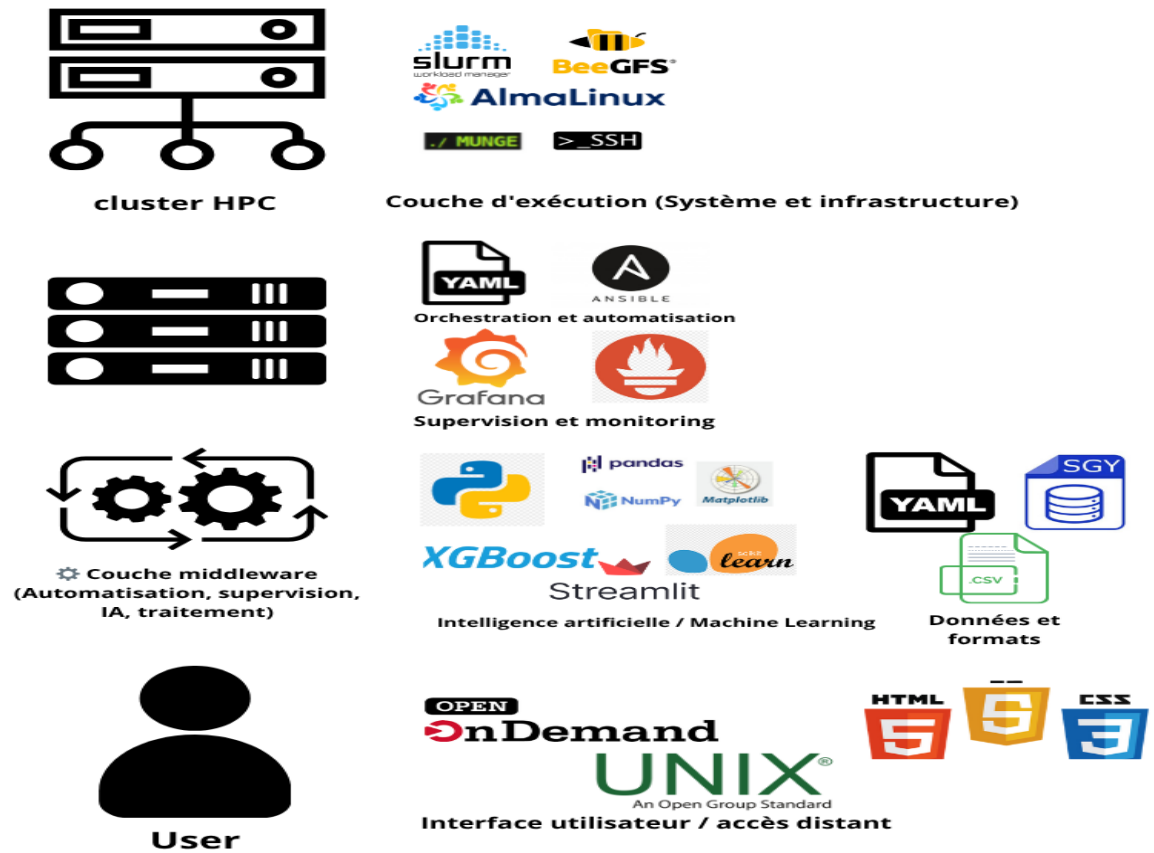


Figure Annex La Pile Logicielle

Cluster Configuration

```
ouss@master:~$ ansible -i /etc/ansible/hosts all -m ping
master | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
node2 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
node1 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
[ouss@master ~]$
```

Ansible Connectivity

```
[ouss@master ~]$ sudo systemctl status beegfs-mgmt
[ouss@master ~]$ sudo systemctl status beegfs-mgmt
[sudo] password for ouss:
● beegfs-mgmt.service - BeeGFS Management Server
   Loaded: loaded (/usr/lib/systemd/system/beegfs-mgmt.service; enabled; pre
   Active: active (running) since Mon 2025-04-28 14:42:17 CEST; 28min ago
     Docs: https://doc.beegfs.io
   Main PID: 45195 (beegfs-mgmt)
      Tasks: 27 (limit: 1646744)
     Memory: 26.3M
        CPU: 678ms
    CGroup: /system.slice/beegfs-mgmt.service
            └─45195 /opt/beegfs/sbin/beegfs-mgmt --log-target=journald

Apr 28 14:42:17 master systemd[1]: Starting BeeGFS Management Server...
```

```
[ouss@master ~]$ sudo systemctl status beegfs-client
● beegfs-client.service - Start BeeGFS Client
   Loaded: loaded (/usr/lib/systemd/system/beegfs-client.service; enabled; preset: disabled)
   Active: active (exited) since Mon 2025-04-28 14:51:16 CEST; 20min ago
     Process: 45521 ExecStart=/etc/init.d/beegfs-client start (code=exited, status=0/SUCCESS)
   Main PID: 45521 (code=exited, status=0/SUCCESS)
      CPU: 171ms
```

```
● beegfs-meta.service - BeeGFS Metadata Server
   Loaded: loaded (/usr/lib/systemd/system/beegfs-meta.service; enabled; preset: disabled)
   Active: active (running) since Mon 2025-04-28 14:45:37 CEST; 25min ago
     Docs: http://www.beegfs.com/content/documentation/
   Main PID: 45318 (beegfs-meta/Mai)
      Tasks: 155 (limit: 1646744)
     Memory: 366.6M
        CPU: 3.362s
    CGroup: /system.slice/beegfs-meta.service
            └─45318 /opt/beegfs/sbin/beegfs-meta cfgFile=/etc/beegfs/beegfs-meta.conf runDaemonized=false
```

BeeGFS Management, Metadata et client Server in Master Node

```
[ouss@node1 ~]$ sudo systemctl status beegfs-client
● beegfs-client.service - Start BeeGFS Client
   Loaded: loaded (/usr/lib/systemd/system/beegfs-client.service; enabled; preset: disabled)
   Active: active (exited) since Mon 2025-04-28 14:50:37 CEST; 21min ago
     Process: 93348 ExecStart=/etc/init.d/beegfs-client start (code=exited, status=0/SUCCESS)
   Main PID: 93348 (code=exited, status=0/SUCCESS)
      CPU: 173ms
```

```
[ouss@node1 ~]$ sudo systemctl status beegfs-storage
● beegfs-storage.service - BeeGFS Storage Server
   Loaded: loaded (/usr/lib/systemd/system/beegfs-storage.service; enabled; preset: disabled)
   Active: active (running) since Mon 2025-04-28 14:50:03 CEST; 22min ago
     Docs: http://www.beegfs.com/content/documentation/
   Main PID: 93303 (beegfs-storage/)
      Tasks: 21 (limit: 408115)
     Memory: 51.7M
        CPU: 151ms
    CGroup: /system.slice/beegfs-storage.service
            └─93303 /opt/beegfs/sbin/beegfs-storage cfgFile=/etc/beegfs/beegfs-storage.conf runDaemonized=false
```

BeeGFS Stockage et client Server in Node1(Meme Pour Node2)

```
[ouss@master ~]$ sudo systemctl status slurmdbd
● slurmdbd.service - Slurm DBD accounting daemon
   Loaded: loaded (/usr/lib/systemd/system/slurmdbd.service; enabled; preset: disabled)
   Active: active (running) since Thu 2025-05-29 16:44:38 CEST; 4 days ago
   Main PID: 1597 (slurmdbd)
      Tasks: 6
     Memory: 4.0M
        CPU: 2.332s
    CGroup: /system.slice/slurmdbd.service
            └─1597 /usr/sbin/slurmdbd -D -s
```

Slurm Data Base

```
[oussa@master ~]$ sudo systemctl status slurmctld
● slurmctld.service - Slurm controller daemon
   Loaded: loaded (/usr/lib/systemd/system/slurmctld.service; enabled; preset: disabled)
   Active: active (running) since Mon 2025-06-02 12:03:59 CEST; 21h ago
     Main PID: 56680 (slurmctld)
        Tasks: 17
       Memory: 6.6M
          CPU: 28.185s
      CGroup: /system.slice/slurmctld.service
              └─56680 /usr/sbin/slurmctld -D -s
                 56685 "slurmctld: slurmscriptd"
```

```
[oussa@master ~]$ sudo systemctl status munge
● munge.service - MUNGE authentication service
   Loaded: loaded (/usr/lib/systemd/system/munge.service; enabled; preset: disabled)
   Active: active (running) since Thu 2025-05-29 16:44:24 CEST; 4 days ago
     Docs: man:munged(8)
   Process: 1368 ExecStart=/usr/sbin/munged (code=exited, status=0/SUCCESS)
     Main PID: 1384 (munged)
        Tasks: 4 (limit: 1646758)
       Memory: 2.2M
          CPU: 2.258s
      CGroup: /system.slice/munge.service
              └─1384 /usr/sbin/munged
```

Slurm Controller Daemon et Munge Dans Master Node

```
[oussa@node1 ~]$ sudo systemctl status slurmd
● slurmd.service - Slurm node daemon
   Loaded: loaded (/usr/lib/systemd/system/slurmd.service; enabled; preset: disabled)
   Active: active (running) since Mon 2025-06-02 12:03:21 CEST; 21h ago
     Main PID: 18490 (slurmd)
        Tasks: 1
       Memory: 1.3M
          CPU: 606ms
      CGroup: /system.slice/slurmd.service
              └─18490 /usr/sbin/slurmd -D -s
```

```
[oussa@node1 ~]$ sudo systemctl status munge
● munge.service - MUNGE authentication service
   Loaded: loaded (/usr/lib/systemd/system/munge.service; enabled; preset: disabled)
   Active: active (running) since Thu 2025-05-29 16:43:28 CEST; 4 days ago
     Docs: man:munged(8)
   Process: 1049 ExecStart=/usr/sbin/munged (code=exited, status=0/SUCCESS)
     Main PID: 1067 (munged)
        Tasks: 4 (limit: 408124)
       Memory: 2.2M
          CPU: 1.357s
      CGroup: /system.slice/munge.service
              └─1067 /usr/sbin/munged
```

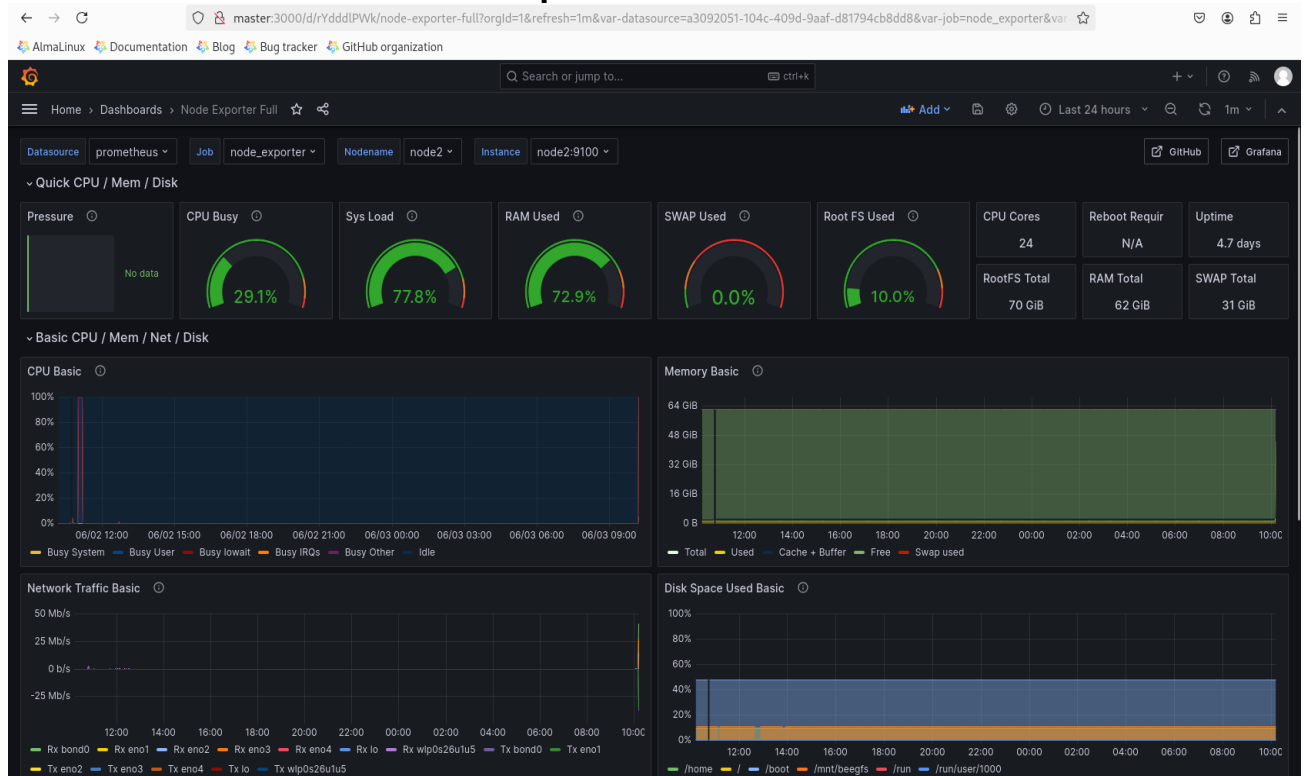
Slurm Daemon et Munge Dans Node1 (Comme Node2)

```
[oussa@master ~]$ scontrol show nodes
NodeName=node1 Arch=x86_64 CoresPerSocket=1
  CPUAlloc=0 CPUEffctv=4 CPUTot=4 CPULoad=0.01
AvailableFeatures=(null)
ActiveFeatures=(null)
  Gres=(null)
NodeAddr=node1 NodeHostName=node1 Version=22.05.9
OS=Linux 5.14.0-503.40.1.el9_5.x86_64 #1 SMP PREEMPT_DYNAMIC Mon May 5 06:06:04 EDT 2025
RealMemory=62000 AllocMem=0 FreeMem=60956 Sockets=2 Boards=1
State=IDLE ThreadsPerCore=2 TmpDisk=0 Weight=1 Owner=N/A MCS_label=N/A
Partitions=debug
BootTime=2025-05-29T16:43:06 SlurmdStartTime=2025-06-02T12:03:21
LastBusyTime=2025-06-02T12:30:29
  CfgTRES=cpu=4,mem=62000M,billing=4
  AllocTRES=
  CapWatts=n/a
CurrentWatts=0 AveWatts=0
ExtSensorsJoules=n/s ExtSensorsWatts=0 ExtSensorsTemp=n/s

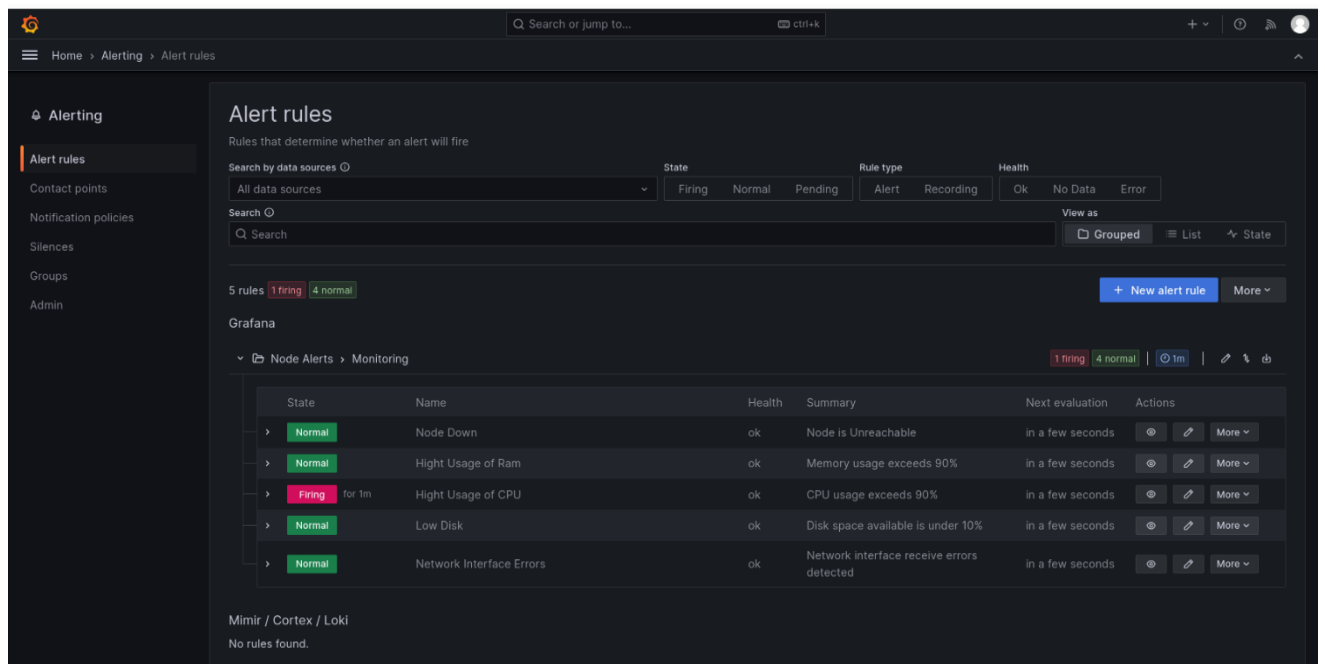
NodeName=node2 Arch=x86_64 CoresPerSocket=6
  CPUAlloc=0 CPUEffctv=24 CPUTot=24 CPULoad=0.00
AvailableFeatures=(null)
ActiveFeatures=(null)
  Gres=(null)
NodeAddr=node2 NodeHostName=node2 Version=22.05.9
OS=Linux 5.14.0-503.40.1.el9_5.x86_64 #1 SMP PREEMPT_DYNAMIC Mon May 5 06:06:04 EDT 2025
RealMemory=62000 AllocMem=0 FreeMem=61156 Sockets=2 Boards=1
State=IDLE ThreadsPerCore=2 TmpDisk=0 Weight=1 Owner=N/A MCS_label=N/A
Partitions=debug
BootTime=2025-05-29T16:44:37 SlurmdStartTime=2025-06-02T09:45:23
LastBusyTime=2025-06-02T12:03:59
  CfgTRES=cpu=24,mem=62000M,billing=24
  AllocTRES=
  CapWatts=n/a
CurrentWatts=0 AveWatts=0
ExtSensorsJoules=n/s ExtSensorsWatts=0 ExtSensorsTemp=n/s
```

Nodes Information

Supervision



Grafana Interface



Prometheus Alertes

Partie Utilisateur



Ressources Libre Dans Open OnDemand

Open OnDemand Jobs Clusters Interactive Apps My Interactive Sessions Help Logged in as ouss Log Out

All Jobs All Clusters

Active Jobs

Show 50 entries Filter:

ID	Name	User	Account	Time Used	Queue	Status	Cluster	Actions
130	hpl	ouss	default	00:00:09	debug	Completed	cluster	
131	hpl	ouss	default	00:00:34	debug	Running	cluster	

Showing 1 to 2 of 2 entries Previous 1 Next

Active Jobs

HPC Advisor : Prédicteur CPU & Temps d'exécution

Cette application prédit les ressources nécessaires à partir de votre configuration de job.

Paramètres du job

Nom du job: migration

Partition: cpu

Taille des données (MB): 500

Lancer la prédiction

CPU recommandé : 9

Temps estimé : 4.28 minutes

2 Nettoyer les données

Nettoyer et transformer

3 Entraîner les modèles

Entraîner IA

Modèles entraînés avec XGBoost.

MAE (temps) : 0.39 minutes
R² (temps) : 0.33
MAE (CPU) : 1.33
R² (CPU) : 0.91

4 Prédiction

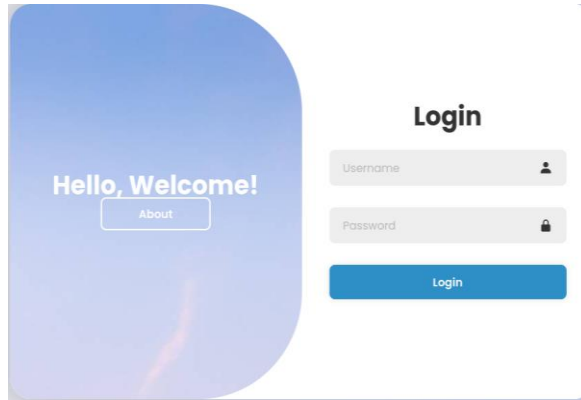
Nom du job: test_job

Partition: debug

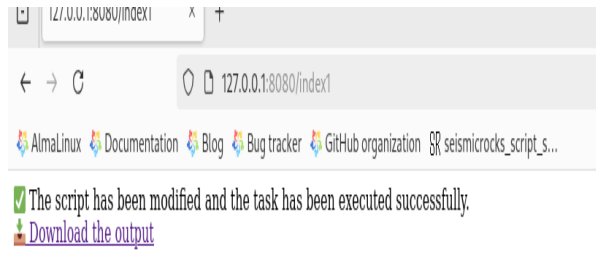
Taille des données (Go): 1.5

Ratio CPU/Mémoire (approximatif): 1 100

IA Model



Login Page



Downloade Page

Upload Files & Fill Parameters

Segy file :

Upload File:

Browse... No file selected.

fichier de modèle de vitesse :

Upload File:

Browse... No file selected.

Select Node:

Choose a node ▾

Select CPU:

Choose CPU count ▾

Intervalle d'échantillonnage dans le temps (dt):

Ex: 0.004

Pas d'échantillonnage spatial horizontal (dx):

Ex: 25

Pas d'échantillonnage spatial vertical (dz):

Ex: 25

Select Node:

Choose a node ▾

Select CPU:

Choose CPU count ▾

Intervalle d'échantillonnage dans le temps (dt):

Ex: 0.004

Pas d'échantillonnage spatial horizontal (dx):

Ex: 25

Pas d'échantillonnage spatial vertical (dz):

Ex: 25

Nombre de points en profondeur (nz):

Ex: 1000

Submit

Form Page

Résumé

Ce mémoire porte sur la conception, le déploiement et l'optimisation d'une plateforme de calcul haute performance (HPC) destinée au traitement sismique, en réponse aux besoins croissants en puissance de calcul dans les domaines géophysiques et industriels. Dans le contexte des systèmes HPC open source, ce travail s'est intéressé aux défis liés à l'automatisation, à la gestion des ressources, à l'accessibilité des outils de traitement, ainsi qu'à l'intégration de l'intelligence artificielle pour la prédiction des performances.

Pour répondre à ces problématiques, une architecture complète a été conçue et mise en œuvre en s'appuyant sur des technologies open source telles que SLURM, BeeGFS, Prometheus, Grafana, Open OnDemand et Ansible. En complément, une interface intelligente, développée avec Streamlit et intégrant des modèles d'apprentissage automatique (XGBoost, Random Forest, SVM), permet d'anticiper les ressources optimales à allouer pour chaque job. Une interface web dédiée a également été réalisée afin de rendre les outils de traitement sismique plus accessibles aux utilisateurs non techniques.

Les résultats obtenus démontrent une amélioration notable en termes de performance, d'efficacité de gestion, et d'expérience utilisateur. Ce travail apporte une contribution concrète à la mise en place d'un environnement HPC intelligent, automatisé et ouvert, adapté aux besoins industriels de SONATRACH et potentiellement transposable à d'autres domaines à forte intensité de calcul.

Mots clés : HPC, traitement sismique, SLURM, open source, Streamlit, Ansible, intelligence artificielle, prédiction, cluster, SONATRACH.